Sicurezza
Sapienza Università di Roma
A.A 2024-2025

Parte del corso di Sicurezza del Prof. Visconti (1/2)

Indice

1 Introduzione **6**
- 1.1 Introduzione ai computer quantistici . . . 6
- 1.2 Cifratura dei dati . . . 6
  - 1.2.1 Esempio: cifrario di Cesare . . . 7
  - 1.2.2 Quando possiamo dire che un sistema di cifratura simmetrico è sicuro? . . . 7
  - 1.2.3 One-Time Pad (1917): Vernon-Shannon . . . 8

2 Threat model **10**
- 2.1 Threat Model per la cifratura dei dati . . . 10
  - 2.1.1 Pseudorandom generator (PRG) . . . 10
  - 2.1.2 Pseudorandom Functions . . . 11
  - 2.1.3 CPA-Secure Encryption . . . 12
  - 2.1.4 CPA-Secure Encryption: messaggi lunghi . . . 12

3 Block ciphers **14**
  - 3.0.1 Esercizio . . . 14
- 3.1 Electronic Code Book (ECB) . . . 14
- 3.2 CBC mode . . . 15
- 3.3 Counter (CTR) mode . . . 16
- 3.4 Stream cipher vs block cipher . . . 17

4 Integrità dei messaggi **17**
- 4.1 MAC . . . 17
  - 4.1.1 Funzione di verifica . . . 18
  - 4.1.2 Schema del MAC . . . 18
  - 4.1.3 Attacco per schema a lunghezza fissa . . . 19
  - 4.1.4 Conclusioni . . . 20

5 Authenticated Encryption **20**
- 5.1 Authenticated Encryption with Associated Data (AEAD) . . . 21

6 Collision-Resistant Hash Functions **21**
- 6.1 Uso nelle tabelle hash . . . 21
  - 6.1.1 Il paradosso del compleanno . . . 22
- 6.2 Sicurezza di SHA-256 . . . 22
- 6.3 Applicazioni delle CRHF . . . 23
  - 6.3.1 MAC a taglia costante da un MAC lineare . . . 23
  - 6.3.2 Hash come random oracle . . . 23
  - 6.3.3 Fingerprinting di file (e virus) e deduplicazione . . . 23

6.3.4 Uso di CRHF per MAC . . . 23
6.3.5 HMAC . . . 24
6.3.6 Utilizzo delle CRHF per l'autenticazione di password: . . . 24
6.3.7 Cifrare i file: PBKDF2 . . . 25
6.3.8 Puzzle crittografico parametrizzato . . . 26

7 OpenSSL **27**
7.1 Randomicità . . . 27
7.1.1 Rand . . . 27
7.2 Cifratura . . . 28
7.2.1 AES-256-CTR . . . 28
7.2.2 Decifratura . . . 29
7.2.3 AES-256-CBC . . . 29
7.2.4 AES-256-ECB (insicuro) . . . 29
7.2.5 Test di velocità del cipher . . . 30
7.2.6 Lunghezza chiavi e IV . . . 30
7.3 OpenSSL-MAC . . . 30
7.3.1 MAC e HASH . . . 30
7.3.2 Generazione di chiavi a partire da password - PBKDF2 . . . 31

8 Crittografia asimmetrica **31**
8.1 Limiti della crittografia simmetrica . . . 31
8.1.1 Merkle puzzle: distribuzione della chiave in modo sicuro senza incontro fisico 31
8.2 Nozioni di base . . . 32
8.3 Discrete Logarithm (DLog) Problem . . . 34
8.4 L'avvento delle curve ellittiche (ECC) . . . 34
8.5 Tabelle riepilogative . . . 36
8.5.1 CCA-secure vs CPA-secure . . . 36
8.5.2 Tabella confronto Block Ciphers . . . 37
8.6 Diffie-Hellman . . . 38
8.6.1 Spiegazione Assunzione Computazionale DH . . . 38
8.6.2 Scambio di chiavi . . . 38
8.6.3 Errore: chiave casuale . . . 40
8.6.4 Decisional Diffie-Hellman Function (DDH) . . . 40

9 Crittografia a chiave pubblica **41**
9.1 DH e RSA . . . 41
9.1.1 Diffie-Hellman e Man-in-the-Middle Attack . . . 41
9.1.2 DH e notazione di Public Key Cryptography . . . 42
9.1.3 Alba e caduta di RSA (1978) . . . 42
9.1.4 Omomorfismo . . . 43
9.1.5 Cifratura ibrida . . . 43
9.2 Firma digitale . . . 43
9.2.1 CRHFs: di nuovo in azione . . . 44
9.2.2 Schnorr's Signature Scheme . . . 44
9.2.3 Continuo dell'attacco attivo in DH . . . 44
9.2.4 In molti scenari serve una PKI . . . 45
9.2.5 Certificato digitale . . . 45
9.2.6 CA: Autorità di Certificazione . . . 45
9.2.7 X509v3 . . . 47
9.3 TLS: Transport Layer Security . . . 48
9.3.1 Accordo di chiavi effimero . . . 49
9.4 Accordo di chiavi . . . 49
9.4.1 TLS 1.2 e 1.3 . . . 50
9.4.2 Esempio di attacco Man-in-the-Middle e controllo di massa . . . 51
9.4.3 Firme digitali standard . . . 53

9.5 OpenSSL – Da terminale . . . 53
9.5.1 DH Key Exchange . . . 53
9.5.2 DSA KEYs and signatures . . . 56
9.5.3 OpenSSL – ECDSA KEYs and signatures . . . 59
9.5.4 RSA - Firme e cifratura asimmetrica . . . 60
9.5.5 OpenSSL – RSA cifratura ibrida . . . 62
9.5.6 OpenSSL – RSA Decifratura Ibrida . . . 63
9.5.7 Esercizio: note aggiuntive . . . 63
9.5.8 OpenSSL – Certificate request . . . 64
9.5.9 CA - playground . . . 65
9.5.10 OpenSSL – Rilascio di un certificato . . . 66
9.5.11 Un web server con HTTPS . . . 66
9.5.12 Sicurezza e integrità dei Messaggi . . . 68

10 Timing Attack e Side Channel Attacks **69**
10.1 Introduzione ai Side Channel Attacks . . . 69
10.2 Timing attacks . . . 69
10.2.1 Esempio storico: l'attacco alla Xbox . . . 69
10.2.2 Vulnerabilità nelle implementazioni di MAC . . . 69
10.2.3 Analisi dettagliata della vulnerabilità . . . 70
10.2.4 Implementazione sicura a tempo costante . . . 70
10.3 Dimostrazione pratica di un Timing Attack . . . 71
10.3.1 Setup dell'esperimento . . . 71
10.3.2 Spiegazione del codice . . . 72
10.3.3 Principio dell'attacco . . . 72
10.3.4 Risultati dell'esperimento . . . 73
10.4 Contromisure contro i Timing Attack . . . 73
10.5 Considerazioni generali sui Side Channel Attacks . . . 73

11 Anonimato su Internet: Onion Routing **74**
11.1 Il problema dell'anonimato nelle connessioni . . . 74
11.2 Onion Routing come soluzione . . . 74
11.2.1 Principio di base . . . 74
11.3 Funzionamento dettagliato dell'Onion Routing . . . 74
11.3.1 Fase di preparazione e invio del messaggio . . . 74
11.3.2 Fase di inoltro attraverso i server . . . 75
11.4 Gestione del percorso di ritorno . . . 75
11.5 TOR: The Onion Router . . . 75
11.6 Considerazioni sull'uso dell'Onion Routing . . . 76
11.6.1 Vantaggi . . . 76
11.6.2 Limitazioni . . . 76

12 Esempi di domande d'esame **77**
12.1 Domanda 1: Pseudorandom Function . . . 77
12.1.1 Testo della domanda . . . 77
12.1.2 Analisi . . . 77
12.1.3 Risposta corretta . . . 77
12.2 Domanda 2: Chosen-Plaintext Attack (CPA) . . . 78
12.2.1 Testo della domanda . . . 78
12.2.2 Analisi . . . 78
12.2.3 Risposta corretta . . . 78
12.3 Domanda 3: Collision Resistance . . . 78
12.3.1 Testo della domanda . . . 78
12.3.2 Analisi . . . 78
12.3.3 Risposta corretta . . . 78
12.4 Domanda 4: Certificati HTTPS . . . 79

12.4.1 Testo della domanda: 79
12.4.2 Analisi 79
12.4.3 Risposta corretta 79
12.5 Domanda 5: Comandi OpenSSL 79
12.5.1 Testo della domanda: 79
12.5.2 Analisi 79
12.5.3 Risposta corretta 79

Disclaimer

Questi appunti potrebbero contenere imprecisioni o errori. Verificare sempre con il materiale ufficiale del corso. L'autore non si assume responsabilità per eventuali problematiche.

1 Introduzione

1.1 Introduzione ai computer quantistici

La **potenza dei computer quantistici** non risiede nella capacità di eseguire un numero elevato di iterazioni (ad esempio raggiungendo 2^{100} in un ciclo for, irraggiungibile da nessun computer al mondo), ma nella **possibilità di implementare algoritmi** che sfruttano fenomeni quantistici come la **sovrapposizione** (consente a un qubit di esistere simultaneamente in più stati (0 e 1) fino a quando non viene misurato) e l'**entanglement** (fenomeno in cui due o più qubit diventano correlati in modo tale che lo stato di uno dipende istantaneamente dallo stato dell'altro, indipendentemente dalla distanza che li separa).

Ad oggi, i prototipi di computer quantistici esistenti **sono ancora limitati** in termini di qubit e di tolleranza agli errori, e pertanto non sono in grado di eseguire algoritmi quantistici su larga scala o risolvere problemi computazionali complessi che richiederebbero un'enorme quantità di qubit affidabili.

La minaccia principale per la sicurezza informatica, in presenza di computer quantistici, non deriva dal semplice raggiungimento di un numero elevato di operazioni (come 2^{100}), **ma dall'adozione di algoritmi specifici**. Ad esempio, l'algoritmo di Shor può fattorizzare numeri interi in modo efficiente, mettendo a rischio la sicurezza dei sistemi di crittografia **asimmetrica** (come RSA e Diffie-Hellman).

Per quanto riguarda la crittografia **simmetrica**, i computer quantistici non offrono una potenza di calcolo superiore in senso classico, ma consentono uno **speed-up** degli attacchi brute force grazie all'algoritmo di Grover. Questo algoritmo **riduce il numero di tentativi necessari** passando da $O(2^n)$ a $O(2^{n/2})$. Di conseguenza, una chiave a 128 bit, che in un attacco classico richiederebbe 2^{128} tentativi, avrebbe una sicurezza equivalente a una chiave a circa 64 bit in presenza di attacchi quantistici. Per questo motivo, è necessario considerare chiavi di lunghezza maggiore o sviluppare nuove tecniche crittografiche resistenti agli attacchi quantistici.

1.2 Cifratura dei dati

La cifratura è una tripla di algoritmi efficienti $(\text{Gen}, \text{Enc}, \text{Dec})$:

- $K = \text{Gen}(1^n)$ genera una chiave segreta di n bit. La chiave non può essere di un solo bit, perché con sole due possibilità l'avversario potrebbe indovinarla facilmente provandole entrambe. Per garantire sicurezza, servono almeno n bit. [1]
  - Un numero rappresentato su n bit può assumere 2^n valori distinti. Ad esempio, un miliardo equivale circa a 2^{30}, quindi bastano 30 bit per rappresentarlo (dato da $\log_2 n$.
  - Un algoritmo che prende in input 30 bit e genera un miliardo di bit non è efficiente: sarebbe esponenziale rispetto all'input. Un algoritmo efficiente dovrebbe avere complessità almeno lineare, ovvero produrre un output proporzionale all'input, è una technicality[2]

  L'algoritmo Gen non viene spesso esplicitato perché è considerato banale: si limita a generare una chiave casuale di n bit.

  Generazione di numeri casuali La funzione prevede un intero come input e un output che corrisponde alla chiave di n elementi (bit). Per generare numeri casuali si potrebbe usare `Rand()`, ma questo è prevedibile. Per migliorarlo, si può usare `srand()` per impostare un seed iniziale, ma:

  1. `Rand()` è invertibile: conoscendo l'output si può risalire al seed.
  2. Il seed ha una dimensione insufficiente.
  3. L'inizializzazione con `time()` è insicura: spesso si usa il numero di millisecondi dal 1970, che può essere forzato con un attacco brute-force.

[1] Esiste sempre un trade-off tra sicurezza e prestazioni: aumentando la lunghezza della chiave si incrementa la sicurezza, ma si riducono le prestazioni.

[2] A point of law or a small detail of a set of rules, as contrasted with the intent or purpose of the rules.

Un'alternativa più sicura è l'uso di fonti di entropia del sistema operativo, come i file speciali virtuali `/dev/random` e `/dev/urandom` su Unix/Linux. La differenza tra i due è che `random` si blocca se non ci sono abbastanza dati casuali disponibili, mentre `urandom` continua a generare output anche in assenza di entropia sufficiente.

- $c = \text{Enc}(k, m)$ è la funzione di cifratura, che prende in input una chiave k e un messaggio m, restituendo un *ciphertext* (testo cifrato).
- $m = \text{Dec}(k, c)$ è la funzione di decifratura, che prende in input la chiave k e un testo cifrato c, restituendo il messaggio originale m.

Importanza della chiave È inevitabile che ci sia una chiave segreta per garantire che solo il destinatario autorizzato possa leggere il messaggio. Senza una chiave, un attaccante potrebbe decifrare facilmente il messaggio usando un semplice algoritmo di decrittazione.

Pubblicità degli algoritmi di cifratura Ma gli algoritmi Gen, Enc e Dec devono essere pubblici o segreti? In ambito accademico e nella pratica moderna, gli **algoritmi crittografici sono pubblici**, mentre la chiave deve restare segreta.

- Se l'algoritmo fosse privato, sarebbe più difficile individuarne eventuali vulnerabilità, perché meno persone potrebbero analizzarlo.
- La sicurezza deve dipendere solo dalla segretezza della chiave, non dall'oscurità dell'algoritmo.

Anche su Internet gli algoritmi di cifratura sono noti, mentre la chiave resta privata.

1.2.1 Esempio: cifrario di Cesare

Supponiamo di voler proteggere le comunicazioni. Ad esempio, Cesare scriveva lettere segrete e, per evitare che fossero intercettate lungo il percorso (un canale pubblico), le cifrava. La cifratura richiede una chiave condivisa tra il mittente (Cesare) e il destinatario (Cicerone).

Esempio pratico del cifrario di Cesare Nel cifrario di Cesare, il messaggio viene cifrato con uno *shift* delle lettere dell'alfabeto:

$$\text{Enc}(k, m) = \text{shift}(m, k)$$

$$\text{Dec}(k, c) = \text{shift}(c, -k)$$

Esempio:

- Cesare vuole inviare il messaggio $m =$"ciaobello".
- La chiave k è 3, quindi si applica uno shift a destra di 3 posizioni:

$$\text{ciaobello} \rightarrow \text{fldrehoor}$$

- Cicerone riceve il messaggio cifrato e lo decifra applicando uno shift a sinistra di 3 posizioni, recuperando il messaggio originale.

Vulnerabilità Il cifrario di Cesare presenta una grave vulnerabilità a un attacco brute-force, poiché ci sono solo 26 possibili chiavi. Un attaccante può provarle tutte e recuperare il messaggio originale in tempi rapidissimi.

1.2.2 Quando possiamo dire che un sistema di cifratura simmetrico è sicuro?

Un sistema di cifratura si può definire **sicuro** in modo dimostrabile (matematicamente, scientificamente, ecc.) quando non consente all'avversario di ottenere alcuna informazione utile sul messaggio o sulla chiave. Ma è corretto affermare che un **sistema è sicuro solo se l'avversario non può recuperare la chiave**? **No**, perché un avversario potrebbe riuscire a dedurre parte del messaggio senza necessariamente conoscere la chiave.

Esempio di sistema in cui la chiave è irrecuperabile, ma il messaggio è vulnerabile Consideriamo il seguente sistema di cifratura:

$$\text{Gen}(1^n) \to K \xleftarrow{R} \{0,1\}^n$$

$$\text{Enc}(k, m) \to m, \quad \text{Dec}(k, c) \to c$$

In questo caso, la probabilità di indovinare la chiave è $1/2^n$, rendendola di fatto irrecuperabile. Tuttavia, la sicurezza di un sistema di cifratura non si limita alla segretezza della chiave, ma deve garantire la protezione del messaggio stesso. Non è ammissibile che un avversario possa recuperare anche una parte del messaggio.

Definizione di sicurezza: quali proprietà sono necessarie? È fondamentale comprendere gli obiettivi di sicurezza e individuare le proprietà richieste per un sistema crittografico affidabile. Valutiamo alcune possibili condizioni:

- **L'avversario non può recuperare la chiave?** *Non basta.* Anche senza conoscere la chiave, potrebbe comunque dedurre informazioni sul messaggio.
- **L'avversario non può recuperare il messaggio?** *Non basta.* Potrebbe comunque riuscire a ricostruire alcune proprietà del messaggio, come la distribuzione statistica dei bit.
- **L'avversario non può recuperare neanche un singolo bit del messaggio?** *Non basta.* Anche se ogni bit singolarmente è protetto, un attaccante potrebbe comunque dedurre funzioni di più bit. Ad esempio, potrebbe capire se ci sono più 0 o più 1 nel messaggio, violando la sicurezza in scenari dove tali informazioni non devono trapelare.
- **Il ciphertext non fornisce alcuna informazione utile all'avversario? Sì, questa è la definizione corretta di sicurezza.** Un sistema di cifratura è sicuro se, osservando il ciphertext, l'avversario non è in grado di ottenere alcuna informazione aggiuntiva sul messaggio originale, né direttamente né attraverso funzioni statistiche. Quindi, anche qualora un avversario riuscisse a ottenere il ciphertext, non riuscirebbe a ottenere alcuna informazione

Un sistema di cifratura simmetrico è sicuro se il ciphertext è indistinguibile da un output casuale e non rivela alcuna informazione significativa all'avversario. Questo è il principio alla base della **sicurezza semantica** (*semantic security*), un concetto chiave nella crittografia moderna.

1.2.3 One-Time Pad (1917): Vernon-Shannon

Nel 1917, Gilbert Vernam propose un metodo di cifratura basato sull'operazione XOR, che fu successivamente analizzato da Claude Shannon, il quale dimostrò che il sistema garantisce **sicurezza perfetta**. Tuttavia, presenta un problema di usabilità che ne limita l'adozione pratica.

Schema del One-Time Pad Il One-Time Pad utilizza una chiave casuale lunga almeno quanto il messaggio:

$$\text{Gen}(1^n) \to k \xleftarrow{R} \{0,1\}^n$$

L'operazione di cifratura si basa sullo XOR bit a bit tra il messaggio e la chiave:

$$\text{Enc}(k, m) = m \oplus k$$

Esempio pratico Supponiamo di avere il seguente messaggio m e una chiave k generata casualmente:

$$m = 0110011$$
$$k = 1101001$$

Applicando lo XOR bit a bit:

$$c = m \oplus k = 1011010$$

Per decifrare il messaggio, si applica nuovamente lo XOR tra il ciphertext e la chiave:

$$\text{Dec}(k, c) = c \oplus k = m$$

Proprietà di sicurezza La caratteristica chiave del One-Time Pad è che il **ciphertext non fornisce alcuna informazione utile all'avversario**:

- Ogni bit cifrato può corrispondere sia a un bit `0` che a un bit `1` del messaggio originale, a seconda della chiave casuale.
- Non esiste alcuna correlazione tra il ciphertext e il messaggio, rendendo impossibile dedurre informazioni dal testo cifrato.
- La sicurezza è perfetta se la chiave è completamente casuale, della stessa lunghezza del messaggio e usata una sola volta.

Limiti del One-Time Pad Sebbene il One-Time Pad garantisca **sicurezza perfetta**, presenta una grave limitazione:

- **Gestione della chiave**: La chiave deve essere almeno lunga quanto il messaggio e non può essere riutilizzata, rendendo la distribuzione delle chiavi un problema logistico complesso.

Sicurezza imperfetta e praticità Poiché la gestione delle chiavi nel One-Time Pad è poco pratica, nella crittografia moderna si preferisce adottare sistemi con **sicurezza imperfetta**, che offrono un livello di protezione elevato, ma non assoluto.

- Un sistema è considerato sicuro se il tempo necessario per violarlo è *computazionalmente impraticabile*. Ad esempio, se un attacco richiede 2^{80} operazioni, il tempo necessario sarebbe di milioni di anni con la tecnologia attuale, rendendolo una scelta accettabile.
- La sicurezza imperfetta è calibrabile in base alla lunghezza della chiave e alla complessità della cifratura.

Mentre il One-Time Pad è teoricamente il metodo più sicuro, i moderni schemi crittografici si basano su modelli di sicurezza computazionale, che bilanciano protezione e usabilità.

2 Threat model

Lavoro di Turing Alan Turing e il team di Bletchley Park sfruttarono parole probabili (*cribs*), errori operativi e analisi sistematica per decifrare i messaggi cifrati di *Enigma*. Si ipotizza che schemi ricorrenti, come *"Heil Hitler"*, abbiano facilitato la decrittazione.

2.1 Threat Model per la cifratura dei dati

Analizziamo quali risorse ha a disposizione un avversario per decifrare i dati. Gli algoritmi di cifratura forniscono garanzie di sicurezza fino a un certo limite, ovvero sono progettati per resistere a determinati tipi di attacchi, ma non necessariamente a tutti.

Sender — Receiver
↑
Avversario

Attacco fisico Un avversario può compromettere la sicurezza in vari modi, incluso il cosiddetto *rubber-hose attack*[3], ovvero costringere fisicamente il receiver a rivelare la chiave o i dati cifrati.

Altro oltre il testo L'avversario, vedendo il ciphertext, magari sta osservando altro oltre al testo stesso. Supporre quindi che un avversario veda solo il ciphertext non è realistico. La storia ci insegna che anche in assenza di una conoscenza diretta dell'algoritmo di cifratura, come nel caso di Alan Turing con Enigma, è possibile riuscire a decifrare un messaggio. Per questo motivo, è fondamentale definire i **threat model**.

- **Known-Plaintext Attack (KPA)**: l'avversario dispone di alcuni messaggi in chiaro e dei corrispondenti ciphertext, ma **non può scegliere quali messaggi analizzare**. Questo tipo di attacco è utile per individuare schemi ricorrenti nella cifratura e dedurre informazioni sulla chiave o sull'algoritmo usato.
- **Chosen-Plaintext Attack (CPA)**: **l'avversario può scegliere liberamente i messaggi in chiaro e ottenere le rispettive cifrature**. Questo gli permette di testare il comportamento dell'algoritmo e, in alcuni casi, dedurre informazioni sulla chiave segreta.
- **Chosen-Ciphertext Attack (CCA)**: oltre a poter scegliere i plaintext (come CPA), l'avversario può fornire ciphertext arbitrari per farli decifrare e osservare il risultato. Tuttavia, **non è in grado di generare un ciphertext valido che venga accettato dal sistema**. Se potesse farlo, potrebbe sfruttare le risposte del receiver per ricostruire informazioni sulla chiave o sul messaggio originale.

Nota: il *One-Time Pad* è sicuro contro gli attacchi CPA, ma ha lo svantaggio che la **chiave deve essere lunga quanto il messaggio** e **non può essere riutilizzata** per più cifrature. Questo perché l'operazione di cifratura si basa su un semplice XOR tra il messaggio e la chiave, facilmente invertibile.
Nella pratica ai giorni d'oggi, **si utilizzano varianti del *One-Time Pad***, dove la chiave non è semplicemente applicata con XOR al messaggio, ma viene usata per generare una sequenza pseudo-casuale che poi viene combinata in XOR con il testo in chiaro.

2.1.1 Pseudorandom generator (PRG)

Un pseudorandom generator (PRG) è una funzione che prende in input un seed generato da una funzione random (es. `srand`). A partire da questo seed, vengono generati e restituiti dei bit che derivano unicamente dalla stringa casuale iniziale. Tuttavia, questi bit non sono realmente casuali; se il PRG è ben progettato, i bit generati risultano indistinguibili da quelli realmente casuali (funzioni `random` e `urandom`). L'obiettivo è rendere i bit generati non distinguibili da una sequenza casuale vera, impedendo di capire se siano effettivamente casuali o meno, per questo il generatore è detto **pseudorandomico**.

[3]The rubber hose attack is extracting secrets from people by use of torture or coercion.

Problemi di sicurezza Il seed deve essere scelto con attenzione: se è troppo piccolo, risulta facilmente attaccabile. Anche un seed grande (es. 100 bit) può essere vulnerabile se inizializzato con il timestamp di sistema (`time`), poiché è prevedibile. Anche utilizzando un seed grande e non inizializzato a `time`, ad esempio utilizzando generatori di entropia più avanzati come `random` o `urandom`, il problema persiste se il PRG utilizzato è invertibile. Un PRG sicuro, quindi, non deve essere invertibile.

Problemi difficili e funzioni one-way Esistono problemi per cui non sono stati trovati metodi di inversione efficienti. Un esempio è un circuito logico con molte porte OR e NOT e un unico filo di output: dato questo circuito, è difficile determinare se esista un input che produca output pari a 1. Non è stato trovato un algoritmo efficiente per risolvere questo problema, e si ipotizza che non esista. Questo suggerisce l'esistenza di problemi difficili e funzioni one-way, ovvero funzioni facili da computare ma difficili da invertire. Esistono funzioni attualmente considerate one-way, e vedremo come realizzare un PRG basato su questi concetti crittografici.

Esempio: OpenSSL RAND Un esempio di PRG sicuro è `RAND` di OpenSSL, utilizzato da diverso tempo. OpenSSL fornisce diversi strumenti per la crittografia, tra cui generatori di numeri casuali crittograficamente sicuri (`CSPRNG`)[4], progettati per resistere ad attacchi anche da parte di avversari potenti.

Sicurezza degli standard Se si utilizza un generatore standard, si può essere ragionevolmente sicuri dell'invertibilità dell'output? Non necessariamente. Un esempio critico è il `DUAL EC DRBG`, un generatore standard approvato dalla NSA, ma che conteneva una *trapdoor*[5] che permetteva all'NSA di decrittare le comunicazioni. Questo dimostra che anche gli standard possono essere corrotti.

PRG e crittografia Un PRG consente di ottenere una chiave pseudocasuale a partire da un seme (seed) iniziale. Se si tiene traccia di quanto output è stato utilizzato, nelle esecuzioni successive si può riutilizzare la stessa chiave, scartando le parti già usate. L'efficienza crittografica e la complessità si basano tutte sulla sicurezza del PRG. Con un PRG, è possibile ottenere una cifratura in cui la chiave è più corta del messaggio, a differenza dell'uso diretto di XOR con una chiave della stessa lunghezza del testo. Utilizzando un PRG in combinazione con XOR, si ottiene una cifratura **stateful**, ovvero una cifratura che dipende dallo stato interno del generatore e dall'output generato fino a quel momento[6]. Questo offre vantaggi in termini di sicurezza rispetto all'uso di XOR puro con una chiave statica, ma può risultare meno pratico poiché richiede la gestione dello stato per la corretta decifratura.

2.1.2 Pseudorandom Functions

Consideriamo un esperimento $ExpU_A(n)$ in cui un avversario interagisce con un vettore di stringhe casuali. L'avversario può richiedere il valore associato a un determinato indice e riceve in risposta la stringa casuale corrispondente:

1	abeefefcd
2	cbaeff
3	sfvfefwfef
...	...

[4]Nella sicurezza informatica un generatore di numeri pseudocasuali crittograficamente sicuro (detto in genere **CSPRNG** da Cryptographically Secure Pseudo-random Number Generator) è un generatore di numeri pseudo-casuali le cui proprietà lo rendono adatto all'uso in crittografia

[5]Una **trapdoor** è una particolare backdoor che funziona da punto di ingresso non documentato a un modulo o a un programma che un utente maligno potrebbe sfruttare per accedere al sistema.

[6]Ciò significa che l'output dipende non solo dall'input corrente, ma anche dalla sequenza di operazioni precedenti, rendendo necessario sincronizzare stato e chiavi tra mittente e destinatario per la corretta decifratura.

In un secondo esperimento, invece della tabella, viene utilizzata una funzione: $ExpPRF_A(n)$. Qui, il vettore di stringhe casuali è sostituito da una funzione pseudorandom (PRF), indicata come $PRF(s, \cdot)$, dove s è una stringa casuale di n bit e il secondo argomento è l'indice richiesto.

Indistinguibilità tra i due esperimenti L'avversario non è in grado di distinguere se i valori ricevuti provengano da una tabella statica o da una funzione. Una funzione è considerata una pseudorandom function (PRF) se nessun avversario efficiente riesce a distinguere se si trova nel primo o nel secondo esperimento. Se questa proprietà è verificata, allora la funzione è una PRF.

Applicazione alle cifrature stateless Questa proprietà consente di ottenere una cifratura senza stato (*stateless encryption*). Vediamo come si realizza.

2.1.3 CPA-Secure Encryption

Consideriamo un messaggio fisso m di n bit. In uno schema di cifratura CPA-secure, possiamo **cifrare più volte lo stesso messaggio senza dover cambiare la chiave**. Per garantire che ogni cifratura produca un ciphertext diverso, utilizziamo una stringa casuale r, che viene scelta casualmente a ogni cifratura e poi usata come **indice** nella Pseudorandom Function (PRF).

- $Enc(k, m) = (r, PRF(k, r) \oplus m)$.
- $Dec(k, c) = Dec(k, (r, z)) = PRF(k, r) \oplus z$.

Proprietà della cifratura La tabella rimane invariata poiché la chiave k è usata sia in fase di cifratura che di decifratura. Tuttavia, ogni volta che si cifra un nuovo messaggio, r è scelto casualmente, quindi è sempre diverso. Questo garantisce che lo stesso messaggio cifrato più volte generi ciphertext differenti, migliorando la sicurezza. Questo è il metodo di cifratura utilizzato oggi.

Attacco di modifica del ciphertext **Un avversario può creare un ciphertext valido:** se osserva una coppia (r, z), dove z è il ciphertext, può modificare un bit di z e ottenere un messaggio decifrato con un solo bit alterato. Ad esempio, se il messaggio originario termina con il bit 0 (es. il numero 28), l'avversario può modificarlo in 1, trasformandolo in 29 dopo la decifratura.

È anche CCA-sicuro? No, questo schema non è CCA-secure. Un esempio di attacco è quello delle aste online: un avversario potrebbe modificare un'offerta cifrata per aumentarla di 1. Tuttavia, la chiave rimane comunque segreta e non direttamente estraibile dall'attacco.

2.1.4 CPA-Secure Encryption: messaggi lunghi

Problema Come cifrare un messaggio più lungo dell'output di una PRF? Come gestire la criticità di un PRG che richiede di prevedere in anticipo quanti bit saranno necessari?

- **Soluzione con stream cipher:** Possiamo utilizzare uno *stream cipher*, che è flessibile per cifrare messaggi di qualsiasi lunghezza. Questo metodo genera uno stream di bit pseudocasuali da combinare con il messaggio in ingresso tramite operazione XOR man mano che i bit arrivano.
- **Soluzione con block cipher:** Un altro metodo è la cifratura a blocchi (*block cipher*), che opera su blocchi di bit. Tecnicamente, si tratta di una **pseudorandom cipher permutation** (one-to-one), una funzione speciale che, dato un segreto k e un messaggio, restituisce un messaggio cifrato corrispondente. Essendo **invertibile**, permette anche la decifratura.

Vantaggi del Block Cipher: diffusione La differenza principale è che il *block cipher* offre maggiore sicurezza poiché incorpora il concetto di **diffusione**. Questo significa che una piccola variazione nell'input (anche un solo bit) punta a un altro indice diverso della tabella e causa una modifica significativa nell'output di tutto il blocco, rendendo più difficile condurre attacchi crittografici. Tuttavia, l'operazione richiede di aspettare il completamento dell'intero blocco prima di elaborarlo.

Gestione dei blocchi incompleti Se i blocchi sono di dimensione fissa, la cifratura si interrompe al termine del messaggio. Se il messaggio non riempie un intero blocco, si utilizza il **padding** per completarlo.

Block Cipher vs Stream Cipher in comunicazioni IoT Nelle comunicazioni IoT, è spesso più conveniente utilizzare uno *stream cipher* invece di un *block cipher*, poiché non richiede di accumulare un intero blocco prima della cifratura, rendendo il processo più adatto a scenari con risorse limitate e bassa latenza.

3 Block ciphers

Diciamo che uno schema di cifratura è **deterministico** quando la funzione Enc è deterministica. Ciò significa che, dato lo stesso input, la funzione produce sempre lo stesso output.

- **Deterministico:** con gli stessi input si ottiene sempre lo stesso output.
- **Non deterministico:** con gli stessi input si può ottenere un output diverso.[7]

3.0.1 Esercizio

Domanda: *Può uno schema di cifratura deterministico essere* ***CPA-sicuro****?*
Risposta: Se un cifrario è **CPA-sicuro**, significa che l'avversario può ottenere accesso al tipo di cifratura e analizzare i testi cifrati. Tuttavia, se lo schema di cifratura è **deterministico**, allora cifrando lo stesso messaggio più volte si otterrebbe sempre lo stesso testo cifrato. Questo permetterebbe all'avversario di riconoscere messaggi uguali semplicemente confrontando le loro cifrature.

Per garantire la sicurezza CPA, è necessaria una componente **randomica** nel processo di cifratura.[8]

Quindi, la risposta è **no**: uno schema di cifratura deterministico non può essere CPA-sicuro, poiché un avversario potrebbe identificare messaggi uguali osservando i testi cifrati corrispondenti.

3.1 Electronic Code Book (ECB)

Un *stream cipher* funziona eseguendo un'operazione XOR bit a bit tra il messaggio in chiaro e una chiave pseudo-casuale, mentre un *block cipher* cifra interi blocchi di dati in modo indipendente. Una modalità operativa dei cifrari a blocchi è la **Electronic Code Book (ECB)**.

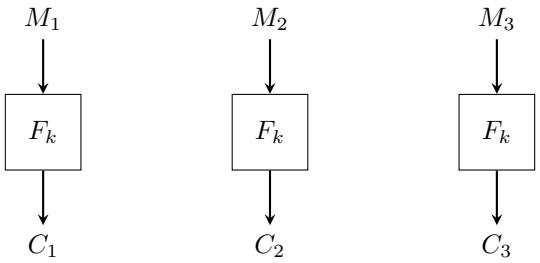


Figura 1: ECB in azione: ogni blocco M_i è cifrato indipendentemente usando la stessa F_k.

Determinismo Questa modalità è **deterministica**: lo stesso blocco di input produce sempre lo stesso blocco cifrato. È una modalità molto semplice ma non completamente sicura, in quanto non garantisce la sicurezza CPA.

Vulnerabilità Anche se il cifrario di base F_k è crittograficamente forte, l'assenza di casualità nella cifratura introduce vulnerabilità. Un avversario che ottiene il ciphertext può sfruttare questa proprietà per riconoscere pattern nei dati cifrati, come mostrato nell'esempio seguente:

[7] Un algoritmo come *Quicksort* può avere un'esecuzione non deterministica se utilizza la randomizzazione, come nel caso della scelta pivot casuale. Tuttavia, nella sua versione classica con pivot fisso, è deterministico.

[8] Ad esempio, se il cifrario utilizza una *Pseudo-Random Function* (PRF), la cifratura diventa non deterministica, poiché la PRF introduce casualità nel processo di cifratura (indice della tabella randomico). Questo garantisce che lo stesso messaggio possa essere cifrato in modi diversi.

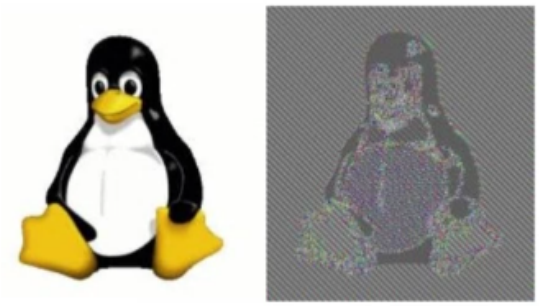


Figura 2: Esempio dell'insicurezza di ECB: l'immagine cifrata preserva la struttura dell'originale, rendendo visibile il pinguino di "Tux" di Linux

Osserviamo come il **determinismo** di ECB è considerato una **vulnerabilità**: ad esempio, un pixel bianco viene sempre cifrato nello stesso modo. Un avversario, pur non conoscendo il valore esatto del pixel, può dedurre che blocchi con lo stesso output cifrato corrispondono a porzioni identiche dell'immagine originale, permettendogli di ricostruire informazioni sensibili.
NOTA: Zoom utilizzava questa modalità di cifratura fino al 2020.

3.2 CBC mode

CBC Cipher Block Chaining è considerato CPA-secure, ma la cifratura è **sequenziale**.

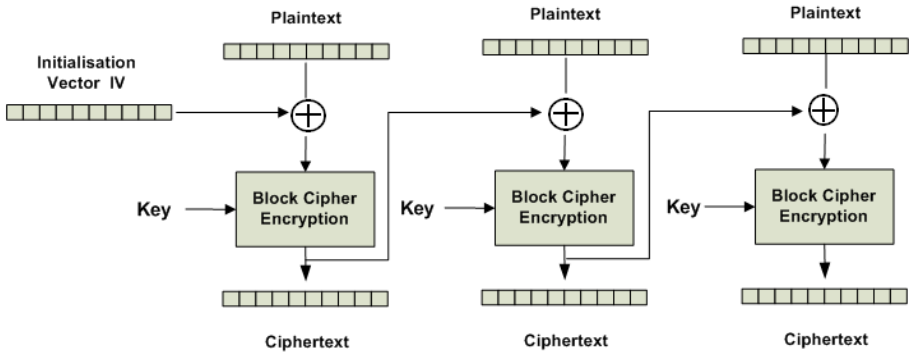


Figura 3: Schema del Cipher Block Chaining con IV, con *Block Cipher Encryption* $\equiv F_k$

Si **suddivide il messaggio in blocchi** della dimensione richiesta dal *block cipher*. Per "randomizzare" il processo viene scelto un vettore di inizializzazione (IV) casuale (stringa), che è inviato in chiaro ma non rivela informazioni utili a un attaccante.
Il primo blocco di testo in chiaro m_1 viene combinato in XOR con l'IV e poi cifrato tramite la funzione F_k (il block cipher con chiave k), producendo il blocco cifrato c_1. Il secondo blocco m_2 viene XORato con c_1 e nuovamente cifrato per ottenere c_2, e così via per tutti i blocchi. Quindi avviene:

1. $(m_1 \to \text{XOR con } IV \to F_k) = c_1$
2. $(m_2 \to \text{XOR con } c_1 \to F_k) = c_2$
3. ...

Formalmente:

$$c_1 = F_k(m_1 \oplus IV), \quad c_2 = F_k(m_2 \oplus c_1), \quad \dots$$

Di conseguenza, anche se $m_1 = m_2$, i corrispondenti blocchi cifrati c_1 e c_2 risulteranno diversi, poiché ogni blocco cifrato dipende da quello precedente e dall'IV. L'overhead in termini di banda è limitato all'IV, che solitamente ha la stessa dimensione di un blocco (processo one-time).

A differenza della modalità ECB, che può essere cifrata in parallelo (ad esempio su più server), la modalità CBC richiede che i blocchi vengano cifrati in **sequenza**, poiché ogni blocco in input dipende dal blocco cifrato precedente. Questo rende la cifratura più lenta in scenari altamente paralleli, pur garantendo migliori proprietà di sicurezza rispetto a ECB.

3.3 Counter (CTR) mode

Nella modalità *Counter* (CTR), il block cipher viene utilizzato come generatore di keystream. Si inizia con un contatore (*counter*) inizializzato in modo casuale (o pseudo-casuale).

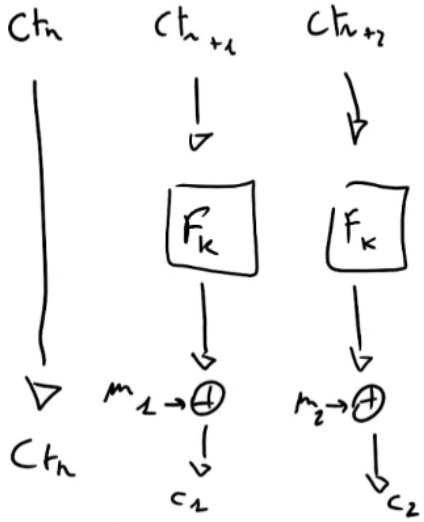


Figura 4: Schema del Counter (CTR) Mode

Per processare il blocco i-esimo, si incrementa il contatore di 1 rispetto al valore iniziale e si passa il contatore incrementato alla funzione di cifratura F_k. Il risultato viene combinato in XOR con il blocco di testo in chiaro m_i, producendo il blocco cifrato c_i. Formalmente:

$$c_i = m_i \oplus F_k(\text{counter} + i).$$

Il contatore viene incrementato per ogni blocco successivo:

$$c_1 = m_1 \oplus F_k(\text{counter} + 1), \quad c_2 = m_2 \oplus F_k(\text{counter} + 2), \quad \ldots$$

Vantaggi (*pro*):

- È considerata **CPA-secure**, a patto che non venga riutilizzato lo stesso contatore/nonce con la medesima chiave.
- **Non richiede l'uso esplicito dell'invertibilità** di F_k durante la cifratura, poiché la decifratura avviene analogamente con XOR.
- **È facilmente parallelizzabile**, poiché i valori del contatore possono essere calcolati in anticipo e i blocchi possono essere cifrati in parallelo.
- Può essere vista come un **ibrido tra stream cipher e block cipher**, poiché genera uno *stream* di chiavi da combinare con il messaggio in XOR.

Svantaggi (*contro*):

- **Se F_k presenta debolezze e il contatore varia di poco** (in modo prevedibile, ad esempio $\text{Counter}_i = \text{Counter}_{i-1} + 1$), un attaccante potrebbe correlare gli output successivi e tentare di estrarre informazioni.
- In caso di lunghi flussi di cifratura (ad esempio 2^{40} blocchi), se il contatore è solo a 40 bit, si rischia di riutilizzare valori del contatore o di non avere sufficiente casualità sui restanti bit (ad esempio su 128 bit, 40 sono dedicati al contatore e i rimanenti potrebbero non variare abbastanza).
- Riutilizzare lo stesso contatore/nonce[9] con la stessa chiave per messaggi diversi porta a gravi vulnerabilità di sicurezza.

3.4 Stream cipher vs block cipher

- **Stream cipher:** Uno *stream cipher* permette di cifrare progressivamente i bit (o i byte) del messaggio uno alla volta e di inviarli immediatamente, mantenendo un buon rapporto (rate) tra la dimensione del ciphertext e quella del plaintext. Tuttavia, il valore di un bit del plaintext influenza soltanto il corrispondente bit del ciphertext, quindi non si ottiene alcuna "diffusione" (nel senso di far dipendere molti bit di output da ogni singolo bit di input). Inoltre, uno stream cipher fornisce un PRG "dinamico", ossia può generare i prossimi bit pseudocasuali "on demand".
- **Block cipher:** Un *block cipher* cifra un intero blocco di dati alla volta ed è meno efficiente se utilizzato per cifrare pochi bit (poiché è necessario aggiungere padding per completare il blocco). Tuttavia, il valore di un singolo bit in input influenza molti bit in output, garantendo una maggiore "diffusione" e rendendo la cifratura più robusta contro attacchi. Inoltre, la modalità con cui si concatenano i vari blocchi (CBC, CTR, ecc.) è fondamentale per preservare la confidenzialità dell'intero messaggio.

4 Integrità dei messaggi

Come possiamo essere sicuri che un messaggio ricevuto provenga da un certo mittente? È sufficiente utilizzare la crittografia per evitare modifiche durante il percorso? In realtà, un sistema crittografico CPA-secure non garantisce l'integrità, poiché un avversario potrebbe alterare il messaggio e farlo recapitare in forma modificata al destinatario.

4.1 MAC

Una funzione MAC (Message Authentication Code) è una funzione (una tripla) utilizzata per calcolare un tag (o etichetta) del messaggio, che viene inviato insieme al messaggio per verificarne integrità e autenticità, per verificare quindi che non sia stato contraffatto. Il destinatario, infatti, utilizza una funzione di verifica (`vrfy`) per controllare la validità del messaggio.

$$\begin{array}{ccc} \text{Mittente } (S_m) & \longrightarrow & \text{Destinatario } (R_k) \\ & \nwarrow & \\ & \text{Avversario} & \end{array}$$

Il mittente, possedendo la chiave segreta k, calcola

$$\text{MAC}(k, m) = \sigma$$

e invia m e σ. Il destinatario riceve il messaggio e il tag, e calcola la funzione di verifica:

$$\text{vrfy}(k, m, \sigma) = \begin{cases} 1 & \text{se } \sigma = \text{MAC}(k, m) \\ 0 & \text{altrimenti} \end{cases}$$

[9] I restanti bit spesso vengono usati come nonce, cioè un valore univoco (per chiave) che cambia a ogni sessione o messaggio

dove il valore 1 indica che il messaggio è autentico, mentre 0 indica che è stato alterato.

L'avversario può intercettare e modificare i messaggi inviati dal mittente; egli ha successo se riesce a far recapitare un messaggio modificato al destinatario, che verrà erroneamente considerato autentico. Questa proprietà, nota come *non-forgery* (unforgeability), garantisce che l'avversario non possa generare (forgiare) un tag valido per un messaggio non inviato dal mittente.

4.1.1 Funzione di verifica

In molte costruzioni canoniche, la funzione di verifica è implementata semplicemente confrontando il tag ricevuto con quello calcolato.

$$\begin{aligned}&\text{vrfy}(k, m, \sigma) :\\ &\quad \sigma' = \text{MAC}(k, m)\\ &\quad \textbf{return } (\sigma = \sigma')\end{aligned}$$

Non serve elaborare una vera e propria funzione, ma è *canonica* in quanto funziona sempre nello stesso modo, senza necessità di una funzione separata di verifica.

4.1.2 Schema del MAC

Lo schema di MAC si realizza senza problemi con una funzione pseudocasuale (PRF). In questo caso, per calcolare il MAC di un messaggio m con chiave segreta k, definiamo:

$$\text{MAC}(k, m) = \text{PRF}(k, m)$$

- **Dimensione fissa:** L'avversario per ogni messaggio vede una entry diversa, per ogni messaggio c'è una entry diversa, e quindi calcolare un nuovo MAC è impossibile senza avere la chiave. Questa definizione è adatta a messaggi la cui lunghezza corrisponde a quella dell'input della PRF, tipicamente un numero fisso di bit (ad esempio, n bit). In questo contesto, la tabella delle possibili uscite della PRF avrebbe 2^n possibili valori, ciascuno di n bit, rendendo praticabile la computazione solo per messaggi di dimensione fissa.
- **Dimensione variabile:** Per messaggi di dimensione maggiore (ad esempio, un file di 3 MB), la semplice applicazione diretta della PRF non è possibile, non a causa di un possibile attacco, ma perché ciò è incompatibile. In questi casi, il messaggio deve essere suddiviso in blocchi compatibili e si devono adottare schemi di estensione sicuri che garantiscano la sicurezza anche per messaggi di lunghezza variabile.

Visualizzando la Figura 5, notiamo che:

- **Schema superiore (lunghezza fissa):** L'overhead sulla rete è molto basso, ma la dimensione del messaggio deve essere fissa.
- **Schema inferiore (lunghezza variabile):** Se invece m può avere dimensioni arbitrarie (quindi non vincolate a una lunghezza fissa), la computazione inizia specificando quanto è effettivamente grande il messaggio m. Questa tecnica garantisce un'elevata usabilità e produce un MAC di lunghezza costante (ad esempio, 256 bit), indipendente dalla lunghezza effettiva del messaggio.

Notare che il meccanismo del primo schema va bene, funziona purché chi calcola il MAC di un messaggio, calcoli sempre il MAC di messaggi di lunghezza fissa. Se in questo schema fosse utilizzato per calcolare messaggi di MAC di lunghezza arbitraria, allora avremmo un attacco. Detto ciò, vediamo come avviene l'attacco.

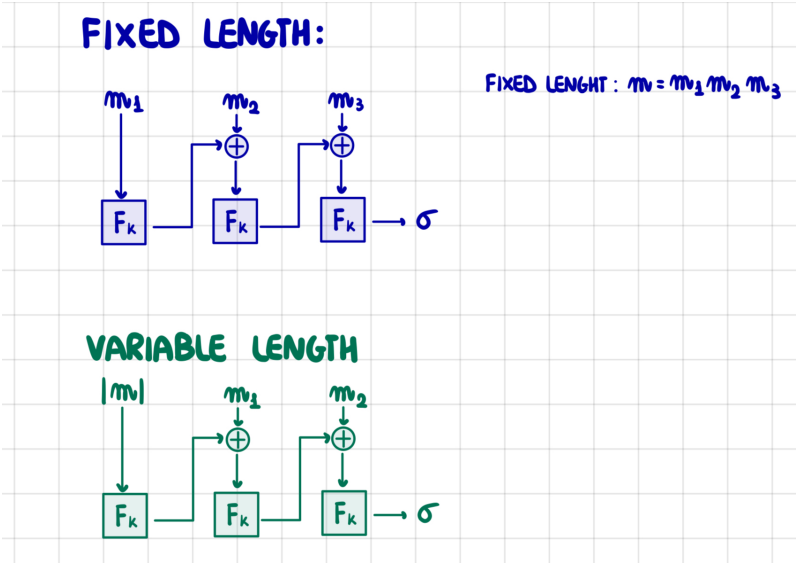


Figura 5: In alto schema per dimensione fissa, in basso per dimensione variabile

4.1.3 Attacco per schema a lunghezza fissa

Un avversario, sfruttando uno schema di autenticazione a lunghezza fissa, può manipolare la lunghezza dei messaggi per ottenere informazioni e inviare falsi messaggi autenticati. In particolare:

1. L'avversario, osservando il comportamento del sistema (attraverso l'ambiente), può cercare di influenzare il sender affinché invii un messaggio specifico al receiver. Poiché il sender calcola e trasmette anche il MAC del messaggio, l'avversario otterrà sia il messaggio che il suo MAC.
2. Supponiamo che il sender debba inviare un messaggio m_1. Per farlo, calcola il MAC associato utilizzando la funzione F_K (m_1 viene messo in input alla funzione F_K), ottenendo $\sigma_1 = F_K(m_1)$. L'avversario acquisisce quindi la coppia (m_1, σ_1), ovvero messaggio1 e il MAC di quel messaggio.
3. L'avversario esegue l'operazione XOR tra σ_1 e un altro messaggio m_2.
4. L'avversario, seguendo questo schema, inserisce m_2 XOR σ_1 in input a F_K. L'output di questa operazione è $\sigma*$, che è il MAC di m_2 XOR σ_1. Formalmente quindi in questo passaggio l'avversario calcola $m_2 \oplus \sigma_1$ e lo utilizza come input alla funzione F_K, ottenendo un nuovo valore $\sigma^* = F_K(m_2 \oplus \sigma_1)$.
5. Il messaggio $(m_2 \oplus \sigma_1)$ e il MAC σ^* vengono quindi trasmessi sul canale di comunicazione.
6. L'avversario intercetta σ^* e ora conosce sia σ_1 che σ^*.
7. Adesso l'avversario, *senza chiedere nulla al sender e senza influenzarlo*, direttamente sul canale di comunicazione inserisce un nuovo messaggio al MAC ricavato, e quindi fa ricevere al receiver un nuovo messaggio *contraffatto*, in modo tale che il receiver pensi che venga dal sender, quando il realtà non è così, e quindi l'avversario vince. Il messaggio inviato dall'avversario è $m_1 \| m_2$ (m_1 concatenato m_2). Il MAC di questo messaggio è $\sigma*$.

Invio legittimo del sender Notiamo che l'attacco va a buon fine perché, se un sender dovesse mandare il messaggio $m_1 \| m_2$, dovrebbe:

1. inserire m_1 in F_K ed esce σ_1;
2. poi $\sigma_1 \oplus m_2$, che è il messaggio della seconda richiesta fatta al sender e, inserendolo in F_K, dà in output $\sigma*$;

3. quindi quando uno calcola il MAC $m_1 \| m_2$ esce $\sigma*$.

L'avversario ha quindi correttamente mandato come $\sigma*$ il MAC e $m_1 \| m_2$ senza aver mai chiesto al sender e senza che il sender lo abbia mai calcolato, il quale non ha mai voluto mandare al receiver un messaggio $m_1 \| m_2$; è però l'avversario che lo manda, e viene **ricevuto come genuino**, quindi l'attacco è **vincente**. Lo schema (rappresentato in Figura 6) è quindi vulnerabile perché assume una lunghezza fissa.

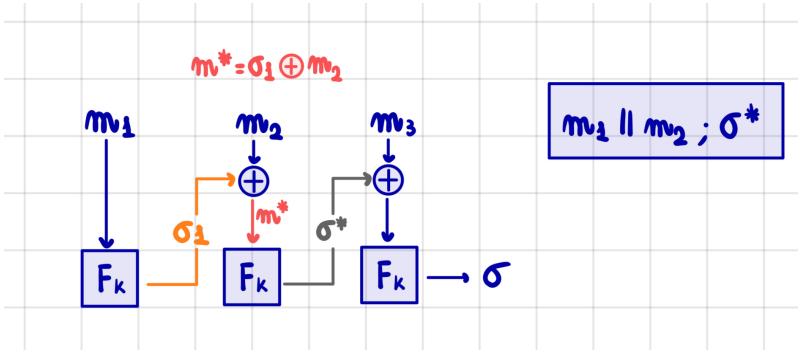


Figura 6: Schema a lunghezza fissa

Differenze con lunghezza variabile Nella seconda variante (lunghezza variabile), invece, lo schema è migliore perché la computazione dipende dalla reale lunghezza di m e questo "disorienta" l'avversario, rendendo l'attacco descritto non applicabile. La miglioria non è gratis, ma ha un blocco in più di overhead, che però lo rende sicuro per calcolare messaggi di lunghezza arbitraria. L'attacco di prima fallisce perché si applicava al fatto che veniva richiesto di calcolare il MAC di un messaggio di una certa lunghezza e dopo veniva attaccato a un messaggio più lungo, ed era utile il primo MAC sul messaggio più piccolo per calcolare il MAC del messaggio più lungo. Nella seconda variante invece, i MAC dei messaggi più piccoli sono *inutili* per calcolare i MAC del messaggio più grande, perché c'è la fase iniziale che va a separare completamente il calcolo del MAC quando cambia la lunghezza.

4.1.4 Conclusioni

Per garantire l'integrità dei messaggi, bisogna considerare i seguenti aspetti:

- **Replay attack:** Senza contromisure adeguate, l'avversario può semplicemente "rigiocare" (ossia reinviare) una coppia (m, σ) valida intercettata dal *sender*. *Esempio:* L'app Immuni generava codici pseudocasuali che cambiavano ogni 15 minuti per evitare il tracciamento (pseudorandom function). Tuttavia, era comunque possibile effettuare un replay attack: un avversario poteva leggere i codici da un altro dispositivo e reinviarli a terzi, sfruttando il fatto che quei codici fossero ancora validi in un certo intervallo temporale.
- **Sincronizzazione / Timestamps:** In molte applicazioni, per contrastare il replay, si ricorre a meccanismi di sincronizzazione o a timestamp. Se un messaggio è troppo vecchio (ovvero il timestamp è scaduto), viene scartato a prescindere.
- **Timing attacks:** Occorre prestare attenzione all'implementazione pratica di un algoritmo, anche se formalmente sicuro. La soluzione crittografica poteva essere valida, ma una gestione non corretta dei tempi rendeva il sistema vulnerabile a un'estrazione graduale di informazioni (ad esempio, "un bit alla volta").

5 Authenticated Encryption

Per ottenere un algoritmo crittografico completo che garantisca sia confidenzialità che integrità, è necessario combinare la cifratura con un codice di autenticazione del messaggio (MAC). Se,

ad esempio, ci si concentra solo sull'integrità, il messaggio m potrebbe essere inviato in chiaro, compromettendo la riservatezza.

Encrypt-then-mac Pertanto, per assicurare entrambe le proprietà si procede combinando cifratura e MAC tramite l'approccio *encrypt-then-mac*, in cui il MAC viene calcolato sul ciphertext. In questo modo il messaggio m è protetto dalla cifratura e si garantisce l'integrità, poiché l'avversario non può creare un nuovo messaggio valido: ogni modifica al ciphertext produce un MAC non corrispondente e il messaggio viene rifiutato. L'approccio *encrypt-then-mac* consente di ottenere una sicurezza CCA (Chosen Ciphertext Attack) superiore alla sicurezza CPA (Chosen Plaintext Attack), poiché anche cambiando il ciphertext quindi il MAC è diverso e il messaggio è rigettato.

5.1 Authenticated Encryption with Associated Data (AEAD)

Confidenzialità e integrità In molti scenari applicativi è necessario garantire la **confidenzialità** dei dati e l'**integrità** non solo dei dati stessi, **ma anche del contesto** in cui vengono utilizzati, ad esempio per prevenire attacchi di replay mediante timestamp o altri dati di contesto. Vorrei che un eventuale uso fuori contesto lo renda invalido. Quando mando dei dati e sono confidenziali li cifro e per integrità mi interessa che non siano modificati così come mi interessa che resti integro nel contesto in cui sono utilizzati. Per risolvere questo problema si usa AEAD (Authenticated Encryption with Associated Data).
Con AEAD, oltre a proteggere il messaggio m, si autenticano anche dei dati associati (associated data) che, pur rimanendo in chiaro, vengono inclusi nel calcolo del MAC per legare il messaggio al suo contesto.

Approccio moderno Questo approccio è comunemente impiegato nelle comunicazioni moderne, specialmente quando si usano modalità di cifratura a blocchi come il **Galois/Counter Mode (GCM)**. Nel GCM, oltre ai dati cifrati, vengono trasmessi anche i dati di contesto nel MAC, garantendo così l'integrità globale della comunicazione.

AES-GCM-256 L'algoritmo AES-GCM con chiave a 256 bit è uno standard consolidato da circa 20 anni e offre un'elevata sicurezza. È inoltre standardizzato anche per chiavi a 128 bit, sebbene, in uno scenario di sicurezza post-quantistica, una chiave da 256 bit risulti preferibile per una protezione più robusta.

6 Collision-Resistant Hash Functions

Le Collision-Resistant Hash Functions sono funzioni che, pur ammettendo l'esistenza di collisioni (per il principio del piccione, se il dominio è più grande dell'immagine, collisioni devono esistere), rendono computazionalmente difficile trovarle. In altre parole, data una funzione hash (fortemente suriettiva) $H : \mathcal{X} \to \mathcal{Y}$ con $|\mathcal{Y}|$ molto inferiore a $|\mathcal{X}|$, si assume che, pur esistendo collisioni, nessun attaccante abbia la capacità computazionale di individuarle.

6.1 Uso nelle tabelle hash

Perché non le usiamo per le tabelle hash? Perché **serve tanta memoria** per quanto è il range della memoria.
Le funzioni hash crittografiche, come ad esempio SHA-256, non sono ideali per l'implementazione di tabelle hash a causa del loro vasto spazio di output. In un hash a 256 bit il numero di possibili valori è 2^{256}, rendendo impraticabile l'allocazione di memoria sufficiente per gestirli tutti. Nelle tabelle hash, invece, si utilizzano funzioni appositamente progettate per distribuire le chiavi in un intervallo limitato, accettando la presenza di collisioni che possono essere gestite tramite tecniche specifiche.

6.1.1 Il paradosso del compleanno

Il paradosso del compleanno illustra come, in uno spazio di n possibili valori, ci si possa aspettare una collisione con una probabilità significativa dopo aver osservato circa $\sqrt{n}$ elementi. Ad esempio, per $n = 365$ (i giorni dell'anno) si arriva a una probabilità rilevante con circa 20-23 persone. Analogamente, per una funzione hash a 256 bit, trovare una collisione richiede in media circa 2^{128} operazioni in ambiente classico. Con l'impiego di computer quantistici, grazie a un miglioramento quadratico, il costo per trovare una collisione si ridurrebbe a circa 2^{64} operazioni.

6.2 Sicurezza di SHA-256

SHA-256 è uno standard consolidato, ha un dominio unrestricted, in cui l'output è di 256 bit. La sua sicurezza si basa sull'inabilità pratica di trovare:

- una preimmagine (un input tale che l'output è quel 256 bit), che richiederebbe circa 2^{256} operazioni in attacchi brute-force (paradosso del compleanno);
- una collisione, ovvero due input distinti che generano lo stesso output. Questo richiederebbe circa 2^{128} operazioni in un ambiente classico e circa 2^{64} operazioni con attacchi quantistici (secondo l'algoritmo di Grover).

Le funzioni hash *collision-resistant* sono fondamentali in crittografia, e SHA-256 è una delle più utilizzate. La sua funzione di hash è definita come:

$$\text{SHA-256} : \{0,1\}^* \to \{0,1\}^{256}$$

6.3 Applicazioni delle CRHF

Le funzioni hash resistenti alle collisioni (*Collision-Resistant Hash Functions*, CRHF) trovano applicazione in numerosi ambiti crittografici. Una proprietà fondamentale è che, se $H(x) = H(y)$, possiamo assumere con elevata sicurezza che $x = y$, poiché la probabilità di collisione è trascurabile. Le CRHF garantiscono un output univoco e non riproducibile per input distinti, rendendole strumenti essenziali in diversi scenari critici, che ora analizziamo.

6.3.1 MAC a taglia costante da un MAC lineare

Supponiamo di avere uno schema di MAC in cui il valore del MAC è grande quanto il messaggio stesso. È possibile convertire tale schema in uno con taglie più piccole (labels più piccole) utilizzando una funzione hash resistente alle collisioni (CRHF). **Invece di calcolare direttamente il MAC del messaggio, si calcola il MAC dell'hash del messaggio.** Poiché l'hash ha una dimensione fissa e ridotta, anche il MAC risultante sarà più piccolo. Grazie alla proprietà di resistenza alle collisioni della funzione hash, l'attaccante, anche conoscendo σ, non è in grado di risalire al messaggio originale, poiché non riesce a trovare collisioni. In questo modo si ottiene un MAC più compatto senza compromettere la sicurezza.

6.3.2 Hash come random oracle

Una funzione hash può essere utilizzata come un oracolo casuale, ovvero come una funzione che produce output apparentemente casuali. La differenza principale rispetto a una funzione pseudocasuale è che qui non è presente un seed iniziale. Ogni volta che si calcola la funzione hash su un dato input, si ottiene un valore che viene trattato come se fosse generato casualmente, anche se in realtà è deterministico. Questo approccio è un abuso teorico, ma viene spesso impiegato in crittografia per simulare l'uso di oracoli casuali.

6.3.3 Fingerprinting di file (e virus) e deduplicazione

Utilizzando CRHF, si può estrarre il *fingerprint* dei file, che sono bit univoci per ogni singolo file. Ciò risulta essere utile in diverse situazioni, quali:

- **Antivirus:** Un tipico antivirus sfrutta la proprietà delle funzioni hash per individuare malware. Poiché i virus hanno pattern specifici, invece di memorizzare l'intero codice di ogni malware noto, gli antivirus mantengono solo gli hash di questi pattern (ad esempio, 256 bit per pattern). Se un file sospetto viene analizzato, il suo hash viene confrontato con quelli nella lista delle minacce note.
- **Cloud e file identici:** Un altro esempio di applicazione è il cloud storage, che utilizza le funzioni hash per evitare la duplicazione di file. Quando un utente carica un file, il sistema calcola il suo hash e verifica se un file con lo stesso hash è già presente. Se esiste, l'utente viene informato, evitando così di memorizzare più copie dello stesso file. Questo metodo è efficiente perché permette di identificare file univocamente tramite un'impronta digitale compatta, riducendo il consumo di spazio e risorse. La resistenza alle collisioni delle funzioni hash garantisce che due file distinti non possano avere lo stesso hash, evitando errori di identificazione.

6.3.4 Uso di CRHF per MAC

Un possibile schema di autenticazione potrebbe essere

$$\mathrm{MAC}(k, m) = H(k \| m)$$

Ma si può realmente utilizzare una CRHF come schema di MAC? A prima vista, questo approccio potrebbe sembrare sicuro, in quanto l'avversario non riesce a ottenere informazioni dirette. Tuttavia, esso **risulta insicuro** perché SHA-256 ha un dominio illimitato (*unrestricted domain*).

Length extension attack In questo schema, SHA-256 processa iterativamente i blocchi del messaggio, *mantenendo uno stato interno che viene aggiornato progressivamente*. Quando tutti i blocchi sono stati elaborati, l'output finale dipende dall'ultimo stato interno e dal messaggio stesso. Questo processo è simile a quello illustrato in Figura 5.

- Il **problema principale** di questa costruzione è che un avversario può richiedere il MAC di un messaggio e ottenere $H(k\|m)$, a condizione di poter stimare la lunghezza di k. Questo è possibile perché lo stato interno dell'hash dopo aver elaborato $(k\|m)$ può essere utilizzato come punto di partenza per continuare l'elaborazione con i blocchi aggiuntivi dell'extension. Lo stato interno è lo SHA-256 di tutti i blocchi processati e ad ogni blocco viene aggiornato lo SHA-256. Se viene richiesto lo SHA-256 di un messaggio piccolo è come se si fosse chiesto lo stato interno del messaggio più lungo.
- Se poi l'attaccante **desidera estendere il messaggio**, può sfruttare lo stato interno dello SHA-256 già calcolato per continuare l'hashing con nuovi blocchi. In altre parole, se si ha il MAC di un messaggio corto, è possibile calcolare il MAC di un messaggio più lungo senza conoscere la chiave segreta. Questa vulnerabilità rende lo schema insicuro.

Metodo non sicuro Questo **non** è un metodo sicuro per costruire un MAC, perché, avendo il MAC di un messaggio corto, si può estendere il messaggio e continuare a calcolare nuovi MAC senza necessità di interazione con la chiave segreta. Questa versione relativamente semplice, quindi, **non** è sicura.

6.3.5 HMAC

Dato quindi che l'uso di CRHF per la creazione di MAC non è sicuro, lo standard[10] adottato per i MAC è **HMAC**, definito come:

$$\text{HMAC}(k, m) = H(k\|H(k\|m))$$

Per verificare che HMAC resiste agli attacchi sopra descritti, consideriamo il seguente scenario: se un avversario richiede l'HMAC di un messaggio breve, viene calcolato $H(k\|m)$, producendo un'uscita σ. Tuttavia, l'attaccante vede solo l'output finale σ, **non** lo stato intermedio. Nel precedente attacco, l'attaccante poteva sfruttare il valore intermedio dello SHA-256 per **concatenare** nuovi blocchi. Con HMAC, questo non è possibile, poiché lo stato interno rimane protetto dalla **doppia applicazione della funzione hash**, impedendo all'attaccante di prolungare il messaggio.

6.3.6 Utilizzo delle CRHF per l'autenticazione di password:

Nei sistemi di autenticazione moderni, i server non memorizzano direttamente le password degli utenti, ma piuttosto il loro **hash**. Questo approccio offre una protezione in caso di violazione del database, poiché l'attaccante non ha accesso immediato alle password in chiaro.

Vulnerabilità Tuttavia, questo metodo presenta vulnerabilità. Un attaccante può preparare un *dizionario di password* (*rainbow table*), calcolando preventivamente gli hash di password comuni. In caso di violazione del database, può confrontare gli hash ottenuti con quelli presenti nel database compromesso, identificando così le password corrispondenti.

Mitigazione tramite salt Per contrastare questi attacchi, si utilizza una tecnica chiamata *salting*. Nel database non viene memorizzato semplicemente `H(password)`, ma una coppia:

`salt, H(salt‖password)`

dove `salt` è una stringa casuale generata per ciascun utente. Quando l'utente tenta di autenticarsi, il server:

1. Recupera il `salt` associato all'utente

[10] Proposto da Hugo Krawczyk.

2. Calcola: `H(salt ‖ password_inserita)`
3. Confronta il risultato con l'hash memorizzato

Questo metodo neutralizza l'efficacia delle rainbow table precomputate, poiché ogni password è hashata con un salt diverso. Un attaccante dovrebbe generare una tabella separata per ogni singolo utente, rendendo l'attacco computazionalmente proibitivo.

- **Limitazioni:** È importante notare che questa è una mitigazione, non una soluzione completa. In caso di intrusione avanzata, l'amministratore dovrebbe comunque disabilitare gli account compromessi per prevenire accessi non autorizzati mentre gli utenti cambiano le loro credenziali.

6.3.7 Cifrare i file: PBKDF2

Per cifrare i dati su disco, si utilizzano algoritmi di crittografia simmetrica come AES. È importante notare che funzioni hash come SHA-256 non sono algoritmi di cifratura e non possono essere utilizzate per decifrare dati.
Per utilizzare AES correttamente, è necessaria una chiave casuale di alta qualità. Utilizzare **direttamente una password come chiave per AES rappresenta una vulnerabilità** significativa, poiché AES richiede chiavi con distribuzione uniforme di casualità.
Per risolvere questo problema, si utilizza **PBKDF2** (*Password-Based Key Derivation Function 2*), un algoritmo standardizzato che genera chiavi crittografiche a partire da password. Un esempio concreto di utilizzo è nel protocollo WPA2 per le reti Wi-Fi:

```
DK = PBKDF2(PRF, Password, Salt, c, dkLen)
```

Ad esempio:

```
Key = PBKDF2(HMAC-SHA1, password, ssid, 4096, 256)
```

Dove:

- **P**: Password fornita dall'utente
- **S**: Salt (valore casuale pubblico, come l'SSID in WPA2)
- **C**: Numero di iterazioni (4096 nell'esempio), un parametro di difficoltà costante
- **len**: Numero di blocchi in output
- **PRF**: Funzione pseudo-random, solitamente un HMAC

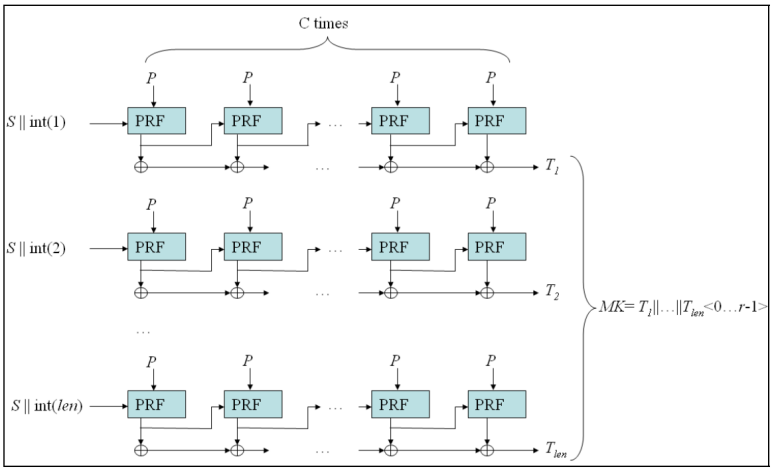


Figura 7: Schema di funzionamento di PBKDF2 con C iterazioni e *len* blocchi di output

Come mostrato in Figura 7, PBKDF2 ripete la PRF per C iterazioni, aumentando significativamente il costo computazionale necessario per un attacco a forza bruta. L'ultimo parametro (256 nell'esempio) indica la dimensione della chiave generata in bit. Dato che SHA-1 produce un output di 160 bit, PBKDF2 genererà più blocchi (160 bit per ogni riga nel diagramma) per raggiungere la dimensione richiesta.

6.3.8 Puzzle crittografico parametrizzato

Un puzzle crittografico parametrizzato permette di regolare dinamicamente la difficoltà computazionale richiesta per la sua soluzione. Il meccanismo funziona come segue:

> Sia $rand$ un valore casuale e Z un sottoinsieme dello spazio degli output della funzione hash. La dimensione di Z rappresenta il parametro di difficoltà: più piccolo è Z, più difficile sarà il puzzle.

Il puzzle consiste nel trovare un valore x tale che:

$$H(rand \| x) \in Z$$

Le caratteristiche principali sono:

- La verifica di una soluzione è estremamente veloce, trovarla invece è molto dispendioso;
- Trovare una soluzione richiede in media $\frac{|H|}{|Z|}$ tentativi, dove $|H|$ è la dimensione dello spazio degli output;
- La difficoltà è facilmente regolabile modificando la dimensione di Z.

Mining BTC Questa architettura è alla base del meccanismo di mining di Bitcoin, dove H è SHA-256 e Z è l'insieme di hash che iniziano con un certo numero di zeri. Per esempio, richiedere k bit iniziali a zero implica una difficoltà media di 2^k tentativi. Attualmente, il mining di Bitcoin richiede un numero di tentativi nell'ordine di $2^{90}/2^{100}$ per trovare un blocco valido, con la difficoltà che si auto-regola ogni 2016 blocchi per mantenere un tempo medio di 10 minuti tra i blocchi.

Originariamente per DDoS Storicamente, i puzzle parametrizzati sono stati proposti come contromisura agli attacchi DDoS: un server sotto attacco può richiedere ai client di risolvere un puzzle di difficoltà appropriata prima di elaborare le loro richieste, aumentando significativamente il costo dell'attacco.

Vediamo un esempio pratico di **puzzle crittografico parametrizzato**:

$$SHA256 \rightarrow 256 \text{ bit}$$

La funzione di hash `SHA-256` genera un output di 256 bit, quindi lo spazio dei possibili hash è costituito da 2^{256} valori distinti.
Se applichiamo `SHA-256` a un qualsiasi input, l'output sarà una stringa di 256 bit. Consideriamo ora un vincolo su questi bit:

- Se imponiamo che il primo bit dell'hash sia 0, lo spazio degli output validi si riduce della metà e la probabilità che un hash casuale soddisfi questa condizione è $\frac{1}{2}$ (scende da 1 a 1/2).
- Se imponiamo che i primi t bit dell'hash siano tutti 0, allora la probabilità di ottenere un hash valido è:

$$\left(\frac{1}{2}\right)^t$$

Questo concetto è alla base della difficoltà computazionale di alcuni puzzle crittografici, come il proof-of-work usato nelle criptovalute.

7 OpenSSL

Il tool che useremo è la libreria OpenSSL[11].

```
$ ./Configure --prefix=PATH     // percorso di installazione
$ make                          // compila
$ make install                  // installa
```

7.1 Randomicità

7.1.1 Rand

Vediamo come gestire i bit casuali. Proviamo questo PRNG (Pseudo-Random Number Generator):

```
$ openssl rand -out r.bin 5555
$ xxd r.bin                 // per visualizzarlo
```

```
000011a0: f608 fec1 c6c6 ab67 872a 83c5 9fe1 c364  .......g.*.....d
000011b0: 46ca a8ce baf4 7358 2be5 f04b a1b4 4604  F.....sX+..K..F.
000011c0: 47d6 e2c6 a807 f130 ebba a599 bf3e 611e  G......0.....>a.
000011d0: 9bc4 4015 7c1a a4b8 3dec 0177 17eb 741e  ..@.|...=..w..t.
000011e0: c901 14ee 65c4 414b 8b91 ecee b7c7 3f93  ....e.AK......?.
000011f0: b058 5c34 d35d 36cc b606 aa56 08b1 9ce2  .X\4.]6....V....
00001200: 906e 15a9 9190 9abd c436 c05b 06bb bce9  .n.......6.[....
00001210: 37e8 0634 b354 43f6 ec74 4ddf 6fdc a76a  7..4.TC..tM.o..j
00001220: b42b 97f6 5af1 1ce1 4641 c916 9fe2 f8f3  .+..Z...FA......
00001230: 4cc7 e88a 5e29 40a6 f792 2840 c943 7308  L...^)@...(@.Cs.
00001240: 8815 cc9b fda1 1656 22bc 3cee 420f 74f8  .......V".<.B.t.
00001250: 25d5 b78f 3860 f41d 0ec2 6c1a 68e6 107c  %...8`....l.h..|
00001260: 5ca5 21a7 81e5 a9a0 7c81 6d89 30dc fa7f  \.!.....|.m.0...
00001270: a8cc 8969 2b46 9f4a 2242 eef3 65f6 e181  ...i+F.J"B..e...
00001280: b80a 99fa c826 dc02 07c6 d2bd 7b4c a90b  .....&......{L..
```

Figura 8: Esecuzione di `openssl rand` da terminale

```
ilgalghi@Dell-Leo:~$ ls -lat r.bin
-rw-r--r-- 1 ilgalghi ilgalghi 5555 Mar 14 09:09 r.bin   // mostra la dimensione
    del file (5555 BYTE)
```

Per vedere il codice C che genera questi numeri casuali, consultare la directory `openssl/apps`.

[11] `https://www.openssl.org/source/`

7.2 Cifratura

Il tool per la cifratura in OpenSSL è `enc`. Per vedere le modalità disponibili:

```
openssl enc -list
```

Notare che sono presenti anche cifrari con vulnerabilità note. Lo standard moderno è AES.

7.2.1 AES-256-CTR

Per cifrare con AES-256 in modalità CTR:

- `-K` specifica la chiave esadecimale a 256 bit (64 caratteri).
- AES lavora con blocchi da 128 bit (32 caratteri esadecimali).
- `-iv` è il vettore di inizializzazione da 128 bit (necessario per CTR).
- `-out` indica il file cifrato.

In `test.txt` c'è il testo "testone":

```
openssl enc -e -aes-256-ctr -in test.txt -K  1234567890
    abcdef1234567890abcdef1234567890abcdef1234567890abcdef -iv 1234567890
    abcdef1234567890abcdef -out c.bin
```

Analizzando il comando, troviamo che:

- `-e`:
  Specifica che vogliamo eseguire la **cifratura** (encrypt). Se volessimo decifrare, useremmo `-d` (decrypt).
- `-aes-256-ctr`:
  Indica l'algoritmo di cifratura scelto:
  - `aes` → Algoritmo di cifratura AES (Advanced Encryption Standard).
  - `256` → Lunghezza della chiave: AES supporta 128, 192 e 256 bit (qui usiamo 256 bit).
  - `ctr` → Modalità di funzionamento (Counter Mode), che trasforma AES in uno stream cipher.
- `-in test.txt`:
  Specifica il file di input, ovvero il file in chiaro che vogliamo cifrare. In questo caso, `test.txt`.
- `-K 1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef`:
  Imposta la chiave di cifratura esadecimale a 256 bit (64 caratteri esadecimali, dove ogni carattere rappresenta 4 bit).
- `-iv 1234567890abcdef1234567890abcdef`:
  Definisce il vettore di inizializzazione (IV) a 128 bit (32 caratteri esadecimali).
  - L'IV è necessario in modalità CTR per generare un flusso di chiavi cifranti differenti a ogni esecuzione.
  - In altre modalità, come CBC, è usato per introdurre casualità nella cifratura.
- `-out c.bin`:
  Specifica il file di output che conterrà i dati cifrati. Qui il risultato verrà salvato in `c.bin`.

Verifichiamo il file cifrato:

```
ls -l c.bin   // Mostra dimensione e permessi del file
xxd c.bin     // Esamina il contenuto in esadecimale
```

7.2.2 Decifratura

Per decifrare, utilizziamo lo stesso comando con l'opzione `-d`:

```
openssl enc -d -aes-256-ctr -K 1234567890
    abcdef1234567890abcdef1234567890abcdef1234567890abcdef -iv  1234567890
    abcdef1234567890abcdef -in c.bin
```

uscirà il file che abbiamo precedentemente crittato.
NOTA: Se si modifica anche un solo bit della chiave, si osserva l'**avalanche effect (diffusione)**, ovvero un cambiamento drastico nell'output che porta a una decrittazione sbagliata.

7.2.3 AES-256-CBC

Proviamo ora la modalità di cifratura CBC:

```
openssl enc -e -aes-256-cbc -in test.txt -K 1234567890
    abcdef1234567890abcdef1234567890abcdef1234567890abcdef -iv 1234567890
    abcdef1234567890abcdef -out c2.bin
```

e otteniamo `c2.bin` di 16 byte.
Osserviamo che:

- `c.bin` ha una dimensione pari alla lunghezza del testo in chiaro più un eventuale piccolo padding. Questo perché CTR (Counter Mode) è una modalità di cifratura che funziona come un **stream cipher** (è *ibrido)*, cifrando i byte del file uno alla volta, quindi vengono cifrati tutti i 7 byte della parola `testone` + 1 byte necessario per completare un blocco da 128 bit.
- `c2.bin` ha una dimensione di 16 byte, perché CBC lavora a blocchi fissi di 128 bit e applica **padding** se necessario. Un blocco da 128 bit corrisponde a 16 byte, poiché ogni byte è composto da 8 bit. Quindi, se un file ha una dimensione di 16 byte, corrisponde esattamente a 1 blocco da 128 bit.

Se tentiamo di decifrare `c.bin` con CBC (era stato cifrato con CTR), otteniamo un errore:

```
bad decrypt
```

Questo accade perché i dati cifrati in CTR non contengono padding, mentre CBC lo richiede.

7.2.4 AES-256-ECB (insicuro)

Proviamo ora AES in modalità ECB:

```
openssl enc -e -aes-256-ecb -in test.txt -K \
1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef -out c3.bin
```

Usando lo stesso comando di prima e cambiando solo `-aes-256-ecb`, ci viene detto che non serve il vettore di inizializzazione (IV) perché ECB è una **cifratura deterministica**: lo stesso input produrrà sempre lo stesso output cifrato. Questa prevedibilità lo rende insicuro per dati sensibili.

Vulnerabilità con AES-256-ECB Proviamo una vulnerabilità usando il testo

`"testonetestina22testonetestina22"`

che è lungo 16+16+1 byte.
Notiamo che, cifrando il testo con AES-256 in modalità `ECB`, nel ciphertext ci saranno dei pattern ripetuti. Questo è un comportamento che deve essere evitato, poiché la modalità ECB è deterministica e non usa un vettore di inizializzazione (`IV`), quindi blocchi di testo uguali generano blocchi di ciphertext identici e rilevabili.
Nel nostro caso, avremo 3 blocchi di 16 byte ciascuno, per un totale di 48 byte. La cosa interessante è che i primi 16 byte del ciphertext sono identici ai successivi 16 byte, creando una ripetizione evidente.

```
openssl enc -e -aes-256-ecb -in test.txt -K 1234567890
    abcdef1234567890abcdef1234567890abcdef1234567890abcdef -out c3.bin
```

Osserviamo infatti che il pattern si ripete, visualizzando il file cifrato con `xxd`:

```
00000000: 8da3 0259...
00000010: 8da3 0259...
```

Questo comportamento è una **vulnerabilità** della modalità `ECB` che non maschera le ripetizioni nei dati. In applicazioni crittografiche moderne, è altamente sconsigliato usare la modalità `ECB` per la cifratura di dati sensibili.

7.2.5 Test di velocità del cipher

```
openssl speed name-cipher  // Esegue un test di velocita
```

7.2.6 Lunghezza chiavi e IV

Circa la lunghezza delle chiavi e del vettore di inizializzazione (IV):

- Se la chiave o l'IV sono troppo corti, vengono allungati con una sequenza di bit zero, il che rappresenta una **grave vulnerabilità**.
- Se la chiave o l'IV sono troppo lunghi, vengono troncati, ignorando i bit in eccesso.
- La lunghezza della chiave è determinata dal numero che segue "-aes-".
- La lunghezza dell'IV è sempre di 128 bit, in quanto questa è la dimensione del blocco dati gestito da AES, indipendentemente dalla lunghezza della chiave.

Authenticated Encryption Non è previsto l'**Authenticated Encryption** (AE) in questo caso, poiché questi tool forniscono l'output man mano che sono calcolati. Con AE, il tag MAC (Message Authentication Code) viene calcolato solo alla fine, per garantire sia la cifratura che l'integrità dei dati. In questo caso, stiamo solo cifrando i dati, senza garantire l'integrità.

7.3 OpenSSL-MAC

OpenSSL può essere utilizzato per calcolare il Message Authentication Code (MAC), incluso il GMAC, un MAC utilizzato nelle modalità AEAD.

```
openssl mac -in test.txt -cipher AES-128-GCM -macopt hexkey
    :01234567890123456789012345678901 -macopt hexiv
    :01234567890123456789012345678901 GMAC
```

L'output consiste nel MAC relativo alla modalità AEAD, ma con Associated Data (AD) vuoto. Combinando questi strumenti, è possibile creare cifrature e MAC robusti.

7.3.1 MAC e HASH

Per calcolare un HMAC (Hash-based Message Authentication Code) con un algoritmo di hash come SHA-256, utilizziamo il seguente comando:

```
openssl mac -in fileinput -digest sha256 -macopt hexkey:1234567890
    abcdef1234567890abcdef HMAC
```

Il comando `openssl list -digest-commands` restituisce la lista delle funzioni di hash disponibili per HMAC.
Si noti che non è necessario un IV (Initialization Vector) per HMAC (essendo SHA-256). Non ci sono vincoli specifici sul numero di caratteri per la `hexkey`, ma è richiesto che la lunghezza della chiave sia pari.

MD5 Negli anni '90, MD5 (da 128 bit) era comunemente usato per HMAC, ma essendo vulnerabile a collisioni, è stato sostituito prima da SHA-1 e successivamente da SHA-256 per una maggiore sicurezza.

7.3.2 Generazione di chiavi a partire da password - PBKDF2

Poiché le password, per loro natura, non sono sufficientemente sicure per essere utilizzate direttamente nelle operazioni crittografiche (in quanto non sono casuali, una caratteristica **fondamentale** per la sicurezza delle chiavi), è necessario derivare una chiave sicura a partire dalla password. A tale scopo, lo standard più utilizzato è PBKDF2 (Password-Based Key Derivation Function 2). PBKDF2 applica una funzione di derivazione della chiave e genera una chiave robusta che può essere utilizzata per la cifratura. In altre parole, PBKDF2 consente di "trasformare" una password in una chiave crittografica sicura.
Ecco come cifrare un file con una password usando PBKDF2:

```
openssl enc -e aes-256-ecb -in test.txt -k uniroma1 -pbkdf2 -out c3.bin
```

Dove `uniroma1` è la chiave. In caso di password sbagliata, viene riportato errore.

8 Crittografia asimmetrica

8.1 Limiti della crittografia simmetrica

La crittografia simmetrica presenta diversi limiti:

- Con n utenti, sono necessarie circa n^2 chiavi univoche.
- Maggiore è il numero di chiavi, più difficile diventa conservarle in modo sicuro.
- Le chiavi devono essere scambiate tramite canali sicuri o corrieri fidati.
- Se esiste una fiducia comune verso un *Key Distribution Center* (KDC), allora è sufficiente avere una chiave sicura condivisa con il KDC.
  - Il KDC diventa un *single point of failure*[12].

Questo approccio veniva utilizzato per la distribuzione delle chiavi senza dover incontrarsi fisicamente.

La crittografia simmetrica è limitata, e con il passare degli anni sono stati sviluppati metodi più avanzati. Negli anni '70, i migliori algoritmi erano quelli privati, mentre quelli pubblici venivano trattati dalle agenzie. Oggi, la situazione si è invertita.

Una domanda fondamentale nel campo della sicurezza è se sia possibile comunicare in modo sicuro attraverso un canale pubblico senza mai incontrarsi fisicamente:

> *Possono Sender e Receiver comunicare in modo sicuro usando un canale pubblico senza mai essersi incontrati prima?*

8.1.1 Merkle puzzle: distribuzione della chiave in modo sicuro senza incontro fisico

Nel 1974, Merkle propose un metodo per distribuire una chiave segreta in modo sicuro utilizzando un canale pubblico. Questo metodo è conosciuto come il *Merkle puzzle*.

Funzionamento Consideriamo (per ora) un **avversario passivo** (origlia sul canale ma non modifica nulla). Il processo funziona come segue:

1. Nel metodo proposto da Merkle, il sender invia al receiver una cifratura di una chiave segreta grande, rappresentata come $\alpha = \text{Enc}(y_1||k_1), \ldots, \alpha_n = \text{Enc}(y_n||K_n)$. Le chiavi $k_1, \ldots, k_n$ sono abbastanza piccole, mentre la lunghezza di ogni α_i è molto inferiore alla lunghezza di k_i, ovvero $|\alpha_i| \ll |k_i|$.

[12] Un componente vulnerabile in un sistema che, se fallisce, può causare il fallimento dell'intero sistema

2. Il receiver, scelto un numero a caso j tra 1 e n dall'insieme J, esegue un attacco a forza bruta per decifrare la j-esima cifratura, ottenendo così K_j e Y_j. Questo attacco è possibile perché la chiave k_j è relativamente piccola. Una volta che il receiver riesce a ottenere la chiave k_j, rimanda una nuova cifratura con la chiave k_j (più grande) al sender, cioè $\text{Enc}_{k_j}(y_j)$.
3. Poiché il receiver impiega tempo per il brute force, il sender, avendo a disposizione tutte le possibili chiavi, è in grado di provare ogni chiave e individuare k_j.

Tuttavia, il *Merkle puzzle* non è mai stato implementato praticamente a causa di due motivi principali:

1. La necessità di eseguire *comunque* un brute force per ogni possibile chiave.
2. Un avversario potrebbe comunque riuscire a eseguire un attacco brute force su tutte le possibili chiavi e ottenere l'intero contenuto.

Nonostante ciò, il *Merkle puzzle* ha avuto una grande importanza per lo sviluppo della crittografia asimmetrica e ha aperto la strada a nuovi metodi di distribuzione sicura delle chiavi.

8.2 Nozioni di base

Un gruppo $(G, \circ)$ è un insieme di elementi G ed un'operazione binaria $\circ$ su di essi tale che:

- **Chiusura**: Se $a, b \in G$, allora $a \circ b \in G$.
- **Elemento identità**: Esiste un elemento $i \in G$ tale che $a \circ i = a = i \circ a$ per ogni $a \in G$. L'elemento i è unico.
- **Elemento inverso**: Per ogni $a \in G$, esiste un elemento $b \in G$ tale che $a \circ b = i$. L'elemento b è unico.
- **Associatività**: Per ogni $a, b, c \in G$, vale la proprietà $(a \circ b) \circ c = a \circ (b \circ c)$.
- **Commutatività** (se G è abeliano): Per ogni $a, b \in G$, si ha $a \circ b = b \circ a$.
- Se $(G, \circ)$ è un gruppo, H è un sottoinsieme di G e $(H, \circ)$ è un gruppo, allora H è un **sottogruppo** di G. *Esempio:* $\{i\}$ è un sottogruppo di G.
- Un gruppo $(G, \circ)$ è un **gruppo finito** se G è un insieme finito.
- *Esempio:* $\mathbb{Z}_n$ con l'operazione di addizione modulo n, ovvero $(\mathbb{Z}_n = \{0, \ldots, n-1\}, +)$, è un gruppo abeliano finito con n elementi.
- **Proprietà di** $(\mathbb{Z}_n, +)$:
  - **Chiusura**: Se $a, b \in \mathbb{Z}_n$, allora $a + b \mod n \in \mathbb{Z}_n$.
  - **Elemento identità**: Esiste $0 \in \mathbb{Z}_n$ tale che:
    $$a + 0 \mod n = a \mod n = 0 + a \mod n, \quad \forall a \in \mathbb{Z}_n.$$
  - **Elemento inverso**: Per ogni $a \in \mathbb{Z}_n$, esiste $b = -a \mod n$ (cioè $n - a$) tale che:
    $$a + b \equiv 0 \mod n.$$
  - **Associatività**: Per ogni $a, b, c \in \mathbb{Z}_n$,
    $$(a + b \mod n) + c \mod n = a + (b + c \mod n) \mod n.$$
  - **Commutatività**: Per ogni $a, b \in \mathbb{Z}_n$,
    $$a + b \mod n = b + a \mod n.$$

- Espressione compatta della somma ripetuta:

  $a+a+a+\cdots+a$ (6 volte) $\mod n$ si denota con $6a \mod n$, $0a \mod n = 0 \mod n$.

Gruppo Moltiplicativo Modulo p

- *Esempio:* $\mathbb{Z}_p^* = (\{1, \ldots, p-1\}, *)$ con p primo è un gruppo abeliano finito di $p-1$ elementi rispetto all'operazione di moltiplicazione modulo p.
- **Proprietà di** $(\mathbb{Z}_p^*, \cdot)$:
  - **Chiusura**: Se $a, b \in \mathbb{Z}_p^*$, allora $a \cdot b \mod p \in \mathbb{Z}_p^*$.
  - **Elemento identità**: Esiste $1 \in \mathbb{Z}_p^*$ tale che:
    $$a \cdot 1 \mod p = a \mod p = 1 \cdot a \mod p, \quad \forall a \in \mathbb{Z}_p^*.$$
  - **Elemento inverso**: Per ogni $a \in \mathbb{Z}_p^*$, esiste $b \in \mathbb{Z}_p^*$ tale che:
    $$a \cdot b \equiv 1 \mod p.$$
  - **Associatività**: Per ogni $a, b, c \in \mathbb{Z}_p^*$,
    $$(a \cdot b \mod p) \cdot c \mod p = a \cdot (b \cdot c \mod p) \mod p.$$
  - **Commutatività**: Per ogni $a, b \in \mathbb{Z}_p^*$,
    $$a \cdot b \mod p = b \cdot a \mod p.$$
  - Espressione compatta della moltiplicazione ripetuta:

    $a \cdot a \cdot a \cdots a$ (6 volte) $\mod p$ si denota con $a^6 \mod p$, $a^0 \mod p = 1 \mod p$.
  - Proprietà delle potenze:
    $$a^c \cdot b^c \mod p = (a \cdot b)^c \mod p, \quad a^c \cdot a^d \mod p = a^{c+d} \mod p.$$
  - Per comodità, possiamo omettere il simbolo $\cdot$.

Proprietà dei Gruppi Finiti

- Se $(G, \circ)$ è un gruppo finito di ordine m, allora per ogni $a \in G$, se applico l'operazione $\circ$ $m-1$ volte (coinvolgendo m volte l'elemento a), ottengo l'elemento identità i. Inoltre, per ogni $a \in G$, vale che effettuare l'operazione $\circ$ $x-1$ volte è equivalente a farla $(x-1) \mod m$ volte.
- *Esempio in* $\mathbb{Z}_p^*$ con p primo:
  $$\mathbb{Z}_p^* = (\{1, \ldots, p-1\}, \cdot)$$
  - Per ogni $a \in \mathbb{Z}_p^*$, vale:
    $$a^{p-1} \equiv 1 \mod p, \quad a^x \equiv a^{x \mod (p-1)} \mod p.$$
- *Esempio in* $\mathbb{Z}_n$ con somma modulo n:
  $$\mathbb{Z}_n = (\{0, \ldots, n-1\}, +)$$
  - Per ogni $a \in \mathbb{Z}_n$, vale:
    $$a \cdot n \equiv 0 \mod n, \quad a \cdot x \equiv a \cdot (x \mod n) \mod n.$$
- Un gruppo $(G, \circ)$ finito di ordine m è detto **ciclico** se esiste un elemento $g \in G$ tale che:
  $$\langle g \rangle = G,$$
  dove $\langle g \rangle$ è l'insieme degli elementi ottenuti iterando l'operazione $\circ$ su g. In tal caso, g è detto un **generatore** di G.
- In generale, se $(G, \circ)$ è un gruppo finito di ordine m, allora $|\langle g \rangle|$ è un divisore di m.
- Se $(G, \circ)$ è un gruppo finito di ordine m e m è primo, allora $(G, \circ)$ è **ciclico** e tutti gli elementi diversi dall'identità sono generatori.

8.3 Discrete Logarithm (DLog) Problem

- È una delle più importanti *one-way functions*[13].
- Non è **post-quantum sicura** (a differenza di AES-256 e SHA-256).
- Si considera un gruppo ciclico di ordine q (dove q ha n bit) con generatore g. Si sceglie un elemento casuale h nel gruppo. La sfida è trovare il logaritmo di h in base g, ossia determinare x tale che:
$$g^x = h.$$
Non si riesce quindi a trovare x a partire da g e/o h, è computazionalmente difficile.

  Non facilmente invertibile Il problema del logaritmo discreto è difficile perché, a differenza dell'aritmetica continua, nei **gruppi ciclici**[14] non esiste un'operazione analoga al *logaritmo* che permetta di invertire direttamente l'esponenziale. Sebbene sia possibile trovare x con la *forza bruta*, i metodi avanzati riducono la complessità dell'attacco da esponenziale a sub-esponenziale (ad esempio, l'algoritmo di *index calculus*).

- **In concreto**:
  - Si sceglie un primo casuale q di n bit tale che $p = 2q + 1$ sia primo. Se non lo trovo, ripeto (i numeri primi sono facilmente individuabili perche sono densi).
  - Si prende g come generatore di G_q, il sottogruppo di $\mathbb{Z}_p^*$ con q elementi.
  - Si sceglie un elemento casuale $x \in \mathbb{Z}_q$ e si calcola:
$$g^x \equiv h \mod p.$$
  - La sfida per l'avversario è trovare x, avendo in input solo p, q, g, h.

8.4 L'avvento delle curve ellittiche (ECC)

- Il problema del logaritmo discreto, nelle sue applicazioni tradizionali basate su gruppi $\mathbb{Z}_p^*$ che usano aritmetica modulare, richiede tempo sub-esponenziale. In particolare, con un primo p di n bit, il tempo richiesto per risolvere il problema è $2^{O(n^{\frac{1}{3}} \log n^{\frac{2}{3}})}$.
- Le curve ellittiche forniscono un'alternativa potente: nei gruppi definiti da curve ellittiche, la difficoltà di risolvere il problema del logaritmo discreto è esponenziale in n, ma più difficile da attaccare rispetto ai gruppi $\mathbb{Z}_p^*$.
- Concretamente, una chiave di 256 bit su curve ellittiche offre la stessa sicurezza di una chiave di 3000 bit in un gruppo $\mathbb{Z}_p^*$. Questo rende le curve ellittiche più efficienti dal punto di vista della memorizzazione e della comunicazione, mantenendo un alto livello di sicurezza.

La sicurezza della crittografia a curva ellittica si basa sulla difficoltà di risolvere il cosiddetto **problema del logaritmo discreto a curva ellittica**. Dato un punto P sulla curva e uno scalare k, è molto difficile determinare il punto Q tale che $Q = k * P$, molto più difficile di quanto non lo sia fattorizzare un numero molto grande.

[13] Una funzione unidirezionale (*one-way function* o OWF) è una funzione matematica "facile da calcolare" ma difficile da invertire.

[14] Un gruppo ciclico è un insieme G con un'operazione binaria (come la moltiplicazione o l'addizione) tale che esiste un elemento generatore g che può generare tutti gli elementi del gruppo tramite ripetute applicazioni dell'operazione.

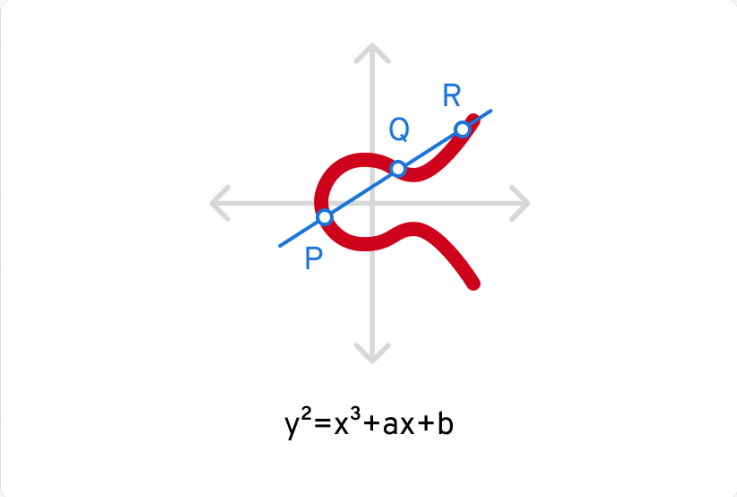


Figura 9: Sicurezza della curva ellittica

8.5 Tabelle riepilogative

8.5.1 CCA-secure vs CPA-secure

La differenza principale tra CPA-secure e CCA-secure è la capacità dell'attaccante di interagire con il sistema di crittografia. Ricordando che:

- **CPA-Secure (Chosen Plaintext Attack Secure):** La sicurezza CPA implica che un sistema di crittografia è sicuro contro un attacco in cui l'attaccante può ottenere cifrature di testi in chiaro scelti. Tuttavia, l'attaccante non può interagire con il sistema dopo aver ricevuto la cifratura, ossia non ha accesso alla decifrazione di testi cifrati.
- **Chosen-Ciphertext Attack (CCA)**: oltre a poter scegliere i plaintext (come CPA), l'avversario può fornire ciphertext arbitrari per farli decifrare e osservare il risultato. Tuttavia, **non è in grado di generare un ciphertext valido che venga accettato dal sistema**. Se potesse farlo, potrebbe sfruttare le risposte del receiver per ricostruire informazioni sulla chiave o sul messaggio originale.

Nel contesto della crittografia, le definizioni di sicurezza CPA e CCA si riferiscono alla resistenza di un sistema di crittografia rispetto a diversi tipi di attacchi.

Proprietà	**CPA-secure (Chosen Plaintext Attack)**	**CCA-secure (Chosen Ciphertext Attack)**
Definizione	Lo schema è sicuro anche se l'attaccante può ottenere i ciphertext relativi a plaintext a sua scelta.	Lo schema è sicuro anche se l'attaccante può ottenere la decifratura di ciphertext a sua scelta (escluso il challenge ciphertext).
Modello di attacco	L'attaccante ha accesso a un oracolo di cifratura: può inviare plaintext e ricevere i ciphertext corrispondenti.	L'attaccante ha accesso sia a un oracolo di cifratura sia a un oracolo di decifratura (tranne per il ciphertext del challenge).
Sicurezza	Meno forte di CCA-secure. Protegge contro attacchi passivi e scelti sui plaintext, ma non contro manipolazioni attive del ciphertext.	**Più forte di CPA-secure**. Protegge anche contro attacchi attivi, in cui l'attaccante può manipolare ciphertext e osservare le decifrazioni.
Resistenza	Non resiste ad attacchi in cui l'attaccante può modificare ciphertext e osservare la reazione del sistema.	Resiste a tentativi dell'attaccante di sfruttare l'oracolo di decifratura per ottenere informazioni sul plaintext.
Applicabilità	Adeguato per ambienti dove non è possibile interagire con un sistema di decifratura (es. dati statici cifrati).	Necessario in scenari dove l'attaccante può interagire con un componente che decifra (es. protocolli di rete, smart card).

Tabella 1: Confronto tra CPA-secure e CCA-secure

8.5.2 Tabella confronto Block Ciphers

Proprietà	**ECB**	**CBC**	**CTR**
Come funziona	Cifra ogni blocco di testo in chiaro indipendentemente con la stessa chiave.	Ogni blocco di testo in chiaro è XORato con il blocco cifrato precedente. Il primo blocco usa un IV.	Un contatore viene cifrato e combinato con ogni blocco di testo in chiaro tramite un'operazione XOR.
Caratteristiche	Semplice, ma vulnerabile a pattern nei dati cifrati.	Cifra i blocchi concatenandoli (chaining).	Utilizza un contatore che varia ad ogni blocco.
Vulnerabilità	Vulnerabile a pattern nei dati cifrati.	Sensibile agli errori nei blocchi precedenti.	Può essere vulnerabile se il contatore è riutilizzato.
Parallellismo	Parallelo.	Encryption **non è parallelo**, decryption può essere in parallelo.	Parallelo.
Usi comuni	Cifratura di piccoli blocchi di dati, **mai raccomandato** per messaggi confidenziali.	Cifratura di file o flussi di dati.	Cifratura ad alta velocità, ideale per lo streaming.
Sicurezza	**Non CPA-secure**, vulnerabile a pattern nei dati cifrati.	**CPA-secure**, grazie all'uso dell'IV **casuale** mai riutilizzato e del chaining.	**CPA-secure**, utilizza un contatore unico che previene pattern ripetuti.

Tabella 2: Confronto tra block cipher ECB, CBC e CTR

No CCA-secure Notare che così **nessuno di questi è CCA-secure**: per ottenere CCA-secure occorre **introdurre un'autenticazione**: le modalità CBC e CTR devono essere autenticate (es. con Encrypt-then-MAC o AEAD, come GCM o ChaCha20-Poly1305). Se si vogliono implementazioni **CCA-secure direttamente**, usa cifrature AEAD (Authenticated Encryption with Associated Data) come AES-GCM.

8.6 Diffie-Hellman

- Nel 1976, Diffie e Hellman introdussero un protocollo rivoluzionario per la **crittografia asimmetrica**. (Nota: Hellman era il supervisore di dottorato di Merkle).
- Si considera un **gruppo finito ciclico** di ordine primo con generatore g.
- Il partecipante P_1 sceglie un valore segreto x e invia $A = g^x$.
- Il partecipante P_2 sceglie un valore segreto y e invia $B = g^y$.
- Entrambi calcolano la chiave condivisa $K = g^{xy}$.
  - Il protocollo è utilizzabile per la **cifratura** e per la generazione di **Message Authentication Code (MAC)**.
- È un **protocollo di accordo di chiavi** (key exchange).
- La sua sicurezza si basa su due assunzioni fondamentali:
  1. **Computational Diffie-Hellman Assumption (CDH)**: Dato un gruppo ciclico a caso di ordine primo q con generatore g, e dati due elementi $h_1 = g^{x_1}$ e $h_2 = g^{x_2}$, il problema è determinare $g^{x_1 x_2}$ conoscendo solo g, h_1 e h_2.
  2. **CDH con random oracle o Decisional Diffie-Hellman (DDH)**, sono varianti di Dlog:
     - **CDH con Random oracle:** Si applica una Key-Derivation Function (KDF) che può essere modellata come un Random Oracle, ad esempio con SHA-256
     - **DDH:** (con conversione mediante key-derivation function) Variante di CDH in cui si deve determinare se un valore fornito sia effettivamente $g^{x_1 x_2}$ o un elemento casuale del gruppo.

8.6.1 Spiegazione Assunzione Computazionale DH

Assumiamo un avversario solamente passivo che opera in internet su una rete *Client - Internet - Server*:

$$\text{Cliente} \longleftrightarrow \text{Internet (Avversario)} \longleftrightarrow \text{Server}$$

Tutto quello che sa il client e il server lo sa anche l'avversario. La crittografia pubblica stabilisce che il server crea un'identità pubblica locale, rappresentata da una stringa di bit P_k, e la può comunicare in chiaro a tutti. Questa stringa viene generata tramite $Gen(1^n)$, che dà la chiave pubblica P_k e genera anche la chiave privata S_k.

Confidenzialità *Come può il client comunicare in modo sicuro con il server?* Supponiamo che il client abbia un messaggio m che desidera inviare al server. Conoscendo P_k, il client può eseguire la cifratura simmetrica $Enc(P_k, m) = c$, dove c è il ciphertext che invia al server. L'avversario può vedere il ciphertext c, ma essendo cifrato non può decifrarlo. Il server, quando riceve c, può decifrare il messaggio con $Dec(S_k, c)$.

Diffie e Hellman hanno pensato che per cifrare m debbano usare la funzione $c' = AES(m, IV, K)$, quindi cifratura simmetrica con una chiave $k \leftarrow \{0,1\}^n$, dove IV è il vettore di inizializzazione. Pertanto, IV e c' devono essere inviati, ma per decifrare è necessaria la chiave K. Questo porta alla necessità di **risolvere il problema di come trasferire in modo sicuro la chiave con l'identità digitale, senza essersi mai incontrati.**

$$C' = AES(m, IV, k) \overset{(IV, C', ???)}{\longrightarrow} \quad \text{(Server)}$$

8.6.2 Scambio di chiavi

Come possono il client e il server mettersi d'accordo sulla chiave? Questo processo è noto come **accordo di chiavi** o **scambio di chiavi** (key exchange). In particolare, abbiamo due utenti, P1 e P2, che **devono concordare su una chiave tramite un canale pubblico**.

Problema logaritmo discreto Per farlo, utilizzano un algoritmo di scambio di chiavi che si basa sul problema del *logaritmo discreto*.

$$\text{P1} \longleftrightarrow \text{P2}$$

P2, che sta eseguendo $Gen(1^n)$, invia G, g, y, dove dà in output:

- G è il gruppo generato,
- g è il generatore del gruppo G,
- x è l'elemento privato
- $y = g^x$ è l'elemento pubblico.

La caratteristica fondamentale è che $g^x = y$, quindi x è il logaritmo discreto, un valore ben nascosto, scelto da $Z_q = 0, .., q-1$ con $q = |\langle g \rangle|$ (grandezza del gruppo generato da g), mentre $G =< g >$ gruppo generato da g, e dove q è un numero primo di n bit. Ogni bit che si aggiunge a q raddoppia il valore del numero.
P2 ha quindi:

- **Chiave pubblica** (P_k): G, g, y
- **Chiave segreta** (S_k): G, g, x, y

P2 invia solo P_k, che arriva a destinazione inalterato. A questo punto, P1 conserva P_k e sceglie un valore casuale $u \in Z_q$, quindi calcola $V = g^u$, che è ancora un elemento del gruppo. Il valore di u viene mantenuto segreto, mentre V viene inviato a destinazione.

Con questi due messaggi, entrambi i partecipanti possono calcolare la stessa chiave segreta, perché $P1$ calcola la chiave che è la stessa che calcola $P2$, come segue:

- P1 riceve y e calcola y^u, che chiamerà K.
- P2 riceve V e calcola $V^x = K$.

Notiamo che i due valori di K sono uguali, ovvero K è lo stesso nonostante gli input differenti. Vediamo perché:

- Per P1, abbiamo:

$$K = y^u = g^{xu}, \quad \text{ma} \quad g^u = V, \quad \text{quindi} \quad V^x = g^{xu} = K$$

L'avversario non ha accesso alle chiavi segrete e vede solo istanze del problema, ma non è in grado di completare il calcolo poiché gli mancano degli elementi. L'avversario vede y e V, ma non può calcolare la chiave g^{xu} perché non possiede né la chiave privata di P1 né quella di P2 (non possiede nessuno dei due logaritmi discreti).

In teoria, potrebbe esserci un avversario che riesce a calcolare la chiave in modo efficiente (ovvero potrebbe trovare g^{xu}), ma fino ad oggi, dopo più di 50 anni, l'algoritmo Diffie-Hellman non è stato rotto. Questo algoritmo quindi si basa sull'assunzione del problema del logaritmo discreto (ne è un sottogruppo), che è computazionalmente difficile. Se un attaccante riuscisse a risolvere il logaritmo discreto, potrebbe anche rompere Diffie-Hellman. Se si riesce a rompere DH, si potrebbe invece non-rompere il logaritmo discreto (poiché insieme più grande). Al momento però entrambi sono considerati sicuri.

Accordo di chiavi Nel processo di scambio di chiavi, P2 (server) ha la sua chiave pubblica che è appunto pubblica. A questo punto, P1 richiede da internet la chiave P_k di P2, e quindi basta che P1 invii V a P2. Con solo l'invio di V si completa quindi questo scambio, ed entrambi i partecipanti sono in grado di concordare sulla chiave segreta.
Il messaggio finale inviato sarà quindi:

$$C' = AES(m, IV, k) \overset{(IV, C', V)}{\longrightarrow} \quad \text{(Server)}$$

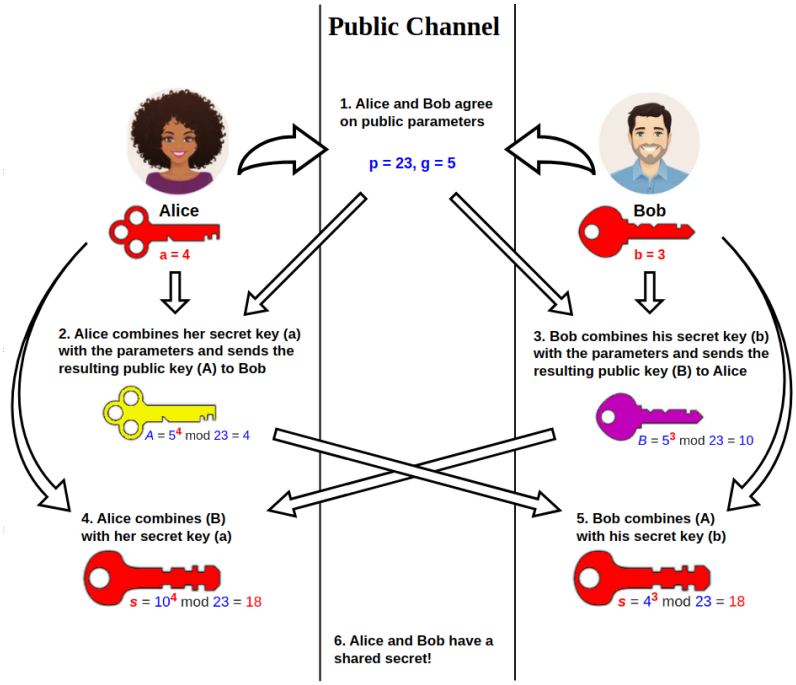


Figura 10: Esempio con Alice e Bob

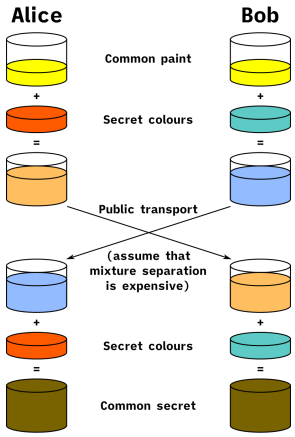


Figura 11: Esempio con colori

8.6.3 Errore: chiave casuale

C'è un errore nel passaggio precedente. Come abbiamo detto, la chiave K deve essere casuale, ma nel modello attuale non è garantito che lo sia: se calcoliamo la chiave con DH questa non è di per sé casuale (anche se comunque difficilmente individuabile), non va bene quindi scegliere $k \leftarrow PK^u$. Considerando che si sta usando AES, se non viene passata una stringa casuale (richiesta da manuale) **non si può garantire la sicurezza**. Ad esempio, se l'avversario conosce alcuni bit dei numeri, potrebbe essere in grado di predire almeno parzialmente la chiave.

La CDH (Computational Diffie-Hellman) assicura che l'avversario non può calcolare la chiave, ma per utilizzare AES è necessaria una chiave completamente casuale. Come possiamo passare da un valore impredicibile (come se fosse una password) a un valore casuale?

PBKDF2 Una possibile soluzione è utilizzare PBKDF2, una funzione di derivazione di chiavi che genera una chiave da una password (anche se la password non è nota). Tuttavia, in questo caso non abbiamo nemmeno una password da utilizzare, quindi non occorrono tutti quei round necessari classici della iterazione C. Se si ha una buona sorgente di casualità, non è necessario passare attraverso tutti i passaggi di PBKDF2, e possiamo ottenere una stringa casuale direttamente.

Random oracle Questo modello va bene anche col random oracle model di SHA256. Nella tabella virtuale di SHA, infatti, l'avversario non riuscirà mai a fare la stessa domanda richiesta dall'utente e quindi non riuscirà mai ad avere come ritorno la stessa stringa casuale dell'utente.

Key Derivation Function (KDF) Poiché non abbiamo bisogno di una password completa, possiamo utilizzare una Key Derivation Function (KDF) per derivare una chiave in modo sicuro senza la necessità di PBKDF2.

8.6.4 Decisional Diffie-Hellman Function (DDH)

La chiave k è così ben nascosta che, anche se l'avversario vede k, non è in grado di determinare che si tratta effettivamente di k, poiché potrebbe essere semplicemente un elemento casuale del gruppo. È come se la chiave fosse praticamente casuale. Questa è una delle assunzioni alla base

della sicurezza di Diffie-Hellman: l'avversario ha più informazioni, ma non riesce a sfruttarle per risolvere il problema, ricordando sempre che è possibile rompere il decisional Diffie-Hellman (DDH) senza necessariamente rompere il problema del logaritmo discreto, ma si assume che questo non sia facilmente realizzabile.

9 Crittografia a chiave pubblica

Cifratura a chiave pubblica La cifratura viene effettivamente eseguita con chiavi pubbliche? In teoria, esistono metodi per cifrare direttamente con chiavi pubbliche senza passare per un accordo di chiavi. Ci sono vari algoritmi che consentono questo tipo di cifratura, ma sono relativamente meno comuni.
Ad oggi, la cifratura a chiave pubblica (come RSA) viene utilizzata principalmente per due motivi:

- Se sono necessarie proprietà aggiuntive rispetto alla cifratura simmetrica (intesa come AES), come nel caso della cifratura omomorfica o in applicazioni di nicchia.
- Un tempo veniva utilizzata per l'accordo di chiavi, poiché consente di stabilire una chiave condivisa che è conosciuta solo dal client e dal server. Se si utilizza la cifratura a chiave pubblica direttamente, il client invia la sua chiave pubblica al server, ma questa modalità è stata utilizzata principalmente prima dell'introduzione di Diffie-Hellman, che ha successivamente surclassato questa tecnica.

Analizziamo meglio nella sezione successiva.

9.1 DH e RSA

9.1.1 Diffie-Hellman e Man-in-the-Middle Attack

Un attacco man-in-the-middle (MITM) nel protocollo Diffie-Hellman espone vulnerabilità critiche nel key exchange:
Scenario:

- Client (C) e Server (S) tentano di stabilire una chiave condivisa
- Avversario (A) intercetta e manipola la comunicazione

Fasi dell'attacco Le fasi di un possibile attacco sono quindi:

1. Il server invia al client il parametro P_k.
2. Il client invia un valore V al server.
3. **Avversario intercetta questi valori**
4. Attaccante inietta parametri contraffatti:
   - Invia P'_k al client con $g = s^u$
   - Invia $v' = g^t$ al server

L'avversario conosce sia t che s, poiché li sceglie.

Calcolo delle chiavi Ricordando che $v = g^n$ e $PK = g^x$, ora accade che se il client vuole calcolare la chiave, calcola $k = pk'^u$ (contraffatto dall'avversario) e il server calcola con V'^x (contraffatto dall'avversario). Si ha quindi che:

- Client calcola: $K = P'_K = g^{su}$
- Il server calcola: $g^{tx} = V'^x$

L'avversario, conoscendo sia t che s, può facilmente calcolare:

- la chiave $K = V^s = g^{su}$.
- la chiave $k' = P_K^t = g^x t$.

L'avversario può quindi **decifrare l'intera conversazione**.

Lettura completa Se l'avversario è attivo, può compromettere il protocollo di scambio delle chiavi, ingannando così l'accordo sulle chiavi. In questo modo, l'avversario potrebbe decifrare e leggere tutta la conversazione, senza che il client e il server notino alcuna differenza (rimandano la conversazione originaria, ma leggendola).

Vulnerabilità Pertanto, *Diffie-Hellman* risulta essere vulnerabile a un attacco man-in-the-middle quando l'avversario è attivo. Per garantirne la sicurezza in scenari reali, è necessario rafforzare il protocollo, ad esempio tramite l'uso di firme digitali o certificati, per autenticare i partecipanti e prevenire questo tipo di attacco.

9.1.2 DH e notazione di Public Key Cryptography

- Possiamo integrare in questo caso anche gli attacchi CPA (Chosen Plaintext Attack) e CCA (Chosen Ciphertext Attack).
- Firma digitale (*digital signature*).

9.1.3 Alba e caduta di RSA (1978)

RSA è ancora oggi utilizzato. La tecnica RSA è una forma alternativa al logaritmo discreto e rappresenta un metodo per la **cifratura diretta**. Poco dopo lo sviluppo di Diffie-Hellman, RSA ha anche introdotto la firma digitale. Il fatto che questa tecnica possa svolgere entrambe le funzioni — cifratura e firma digitale — insieme alla sua velocità, ha permesso ai ricercatori di industrializzare rapidamente il prodotto. RSA ha quindi dominato indiscussamente fino agli inizi degli anni 2000. Tuttavia, con l'avvento delle curve ellittiche, che permettono di ottenere chiavi più piccole, e che funzionano esclusivamente con il logaritmo discreto, tramite queste ultime c'è stata una notevole miglioria in termini di efficienza e sicurezza, per cui l'uso di RSA è stato notevolmente ridotto anche se non totalmente.

Chiavi pubbliche errate Inoltre, una chiave pubblica nel contesto del logaritmo discreto non nasconde insidie, poiché non è possibile generare una chiave pubblica "errata", mentre in RSA una chiave pubblica può essere calcolata erroneamente, introducendo vulnerabilità.

Fattorizzazione e limitazioni Nonostante RSA sia stato predominante e ancora oggi sia utilizzato, verrà probabilmente sostituito nel tempo, poiché non si basa sul logaritmo discreto, ma sulla **fattorizzazione di numeri**. Questo porta a chiavi più lunghe, non verificabili e non aggregabili, che presentano delle limitazioni.

Computer quantistici Sia RSA che il logaritmo discreto (Diffie-Hellman) sono vulnerabili ad attacchi effettuati con computer quantistici, i quali potrebbero rendere queste tecniche facilmente attaccabili, probabilmente tra dieci anni. Sebbene dieci anni possano sembrare un periodo relativamente breve, la storia insegna che gli aggiornamenti tecnologici richiedono tempo. Ad esempio, il *Millennium Bug* ha richiesto anni di lavoro per risolvere un semplice problema, in quanto negli anni '80 non si prevedeva l'arrivo del nuovo millennio, e sono stati necessari numerosi aggiornamenti software a livello globale. In modo simile, la digitalizzazione attuale richiede aggiornamenti su larga scala, non solo per piccoli parametri di dati, ma per l'intera infrastruttura.

9.1.4 Omomorfismo

La proprietà di omomorfismo è la seguente:

$$PK_1 = g^{x_1}, \quad PK_2 = g^{x_2}$$

L'omomorfismo consente di combinare due chiavi pubbliche in modo tale che:

$$PK_1 \cdot PK_2 = PK = g^{x_1+x_2}$$

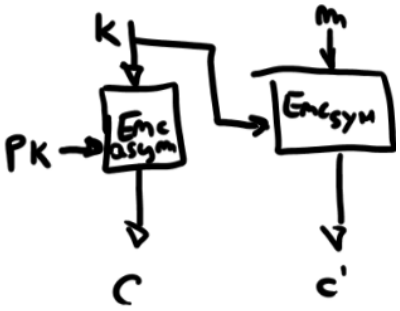
Le caratteristiche dell'omomorfismo sono le seguenti:

$$g \cdot K = g^{x_1 x_2} \cdot g = g^{x_1 x_2 + 1}$$

Questa proprietà, ad esempio, **non è presente in RSA**. L'omomorfismo è utile in contesti come le firme di gruppo, dove è necessario combinare più firme o valori senza compromettere la sicurezza individuale.

9.1.5 Cifratura ibrida

La cifratura ibrida è un metodo che combina la cifratura simmetrica e quella asimmetrica. Se la cifratura simmetrica è CCA-secure (Chosen Ciphertext Attack secure), allora anche la cifratura ibrida rimarrà CCA-secure, ma persistono i problemi legati all'avversario attivo, che può compromettere la sicurezza del sistema.



9.2 Firma digitale

Oltre alla cifratura a chiave pubblica e all'accordo di chiavi, esiste anche la firma digitale.

- È il terzo pilastro della crittografia a chiave pubblica.
- Risolve la stessa funzionalità del MAC (Message Authentication Code), ma senza la necessità di una chiave segreta condivisa. Il MAC richiede una chiave segreta condivisa per verificare l'autenticità di un messaggio, mentre la firma digitale no. Aggiungendo una firma digitale, posso dimostrare che il messaggio proviene da me in modo sicuro, senza mai essersi incontrati prima. Questo è l'obiettivo della firma digitale.
- È una tripla di algoritmi efficienti: (G, Sig, Ver).
  - $G(1^n)$ genera la coppia di chiavi (PK, SK).
  - $Sig(SK, \text{m})$ produce una firma s.
  - $Ver(PK, \text{m}, \text{s})$ restituisce un bit b di output (funzione di verifica)
- **Correctness**: Se (PK, SK) è la coppia di chiavi generata da G, allora per ogni messaggio m vale che:

$$Ver(PK,\ m,\ Sig(SK,\ m)) = 1$$

- **Unforgeability**: Un avversario probabilistico polinomiale (PPT) può richiedere firme per messaggi a sua scelta, ma solo con probabilità trascurabile riuscirà a produrre una coppia (m, s) tale che:

$$Ver(PK, m, s) = 1$$

e m non è nella lista delle firme richieste. L'avversario può vedere e richiedere firme, ma non riuscirà mai a generare una firma valida che sia mia, mantenendo una sicurezza polinomiale.

9.2.1 CRHFs: di nuovo in azione

Per firmare messaggi di grandi dimensioni non è necessario uno schema di firma sofisticato.

- Se uno schema di firma funziona per messaggi di una certa lunghezza L, allora può funzionare anche per messaggi di qualsiasi lunghezza, purché esista una *CRHF* (Collision-Resistant Hash Function) che possa prendere tali messaggi e restituire in output stringhe di L bit.
- Questa modalità è chiamata **Hash-and-Sign** e grazie ad essa possiamo firmare messaggi di grandi dimensioni molto velocemente, ottenendo firme di dimensioni ridotte.
- Problema: come firmiamo i messaggi?
  - Esistono diversi modi per istanziare la tripla di algoritmi (G, Sig, Ver).
  - Storicamente è stato utilizzato RSA, ma progressivamente è diventato obsoleto.
  - Uno schema chiamato DSA (Digital Signature Algorithm), con il suo mapping su curve ellittiche tramite *ECDSA* (Elliptic Curve DSA), è stato utilizzato a lungo. Questo schema è parecchio confusionario ma comunque valido, tuttavia è stato usato solo perché uno schema migliore, ovvero lo *Schnorr's Signature Scheme*, era sotto brevetto e quindi costoso da utilizzare.[15]
  - La diversificazione degli schemi di firma standardizzati è importante, in modo che se uno degli schemi standardizzati dovesse essere compromesso, siano disponibili altre alternative sicure.

9.2.2 Schnorr's Signature Scheme

Lo standard utilizzato inizialmente è stato *ECDSA* (Elliptic Curve Digital Signature Algorithm) poiché lo schema *Schnorr* non poteva essere utilizzato a causa di un brevetto. Tuttavia, nel 2008 il brevetto è scaduto, e quindi è iniziata l'implementazione dello schema *Schnorr*. Ad esempio, Bitcoin, che è nato nel 2009 e utilizza *ECDSA*, ha implementato *Schnorr* nel 2021. I due schemi hanno procedure di generazione delle chiavi diverse; è possibile però la conversione con procedure di mapping specifiche, ma ciò richiede attenzione nella conversione per mantenere la sicurezza crittografica.

9.2.3 Continuo dell'attacco attivo in DH

Siamo rimasti che l'avversario sta spiando la conversazione. È possibile **contrastarlo utilizzando la firma digitale**, in particolare modificando il fatto che venga firmato P_k. Il server ha quindi una chiave di firma. Il server invia P_k e lo firma con la propria chiave segreta di firma SK_S, mentre la firma viene verificata con la chiave pubblica PK_S. Quando l'avversario tenta di modificare il messaggio, non avendo la chiave segreta di firma, il client può verificare la firma e rilevare qualsiasi modifica. In questo modo, l'avversario non può alterare alcun bit del messaggio. Il destinatario, quindi, confronta la firma e verifica se è valida.
La firma può essere controllata anche dal browser. Ad esempio, negli strumenti di Chrome è possibile visualizzarla facilmente tramite i *Developer Tools*.

[15] Lo schema ECDSA (Elliptic Curve Digital Signature Algorithm) è noto per essere piuttosto complesso e non immediatamente intuitivo. La comunità crittografica era consapevole che lo schema di Schnorr era matematicamente più elegante e potenzialmente più efficiente di ECDSA, ma le restrizioni brevettuali hanno spinto verso l'adozione di alternative.

9.2.4 In molti scenari serve una PKI

PKI = *Public Key Infrastructure* (Infrastruttura a Chiave Pubblica)

Problema Quando comunico in rete con qualcuno, come posso essere sicuro che si tratti della giusta organizzazione (ad esempio, *unisa.it*, *banca.it*) con cui desidero dialogare in modo sicuro? Non ci si può fidare del TCP/IP da solo. Se non ricevo la chiave pubblica dell'organizzazione in un ambiente sicuro, come posso utilizzarla in modo affidabile?

- Tutto si riduce a un concetto simile alla vita reale, alla frase "patente e libretto".
- L'identificazione avviene mediante un attestato che associa informazioni anagrafiche a una fotografia.

In modo simile, per conoscere l'identità di chi comunica con noi, dobbiamo chiedergliela direttamente.

- L'attestato è più propriamente chiamato *certificato digitale*, e associa le informazioni dell'organizzazione a una chiave pubblica. Viene rilasciato da un ente certificatore di fiducia, ed è firmato digitalmente.
- Con la carta di identità elettronica italiana, ad esempio, si può creare un PDF firmato, e in questo caso viene utilizzato un certificato digitale effettivo.
- I server in tutto il mondo possiedono anche una firma digitale che viene inviata al client durante la connessione. Questa firma è comunque verificata dal browser.
- Se il browser non è sicuro del sito o della firma, appare un avviso di sicurezza.

9.2.5 Certificato digitale

Come per i documenti che usiamo nel mondo fisico, anche un certificato digitale contiene:

- Le generalità del soggetto (equivalenti a nome, data di nascita, ecc.);
- La chiave pubblica (equivalente della foto e della firma);
- Le generalità dell'autorità che rilascia il certificato;
- La data di rilascio, la scadenza e lo scopo del certificato.

Formato standard È evidente che, per "automatizzare" l'uso dei certificati digitali, è necessario un formato standard. Lo standard per le firme digitali è *X.509*. X.509 è uno standard della *PKI* (Public Key Infrastructure) ed è utilizzato per definire il formato dei certificati digitali, che servono per autenticare l'identità di utenti, dispositivi o server.

9.2.6 CA: Autorità di Certificazione

Chiunque rilasci un certificato (cioè una *CA*) dovrebbe a sua volta avere un certificato rilasciato da un'altra autorità, oppure un certificato auto-rilasciato.

- Possono essere create **gerarchie** simili a un albero, in cui le *root* sono le CA autoproclamate, i nodi interni sono CA riconosciute come tali dai loro *parent*, e le foglie sono certificati di client/server (non-CA).
- Un'alternativa è il *web of trust*, in cui tutti rilasciano certificati e la decisione di fidarsi su un'identità digitale, anziché essere delegata a un'autorità centralizzata, è affidata a meccanismi come i *common friends*.
- La CA dovrebbe occuparsi della revoca di un certificato (ad esempio, pubblicando una *Certificate Revocation List*, CRL).

- La CA dovrebbe fare in modo che il suo certificato raggiunga coloro che si fidano di essa, o, più brutalmente, convincere i principali software a includere tale certificato nel database delle *CA trusted*.

Le autorità di certificazione di livello inferiore si rivolgono alle autorità di certificazione di livello superiore per farsi riconoscere, creando così una gerarchia. In questo modo, è possibile avere sia un certificato "piccolo" che un certificato "grande" rilasciato da un'entità superiore. Il browser verifica solo i certificati più "grandi", se necessario.

continuando le CA.

Dominio con firma Supponiamo di acquistare un dominio, ad esempio `www.uniroma1.it`. Quando un utente si collega con il browser, la confidenzialità della comunicazione viene garantita mediante il protocollo Diffie-Hellman (DH). Tuttavia, per garantire l'autenticità del server, è necessaria una firma digitale.

Chiavi del dominio Per ottenere questa firma, il server utilizza una coppia di chiavi: una chiave pubblica (PK) e una chiave privata (SK). Tuttavia, affinché il certificato digitale sia riconosciuto come valido, il server deve rivolgersi a un'Autorità di Certificazione (CA), che verifica l'identità del proprietario del dominio. Dopo il pagamento di una tariffa e il completamento dei controlli necessari (verifica della proprietà del dominio e del possesso della chiave segreta), la CA rilascia un certificato firmato (proprio dalla stessa CA), attestante che la chiave pubblica associata a `www.uniroma1.it` è autentica.

Certificato del dominio Quindi, quando il server invia la chiave digitale pubblica al browser client, quest'ultimo verifica il certificato ricevuto, controllando che la firma sia stata apposta da una CA riconosciuta e che appartenga a quel determinato dominio del server. Se il certificato è valido, il browser può proseguire con la negoziazione Diffie-Hellman e il server può firmare con la propria PK.

CA Intermedie Non tutte le CA sono direttamente riconosciute (poiché salvate) dai browser. Esistono CA di livello inferiore (intermedie), le cui firme potrebbero non essere salvate nel browser e quindi i relativi certificati rigettati. Per ovviare a questo problema, una CA intermedia viene certificata da una CA superiore, creando una struttura gerarchica. Se il certificato di `www.uniroma1.it` è stato emesso da una CA intermedia (C1), ma quest'ultima è stata certificata da una CA più autorevole (C2), il browser può verificare la validità di C1 attraverso la certificazione di C2.

CA Autoproclamata Chi garantisce la sicurezza in cima alla gerarchia? Le CA radice (root CA) sono autorità autoproclamate, i cui certificati sono autofirmati. Perché questi certificati siano considerati affidabili, vengono preinstallati nei browser al momento dell'installazione. Tuttavia, questi certificati possono essere aggiornati o revocati in caso di compromissione.

Alternativa: Web of Trust Un'alternativa al modello gerarchico delle CA è il *Web of Trust*, in cui la fiducia non è centralizzata, ma si basa su relazioni di fiducia dirette tra utenti. In questo schema, un utente si fida dei certificati firmati da persone di cui si fida direttamente, creando una struttura a grafo invece che gerarchica. Tuttavia, questo modello non ha avuto un'ampia adozione nel contesto della sicurezza web.

Revoca dei Certificati Se una chiave privata viene compromessa o persa, la CA revoca il certificato associato e lo inserisce in una Certificate Revocation List (CRL). I browser, prima di accettare un certificato, possono consultare la CRL per verificare che il certificato non sia stato revocato.

9.2.7 X509v3

X.509v3 è lo standard in uso dal 1996 per il formato dei certificati digitali (Wikipedia: X.509). Il certificato X.509v3 contiene diversi campi, tra cui:

- **Versione**: indica la versione dello standard X.509 utilizzata (ad es. v1, v2, v3).
- **Certificate Serial Number**: numero univoco assegnato al certificato dall'Authority che lo ha emesso.
- **Issuer**: l'ente (CA) che ha rilasciato il certificato.
- **Subject**: il proprietario del certificato, ossia l'entità a cui è assegnato.

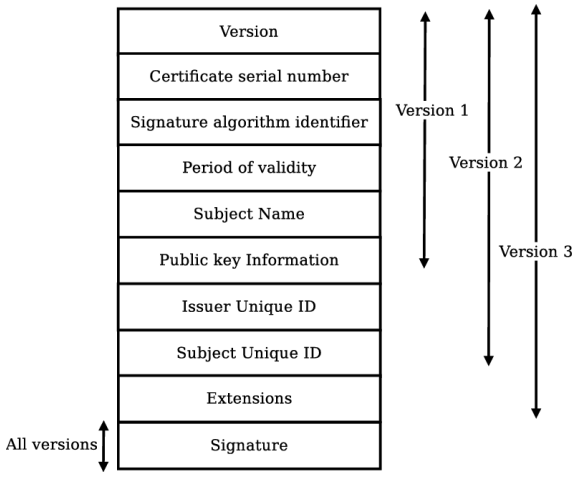


Figura 12: Struttura versioni X509

- **Period of Validity**: indica il periodo di validità del certificato con due date:
  - `Not Before`: data di inizio validità del certificato.
  - `Not After`: data di scadenza del certificato.
- **Public Key**: chiave pubblica dell'entità certificata, utilizzata per la cifratura o la verifica delle firme.
- **Estensioni**: campi opzionali che forniscono informazioni aggiuntive (es. utilizzo della chiave, SAN, CRL distribution points).
- **Firma finale**: la firma digitale apposta dalla CA per garantire l'integrità e l'autenticità del certificato.
- Alcuni codici univoci possono essere ignorati, poiché esistono solo per compatibilità con X.509v2.
- **Issuer/Subject**: L'issuer è l'entità che emette il certificato, mentre il subject è il destinatario del certificato. Queste informazioni vengono suddivise nei seguenti campi (non tutti obbligatori): `C` (Country), `O` (Organization), `OU` (Organizational Unit), `L` (Locality), `S` (State), `CN` (Common Name).
- **Tipiche estensioni**:
  - `X509v3 Extended Key Usage`: TLS Web Server Authentication
  - `X509v3 Subject Alternative Name`: `DNS:*.google.com`
  - `X509v3 CRL Distribution Points`: Full Name: URI: `http://..../xxx.crl`

Con questo, finiamo l'infrastruttura a chiave pubblica.

9.3 TLS: Transport Layer Security

Transport Layer Security (TLS) è un protocollo standard che definisce come avviene la comunicazione sicura tra un client e un server. È stato necessario standardizzare il meccanismo con cui un client (anonimo) si connette a un server dotato di certificato per garantire una comunicazione confidenziale.

Livelli di rete TLS si posiziona sopra il livello `TCP/IP` e sotto il livello applicativo.

Livello Applicativo
TLS (Transport Layer Security)
TCP/IP

Tabella 3: Posizionamento di TLS nello stack di rete

TLS come strato intermedio TCP di per sé non offre alcuna sicurezza: non garantisce né integrità né confidenzialità. TLS funge da strato intermedio che crea un canale sicuro tra i due endpoint di una comunicazione `TCP/IP`, fornendo:

- Confidenzialità dei dati
- Autenticazione
- Integrità delle comunicazioni

HTTPS In passato, solo la fase iniziale di una connessione HTTPS era protetta, mentre i dati effettivi venivano trasmessi in HTTP, esponendoli a possibili attacchi. Oggi, tutto il traffico HTTPS è cifrato end-to-end tramite TLS. Si ricorda che:

$$\text{HTTPS} = \text{HTTP over TLS}$$

HTTPS non funziona senza TLS/SSL: non è un'estensione di HTTP, ma un protocollo che incorpora la cifratura.

TLS come evoluzione di SSL TLS è l'evoluzione del protocollo Secure Sockets Layer (SSL). La prima versione di SSL fu sviluppata da Netscape, ma successivamente il protocollo fu standardizzato e rinominato in TLS, diventando un protocollo pubblico e aperto. Nel corso degli anni, TLS ha subito diversi aggiornamenti, fino alla versione più recente, **TLS 1.3**, che offre miglioramenti in termini di sicurezza e prestazioni.

TLS permette quindi di stabilire un canale di comunicazione sicuro sopra `TCP/IP`, proteggendo i dati da intercettazioni e manomissioni.

9.3.1 Accordo di chiavi effimero

Chiave segreta Si tratta dell'uso di nuovi messaggi Diffie-Hellman (DH) per garantire che la compromissione di una chiave segreta non metta a rischio le sessioni stabilite in passato, fornendo così la cosiddetta **forward secrecy**. TLS può anche essere eseguito in modalità *pre-shared key* (PSK) quando esiste già una chiave condivisa tra le due parti.

Chiavi non riutilizzabili In questo schema, le chiavi non vengono mai riutilizzate: una volta utilizzate per una sessione, vengono **eliminate**. Di conseguenza, la chiave di comunicazione AES derivata non può essere recuperata successivamente, anche ad esempio in seguito ad un attacco subito.

Sicurezza Questo meccanismo offre un vantaggio importante: se la chiave privata del server non viene salvata e il server viene attaccato, un attaccante non può decifrare le comunicazioni passate, garantendo così un alto livello di sicurezza.

9.4 Accordo di chiavi

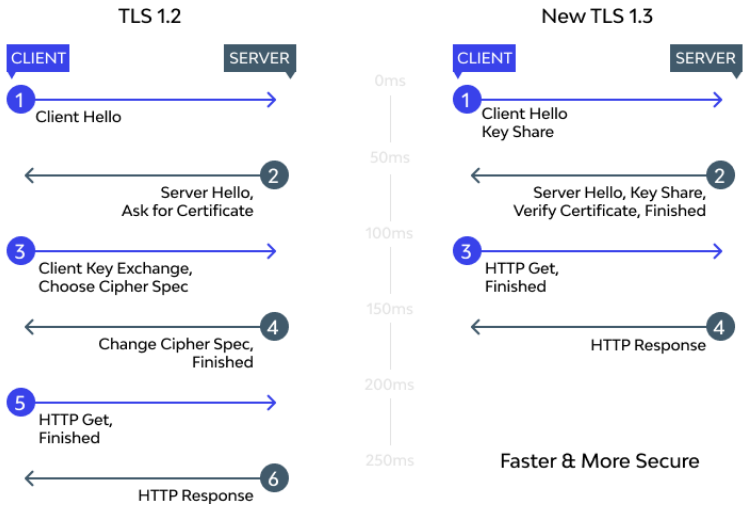
Oltre a Diffie-Hellman (DH), è possibile utilizzare un qualunque schema di cifratura a chiave pubblica per stabilire una chiave condivisa tra due parti.

- Il partecipante **P2** invia la propria chiave pubblica (`PK`).
- Il partecipante **P1** sceglie una chiave segreta `K` e la cifra con `PK`, quindi la invia a **P2**.

- Se `PK` è contenuta nel certificato del server, non è necessario alcun altro passaggio oltre all'invio del certificato stesso.
- Lo stesso principio si applica a Diffie-Hellman, poiché il contributo di ciascun partecipante può essere una chiave pubblica.

Questa ottimizzazione è stata ampiamente utilizzata in passato, ma oggi è sconsigliata perché la **forward secrecy** è considerata una proprietà di sicurezza più importante.

9.4.1 TLS 1.2 e 1.3



In TLS 1.3, l'accordo di chiavi avviene tramite la *key share* (punto 1 e 2). Le chiavi condivise sono effimere, cioè una volta usate non vengono riutilizzate. A differenza di DH, in TLS 1.3 non si utilizza una chiave diretta, ma una *key derivation function* che genera un **seed** ricco di entropia, il quale viene poi utilizzato nelle funzioni di generazione delle chiavi (key generation function).

Una delle caratteristiche principali di TLS 1.3 è che l'*handshake* viene cifrato non appena le chiavi sono condivise. Dopo che le chiavi private sono state scambiate, il resto della comunicazione dell'handshake è già protetto. L'obiettivo dell'handshake TLS è trasformare un canale di comunicazione insicuro (TCP) in un canale sicuro, utilizzando chiavi che vengono generate e condivise durante l'handshake. È importante notare che le chiavi usate per l'handshake sono differenti da quelle utilizzate per cifrare i dati applicativi, sia per qualità che per quantità. Questo assicura che un attaccante debba possedere entrambe le chiavi per riuscire a decifrare il traffico. Nonostante ciò, l'handshake non risulta essere pesante.

Nel punto 2 dell'handshake, viene anche scambiato il **server certificate**, che contiene la chiave pubblica del server e permette al client di verificarne l'autenticità.

Client Hello e Server Hello I messaggi `Client Hello` e `Server Hello` sono inviati in chiaro, permettendo al client di proporre una lista di algoritmi di cifratura supportati. Il server risponde scegliendo l'algoritmo che supporta, garantendo che entrambe le parti siano d'accordo su quale algoritmo utilizzare. Inoltre, entrambi i messaggi contengono dei **bit casuali**, utilizzati per migliorare la sfida nel processo di autenticazione, contribuendo a generare messaggi firmati difficili da prevedere e quindi più sicuri [16].

Cifratura del traffico Una volta completato l'handshake, tutto il traffico HTTP successivo, incluso un `HTTP GET`, viene cifrato nel canale sicuro tramite *authenticated encryption*. Ciò garantisce che i dati trasmessi siano protetti sia da attacchi di intercettazione che da modifiche.

[16] Ulteriori dettagli sono disponibili su `https://tls13.xargs.org/`

Key Derivation e Protezione contro gli Attacchi di Riflesso L'accordo di chiavi produce una chiave derivata da DH, che viene successivamente trasformata e ampliata tramite una *key derivation function*. Oltre alle chiavi utilizzate per proteggere l'handshake, vengono generate 4 chiavi: due per la cifratura e due per l'autenticazione della comunicazione. Questo approccio impedisce il rischio di *reflection attacks*, in cui un attaccante potrebbe tentare di inviare un messaggio al server e ricevere una risposta falsificata. L'uso di chiavi separate e multiple aumenta la sicurezza complessiva della connessione.

Passaggi Osserviamo nel dettaglio i passaggi per TLS 1.3

1. **Key Exchange e Key Share**: Il client e il server stabiliscono un *shared secret* utilizzando il *Diffie-Hellman Key Exchange* (DHKE) o *Elliptic Curve Diffie-Hellman* (ECDH). TLS 1.3 migliora la sicurezza rispetto a TLS 1.2 cifrando l'*handshake* non appena le chiavi vengono condivise. Inoltre, le chiavi usate per proteggere l'*handshake* sono distinte da quelle usate per la protezione dei dati dell'*application layer*, prevenendo attacchi di correlazione.

2. **Negoziazione degli Algoritmi**: I primi due messaggi dell'*handshake* servono per negoziare gli algoritmi di crittografia utilizzati per garantire la sicurezza del canale. Il client invia una lista di algoritmi supportati e il server sceglie il primo disponibile. Inoltre, vengono inviati valori casuali *(random nonces)* necessari per il processo di *key derivation*.

3. **Autenticazione e Identificazione**: Il server deve sempre autenticarsi inviando una catena di certificati X.509v3 e una firma sui messaggi scambiati fino a quel punto. Il client può autenticarsi opzionalmente seguendo la stessa procedura. La correttezza della chiave condivisa è verificata attraverso il messaggio `Finished`, che dimostra l'integrità della comunicazione.

4. **Protezione della Comunicazione**: Una volta completato l'*handshake*, i messaggi dell'application layer possono essere trasmessi in modo sicuro utilizzando *Authenticated Encryption with Associated Data* (AEAD)[17]. Questo garantisce sia la riservatezza sia l'integrità dei dati trasmessi.

5. **Session Resumption**: TLS 1.3 introduce il meccanismo di *session resumption*, che permette di riutilizzare chiavi precedentemente negoziate per evitare un nuovo *full handshake*. Ciò riduce il tempo di latenza e migliora l'efficienza della connessione.

6. **Key Derivation e Sicurezza**: La chiave derivata dall'*Ephemeral Diffie-Hellman* (DHE/ECDHE) viene estesa utilizzando un *Key Derivation Function* (HKDF) per generare quattro chiavi crittografiche distinte: chiavi per la protezione dell'*handshake* e chiavi per la protezione dei dati applicativi. Questo design previene attacchi di riflessione (*Reflection Attack*[18]), in cui un attaccante potrebbe manipolare i messaggi per indurre il server a inviare risposte prevedibili.

9.4.2 Esempio di attacco Man-in-the-Middle e controllo di massa

Consideriamo il caso in cui un utente desideri aggiungere certificati personalizzati a Google Chrome per bypassare i controlli di sicurezza o manipolare la connessione HTTPS.

Scenario: Man-in-the-Middle In questo esempio, si verifica un attacco **Man-in-the-Middle** (MITM), in cui un attaccante (nel caso specifico, il router italiano) si inserisce tra il client e il server legittimo (Google). Durante una comunicazione HTTPS, l'attaccante può intercettare e manipolare i dati senza che il client o il server se ne accorgano.

[17] Si ricorda che AEAD è una modalità di crittografia che garantisce sia la riservatezza sia l'integrità dei dati. AEAD prende in ingresso un **plaintext**, un **nonce** (valore univoco) e dati aggiuntivi non cifrati ma autenticati (**Associated Data, AD**). L'output include il **ciphertext** e un **tag di autenticazione** per verificare l'integrità del messaggio. Algoritmi comuni AEAD sono `AES-GCM` e `ChaCha20-Poly1305`, utilizzati in **TLS 1.3** per proteggere la comunicazione

[18] Tipo di attacco in cui un aggressore induce un sistema a riflettere i propri messaggi contro sé stesso. Questo può avvenire, ad esempio, in protocolli di autenticazione bidirezionali, dove l'attaccante sfrutta un challenge inviato dal server per autenticarlo nuovamente verso il client. L'idea alla base di questo tipo di attacco è di imbrogliare la vittima chiedendogli una soluzione (response) alla sua stessa sfida (challenge)

- L'utente invia una richiesta di connessione HTTPS a `https://gmail.com` tramite il suo browser (client).
- Il **router italiano** (che agisce come attaccante) intercetta la connessione e invia al client una risposta che sembra provenire da Google, ma in realtà è una risposta falsa.
- In questa risposta, il router invia un certificato *tarocco* che sembra appartenere a Google, ma non è legittimo. Il router firma il certificato con una **CA** (Certificate Authority) che sembra valida, ma che in realtà è una CA non riconosciuta dai browser di default.
- Poiché l'utente ha aggiunto questa CA nel suo browser (per esempio, per un controllo di massa o una modifica non autorizzata), il browser non segnala alcun errore e accetta il certificato.
- Il client ora comunica con il router invece che con il server legittimo, permettendo all'attaccante di intercettare e modificare la comunicazione (username, password, dati sensibili).

Controllo di massa e sorveglianza Un esempio di sorveglianza di massa potrebbe essere un governo o una grande organizzazione che installa un proprio certificato radice (root certificate) su dispositivi attraverso una distribuzione forzata (ad esempio, tramite software o hardware). In questo modo, tutte le connessioni HTTPS che passano attraverso il sistema possono essere intercettate e monitorate senza che gli utenti se ne accorgano. L'attaccante, in questo caso, avrebbe accesso alle comunicazioni cifrate senza che il browser mostri alcun avviso di errore, in quanto la CA compromessa è stata aggiunta ai certificati di fiducia.

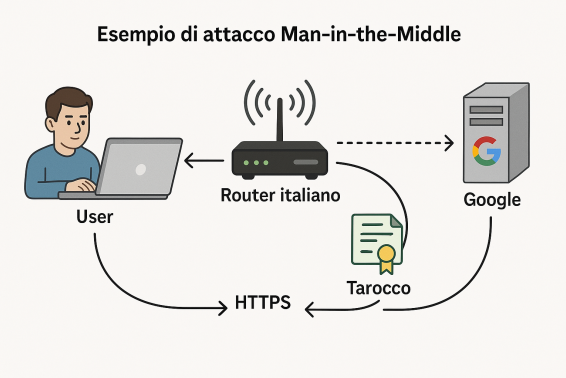


Figura 13: Esempio schematico

9.4.3 Firme digitali standard

Gli schemi di firma più comuni sono: ECDSA ed RSA-PSS. Nei casi specifici, troviamo diversi standard per la firma digitale applicata a documenti elettronici:

- **CAdES** (CMS Advanced Electronic Signatures): firma digitale avanzata basata su CMS (Cryptographic Message Syntax), utilizzata per file generici.
- **PAdES** (PDF Advanced Electronic Signatures): firma digitale avanzata per documenti PDF, conforme agli standard ETSI.
- **PAdES Grafico**: variante di PAdES che include una rappresentazione visibile della firma all'interno del documento PDF.
- **XAdES** (XML Advanced Electronic Signatures): firma digitale avanzata basata su XML, utile per documenti in formato XML.

9.5 OpenSSL – Da terminale

Lezione pratica: utilizziamo OpenSSL per operazioni crittografiche.

9.5.1 DH Key Exchange

- Diffie-Hellman richiede due contributi indipendenti per stabilire una chiave condivisa segreta su un canale insicuro. Per iniziare, generiamo un gruppo in cui il problema del logaritmo discreto sia computazionalmente difficile e quindi adatto per DH:

```
openssl dhparam -out dhparam1.pem 2048 //genera p,g
```

Si ottiene quindi:

```
.................+.....................................................+
//(..testo..)
+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

Il comando genera i parametri `DH (p, g)` dove `p` è un numero primo di 2048 bit e `g` è il generatore del gruppo ciclico. La generazione può richiedere tempo poiché implica la creazione di un numero primo `p` sicuro (tipicamente `p = 2q + 1` dove `q` è anch'esso primo, noto come "safe prime") e la verifica delle proprietà del generatore `g`. Il file `dhparam1.pem` è codificato in Base64. Per esaminarlo in formato leggibile:

```
openssl dhparam -in dhparam1.pem -text
```

si ottiene:

```
    DH Parameters: (2048 bit)
    P:
        00:a6:89:1b:00:a4:c5:a6:50:67:7a:5e:34:c0:ce:
        cb:f9:db:e5:bf:57:36:b2:5c:8a:2b:52:5d:43:e2:
        71:99:09:b5:c9:91:81:e5:7a:8d:08:4d:3b:f5:d2:
        f9:fd:21:6f:fa:7f:cd:3e:f9:3b:5d:d4:79:a9:f7:
        9c:e1:52:2e:aa:ab:8e:93:b4:35:ef:77:32:18:ea:
        7f:2c:b3:54:7c:a2:af:8d:3b:49:b5:21:11:15:44:
        36:b2:8e:e6:33:d7:ae:8f:e2:bf:c1:47:a4:de:e0:
        b9:28:61:a9:49:f8:16:f6:ee:1d:ca:bf:22:16:24:
        03:e3:e9:13:47:a7:f3:6f:6a:b9:09:a1:45:fb:bf:
        5c:66:08:82:47:f9:e7:20:73:f7:49:cc:44:4f:5b:
        b1:a0:35:ae:c0:61:8e:03:8c:19:ab:f1:0a:de:15:
        6a:60:08:2f:64:a7:8a:61:86:57:6c:94:78:1e:5b:
        5b:97:2c:42:3e:60:2b:c2:66:d4:69:31:11:60:3b:
        dc:06:b7:64:f5:6a:e3:d3:a7:9a:0a:f6:5f:75:e0:
        d9:59:c9:2b:10:b8:39:cb:6c:1c:dd:29:96:15:e1:
        34:66:ad:10:d5:da:fd:02:71:6d:91:28:a0:46:4a:
        36:63:f5:f4:05:e8:5c:ed:91:8e:7c:72:16:bd:fd:
        e7:1f
```

```
    G:    2 (0x2)
-----BEGIN DH PARAMETERS-----
MIIBCAKCAQEApokbAKTFplBnel40wM7L+dvlv1c2slyKK1JdQ+JxmQm1yZGB5XqN
CE079dL5/SFv+n/NPvk7XdR5qfec4VIuqquOk7Q173cyGOp/LLNUfKKvjTtJtSER
FUQ2so7mM9euj+K/wUek3uC5KGGpSfgW9u4dyr8iFiQD4+kTR6fzb2q5CaFF+79c
ZgiCR/nnIHP3ScxET1uxoDWuwGGOA4wZq/EK3hVqYAgvZKeKYYZXbJR4HltblyxC
PmArwmbUaTERYDvcBrdk9Wrj06eaCvZfdeDZWckrELg5y2wc3SmWFeE0Zq0Q1dr9
AnFtkSigRko2Y/X0Behc7ZGOfHIWvf3nHwIBAg==
-----END DH PARAMETERS-----
```

Abbiamo quindi sia G che P che necessitiamo. Il gruppo è stato generato.

- **Generazione chiave provata:** Generiamo una chiave privata DH basata sui parametri precedenti:

```
openssl genpkey -paramfile dhparam1.pem -out private_key.pem //genera una
    private key
```

Per visualizzare la chiave privata creata in formato leggibile:

```
openssl pkey -in private_key.pem -text //mostra il contenuto in esadecimale
```

Con il quale otteniamo:

```
-----BEGIN PRIVATE KEY-----
MIICJgIBADCCARcGCSqGSIb3DQEDATCCAQgCggEBAKaJGwCkxaZQZ3peNMDOy/nb
5b9XNrJciitSXUPicZkJtcmRgeV6jQhNO/XS+f0hb/p/zT75013Uean3nOFSLqqr
jpO0Ne93MhjqfyyzVHyir407SbUhERVENrKO5jPXro/iv8FHpN7guShhqUn4Fvbu
Hcq/IhYkA+PpE0en829quQmhRfu/XGYIgkf55yBz90nMRE9bsaA1rsBhjgOMGavx
Ct4VamAIL2SnimGGV2yUeB5bW5csQj5gK8Jm1GkxEWA73Aa3ZPVq49Onmgr2X3Xg
2VnJKxC4OctsHN0plhXhNGatENXa/QJxbZEooEZKNmP19AXoXO2RjnxyFr395x8C
AQIEggEEAoIBAG7zSTzx49ENa/61SV0CP0zHoB1QnuzHYzkXDyeVomb9ArNMv8yy
Op6FY4lBukupxusEicYZ6k0l8wCs7TIXUUcDGXBBd0ikURMNV8lvRWWPCjbUxR5f
HYIDwcmxoTWELYmJ8FdJLXULHNYhU89o9UqoZX49cmfrIrv8NGnuWcejBUmiVXpb
8dpk2+t3cUErJacvqhgDDCLIfYX77oOKFddC8yxlhqELNwksmPxbs6DAZnWdk73J
OPf+2DniDFKugEtG0JkwML4ZiUaEtqKTTRkH3tthQuBr8+s8K9AY18w7vC4kG/1I
7Ng8cBOSnPRtm7olESbgTYbVM2MecjZVlZs=
-----END PRIVATE KEY-----
DH Private-Key: (2048 bit)
private-key:
    6e:f3:49:3c:f1:e3:d1:0d:6b:fe:b5:49:5d:02:3f:
    4c:c7:a0:1d:50:9e:ec:c7:63:39:17:0f:27:95:a2:
    66:fd:02:b3:4c:bf:cc:b2:3a:9e:85:63:89:41:ba:
    4b:a9:c6:eb:04:89:c6:19:ea:4d:25:f3:00:ac:ed:
    32:17:51:47:03:19:70:41:77:48:a4:51:13:0d:57:
    c9:6f:45:65:8f:0a:36:d4:c5:1e:5f:1d:82:03:c1:
    c9:b1:a1:35:84:2d:89:89:f0:57:49:2d:75:0b:1c:
    d6:21:53:cf:68:f5:4a:a8:65:7e:3d:72:67:eb:22:
    bb:fc:34:69:ee:59:c7:a3:05:49:a2:55:7a:5b:f1:
    da:64:db:eb:77:71:41:2b:25:a7:2f:aa:18:03:0c:
    22:c8:7d:85:fb:ee:83:8a:15:d7:42:f3:2c:65:86:
    a1:0b:37:09:2c:98:fc:5b:b3:a0:c0:66:75:9d:93:
    bd:c9:38:f7:fe:d8:39:e2:0c:52:ae:80:4b:46:d0:
    99:30:30:be:19:89:46:84:b6:a2:93:4d:19:07:de:
    db:61:42:e0:6b:f3:eb:3c:2b:d0:18:d7:cc:3b:bc:
    2e:24:1b:fd:48:ec:d8:3c:70:13:92:9c:f4:6d:9b:
    ba:25:11:26:e0:4d:86:d5:33:63:1e:72:36:55:95:
    9b
public-key:
    7e:75:31:7c:5b:ef:23:eb:9f:d1:c9:8d:67:33:33:
    0d:f4:ce:7c:41:3b:6f:33:13:bd:11:cd:19:94:b8:
    ee:e2:07:d7:1a:b5:ef:94:6e:f2:43:2b:77:87:9d:
    07:f5:88:44:db:be:61:47:9a:f4:21:e3:50:13:02:
    01:ba:39:f1:cd:89:86:9c:3a:5b:58:fc:f6:12:1b:
    42:d3:a1:44:69:a2:9e:49:f3:a4:51:09:ed:b3:55:
    67:4a:9f:37:c6:88:a0:12:09:93:aa:e9:31:d8:95:
    b3:c8:4b:12:98:77:16:2f:cb:e0:7e:7b:41:6f:b9:
    ec:40:c1:a5:92:31:21:af:5f:38:29:75:9c:d8:90:
    68:2c:7c:16:22:86:61:80:28:ef:c7:67:dd:1f:ae:
```

```
    67:48:d6:3c:f7:da:4b:73:07:94:8b:4b:14:69:8c:
    63:40:72:0c:2b:09:0d:a2:d1:40:c7:12:1b:d3:a2:
    8c:06:3d:ae:f6:37:7a:3a:3d:49:83:0e:d9:4d:12:
    00:96:39:cb:00:6b:69:b8:c1:e4:a8:d3:98:fc:0f:
    f7:fe:88:b0:c5:91:b6:f1:c3:13:b1:bb:a0:08:4b:
    7e:f6:e0:05:d2:3c:7d:ff:06:28:55:95:d5:cf:5f:
    16:00:a3:44:d7:70:ad:b2:a3:28:c9:36:17:6e:f8:
    21
P:
    00:a6:89:1b:00:a4:c5:a6:50:67:7a:5e:34:c0:ce:
    //(..testo..)
    34:66:ad:10:d5:da:fd:02:71:6d:91:28:a0:46:4a:
    36:63:f5:f4:05:e8:5c:ed:91:8e:7c:72:16:bd:fd:
    e7:1f
G:    2 (0x2)
```

Abbiamo generato tutto quello che serve per ottenere il primo contributo per DH. La chiave privata contiene sia i parametri p e g condivisi, sia la chiave privata generata casualmente e la corrispondente chiave pubblica derivata.

- Per generare la chiave pubblica DH da condividere in rete:

```
openssl pkey -in private_key.pem -pubout -out DH.pem //genera il DH element
openssl pkey -pubin -in DH.pem -text //mostra il contenuto in esadecimale
```

Che genera:

```
-----BEGIN PUBLIC KEY-----
MIICJDCCARcGCSqGSIb3DQEDATCCAQgCggEBAKaJGwCkxaZQZ3peNMDOy/nb5b9X
...
/wYoVZXVz18WAKNE13CtsqMoyTYXbvgh
-----END PUBLIC KEY-----

DH Public-Key: (2048 bit)
public-key:
    7e:75:31:7c:5b:ef:23:eb:9f:d1:c9:8d:67:33:33:
    0d:f4:ce:7c:41:3b:6f:33:13:bd:11:cd:19:94:b8:
    //(..testo..)
    f7:fe:88:b0:c5:91:b6:f1:c3:13:b1:bb:a0:08:4b:
    16:00:a3:44:d7:70:ad:b2:a3:28:c9:36:17:6e:f8:
    21
P:
    00:a6:89:1b:00:a4:c5:a6:50:67:7a:5e:34:c0:ce:
    cb:f9:db:e5:bf:57:36:b2:5c:8a:2b:52:5d:43:e2:
    //(..testo..)
    34:66:ad:10:d5:da:fd:02:71:6d:91:28:a0:46:4a:
    36:63:f5:f4:05:e8:5c:ed:91:8e:7c:72:16:bd:fd:
    e7:1f
G:    2 (0x2)
```

Notiamo la presenza della **public key** e non della chiave segreta.

- **Secondo contributo:** Per generare un accordo DH serve un altro contributo, quindi lo genero (come farebbe l'altra parte della comunicazione) con:

```
openssl genpkey -paramfile dhparam1.pem -out private_key2.pem //genera una
    private key
```

Viene generata un'ulteriore private key, per ottenere `DH2.pem`, ovvero il secondo messaggio Diffie-Hellman.

```
openssl pkey -in private_key2.pem -pubout -out DH2.pem //genera il DH element
```

- **Calcolo del segreto condiviso:** Ora possiamo calcolare il segreto condiviso (l'accordo) DH usando la nostra chiave privata e la chiave pubblica dell'altro partecipante:

```
openssl pkeyutl -derive -inkey private_key.pem -peerkey DH2.pem -out
    shared_secret.bin //genera DH key
```

Si è calcolata quindi la chiave secondo il primo player. Analogamente, l'altra parte calcolerebbe lo stesso segreto condiviso usando la sua chiave privata e la nostra chiave pubblica:

```
openssl pkeyutl -derive -inkey private_key2.pem -peerkey DH1.pem -out
    shared_secret.bin //genera DH key
```

Possiamo verificare che i segreti condivisi siano identici, confermando che il protocollo ha funzionato correttamente:

```
diff shared_secret.bin shared_secret2.bin
```

L'assenza di differenze conferma che entrambe le parti hanno derivato la stessa chiave segreta condivisa senza mai trasmetterla direttamente sul canale di comunicazione.

Note per DH Key Exchange Per quanto riguarda questo tipo di scambio:

- **Sicurezza:** La robustezza del protocollo DH dipende dalla dimensione dei parametri (almeno 2048 bit è considerato sicuro oggi) e dalla casualità nella generazione delle chiavi private. Inoltre, lo scambio DH base è vulnerabile ad attacchi man-in-the-middle, quindi in implementazioni reali è necessario autenticare i partecipanti.
- **Nota tecnica:** Le funzionalità di `dhparam` sono anche disponibili tramite i comandi più generici `genpkey` e `pkeyparam`.

9.5.2 DSA KEYs and signatures

- **Generazione parametri:** Generiamo i parametri per la firma digitale di tipo DSA:

```
openssl dsaparam -out dsaparam1.pem 2048 //genera parametri per DSA
```

Si ottiene, visualizzando con `-text`

```
    DSA-Parameters: (2048 bit)
P:
    00:89:87:e2:57:05:69:78:2a:3d:41:91:37:b7:dc:
    bd:f6:61:e9:95:f9:ba:9e:c6:19:f0:d1:72:a5:44:
    ff:d7:29:d4:6d:7b:38:50:f5:e9:f0:50:24:2e:67:
    0a:12:f2:25:e4:e4:ec:53:34:12:10:97:31:df:f5:
    08:00:a3:11:c8:a4:bb:44:6b:d1:96:c6:2f:c3:86:
    36:57:79:2e:01:41:9b:37:fd:e9:14:e1:7e:ea:f8:
    d9:8f:4a:58:c7:f4:eb:f1:df:ae:cf:14:44:9c:a6:
    01:05:c0:49:e5:e3:1d:e4:82:a9:7c:54:1f:ab:20:
    6b:66:ef:56:99:f4:ea:0c:d7:a8:45:ae:f3:97:13:
    8d:e6:88:75:58:8c:af:8c:fe:8a:1c:b8:12:bd:fa:
    d4:b9:2a:95:d0:39:3d:77:13:e3:2f:7b:68:3d:52:
    82:52:2a:cc:03:41:86:7c:c1:30:5e:58:37:90:22:
    93:64:5f:9e:35:a6:fd:21:0c:ad:44:60:9a:17:a0:
    2a:4c:b2:ef:dc:50:95:23:da:6e:52:b5:bb:80:49:
    d7:50:99:9d:8a:5a:6e:b8:c8:43:53:a9:13:d5:69:
    f9:47:fd:5c:27:52:01:1d:e3:a2:d6:39:b5:03:7b:
    cf:21:20:a0:1e:3b:8d:42:f4:e1:25:0f:46:a8:a5:
    41:af
Q:
    00:a0:42:eb:3e:2e:23:ea:8a:e1:6e:fc:10:60:ac:
    a7:2c:e6:e4:79:4f:8a:51:8c:68:0f:99:81:e1
G:
    23:0c:61:78:89:b0:cc:57:75:55:7c:12:72:df:69:
    83:d9:cb:99:11:41:71:4d:97:de:02:fe:63:4c:1b:
    70:98:7f:d7:00:34:20:e5:a4:36:82:59:38:2c:c9:
    fe:98:a4:0f:b2:da:ea:56:03:19:2b:32:e1:04:46:
    e7:a4:1f:3b:ec:9b:57:8e:05:75:b5:3b:3c:4c:43:
    bf:bb:c0:f8:0a:84:46:0c:95:b6:a4:69:58:34:93:
    70:a3:75:c2:60:cb:1b:d0:29:7d:12:03:bd:9d:be:
    e1:7c:5b:78:ab:14:40:9f:8d:80:2d:58:c9:f2:8c:
    b0:ec:7e:ce:75:06:15:5f:fb:b3:33:3b:37:0a:0a:
    56:2d:c6:b1:5d:a6:e2:5a:59:e6:f7:7d:02:f7:12:
```

```
    9e:bd:75:0b:45:e2:0c:d9:2f:6e:e8:c5:58:d3:77:
    f1:42:9a:d6:10:14:78:e8:2c:6c:dc:df:ad:70:79:
    ca:63:29:84:fb:36:e9:bd:ba:25:1d:60:a0:21:18:
    8a:7b:86:6d:23:5c:43:8c:84:29:26:09:7e:59:2f:
    69:02:5d:0f:de:eb:39:a8:52:8a:9a:fe:b9:d9:b2:
    54:2e:10:8a:d3:0b:97:2a:c4:fa:26:76:a0:2f:18:
    e8:94:78:93:69:7b:36:47:43:0d:fb:a8:50:ff:3b:
    25
-----BEGIN DSA PARAMETERS-----
MIICKAKCAQEAiYfiVwVpeCo9QZE3t9y99mHplfm6nsYZ8NFypUT/1ynUbXs4UPXp
8FAkLmcKEvIl5OTsUzQSEJcx3/UIAKMRyKS7RGvRlsYvw4Y2V3kuAUGbN/3pFOF+
6vjZj0pYx/Tr8d+uzxREnKYBBcBJ5eMd5IKpfFQfqyBrZu9WmfTqDNeoRa7zlxON
5oh1WIyvjP6KHLgSvfrUuSqV0Dk9dxPjL3toPVKCUirMA0GGfMEwXlg3kCKTZF+e
Nab9IQytRGCaF6AqTLLv3FCVI9puUrW7gEnXUJmdilpuuMhDU6kT1Wn5R/1cJ1IB
HeOi1jm1A3vPISCgHjuNQvThJQ9GqKVBrwIdAKBC6z4uI+qK4W78EGCspyzm5HlP
ilGMaA+ZgeECggEAIwxheImwzFd1VXwSct9pg9nLmRFBcU2X3gL+Y0wbcJh/1wA0
IOWkNoJZOCzJ/pikD7La6lYDGSsy4QRG56Qf0+ybV44FdbU7PExDv7vA+AqERgyV
tqRpWDSTcKN1wmDLG9ApfRIDvZ2+4XxbeKsUQJ+NgC1YyfKMsOx+znUGFV/7szM7
NwoKVi3GsV2m4lpZ5vd9AvcSnr11C0XiDNkvbujFWNN38UKa1hAUeOgsbNzfrXB5
ymMphPs26b26JR1goCEYinuGbSNcQ4yEKSYJflkvaQJdD97rOahSipr+udmyVC4Q
itMLlyrE+iZ2oC8Y6JR4k2l7NkdDDfuoUP87JQ==
-----END DSA PARAMETERS-----

```

- **Generazione chiave privata:** Per generare la chiave privata DSA:

```
openssl gendsa -out dsaprivkey.pem dsaparam1.pem //genera chiave privata
```

La visualizziamo con `-text` e otteniamo:

```
read DSA key
Private-Key: (2048 bit)
priv:
    67:07:30:c5:1f:5d:4a:89:50:c1:cc:08:4f:2f:90:
    31:4e:8e:80:14:58:d1:e1:a5:01:37:9e:d1
pub:
    35:38:8a:c0:3c:11:3f:84:ba:89:af:b5:48:ea:50:
    3a:cc:09:52:7b:d0:4c:ef:80:e9:32:65:ed:88:e1:
    cb:91:bd:5a:a7:fb:12:73:b3:60:ce:fb:d6:28:43:
    bb:9a:53:b0:6b:43:f5:74:a5:30:d6:f6:2e:39:de:
    91:ba:19:9c:1e:09:d0:72:b0:84:65:6d:bf:20:aa:
    3b:48:5d:1a:c2:5a:1c:eb:14:f9:36:41:57:bf:b1:
    98:ba:9d:e7:49:6a:4a:09:e6:d8:2b:fb:f2:8a:6a:
    06:08:d9:01:3d:0f:43:d6:22:52:d2:41:b8:89:c5:
    1d:f2:84:a1:1f:da:31:d2:58:36:d5:19:b3:d0:b2:
    de:c4:6e:6a:81:93:89:1f:9f:2f:14:f9:b6:8d:36:
    5b:95:af:9a:e4:b2:f3:04:5d:5b:28:f0:dc:50:d7:
    cd:af:9a:0e:35:ad:25:91:1b:fc:8a:33:f5:83:d1:
    0b:dd:0f:44:88:f5:80:42:84:ec:54:3e:c6:10:c8:
    c8:23:60:37:98:77:15:3d:33:de:84:4e:16:11:dd:
    9a:74:77:51:8a:49:8d:4c:3e:8e:41:b4:7c:5a:5e:
    84:8b:a8:73:78:6b:2c:44:f3:21:64:e7:de:d4:83:
    9e:bd:9e:24:2a:83:ee:d3:cf:69:3f:2f:52:28:cc:
    24
P:
    00:89:87:e2:57:05:69:78:2a:3d:41:91:37:b7:dc:
    bd:f6:61:e9:95:f9:ba:9e:c6:19:f0:d1:72:a5:44:
    //(..testo..)
    f9:47:fd:5c:27:52:01:1d:e3:a2:d6:39:b5:03:7b:
    cf:21:20:a0:1e:3b:8d:42:f4:e1:25:0f:46:a8:a5:
    41:af
Q:
    00:a0:42:eb:3e:2e:23:ea:8a:e1:6e:fc:10:60:ac:
    a7:2c:e6:e4:79:4f:8a:51:8c:68:0f:99:81:e1
G:
    23:0c:61:78:89:b0:cc:57:75:55:7c:12:72:df:69:
    83:d9:cb:99:11:41:71:4d:97:de:02:fe:63:4c:1b:
    //(..testo..)
    54:2e:10:8a:d3:0b:97:2a:c4:fa:26:76:a0:2f:18:
    e8:94:78:93:69:7b:36:47:43:0d:fb:a8:50:ff:3b:
```

```
    25
writing DSA key
-----BEGIN DSA PRIVATE KEY-----
MIIDTQIBAAKCAQEAiYfiVwVpeCo9QZE3t9y99mHplfm6nsYZ8NFypUT/1ynUbXs4
UPXp8FAkLmcKEvIl5OTsUzQSEJcx3/UIAKMRyKS7RGvRlsYvw4Y2V3kuAUGbN/3p
FOF+6vjZj0pYx/Tr8d+uzxREnKYBBcBJ5eMd5IKpfFQfqyBrZu9WmfTqDNeoRa7z
lxON5oh1WIyvjP6KHLgSvfrUuSqV0Dk9dxPjL3toPVKCUirMA0GGfMEwXlg3kCKT
ZF+eNab9IQytRGCaF6AqTLLv3FCVI9puUrW7gEnXUJmdilpuuMhDU6kT1Wn5R/1c
J1IBHeOi1jm1A3vPISCgHjuNQvThJQ9GqKVBrwIdAKBC6z4uI+qK4W78EGCspyzm
5HlPilGMaA+ZgeECggEAIwxheImwzFd1VXwSct9pg9nLmRFBcU2X3gL+Y0wbcJh/
1wA0IOWkNoJZOCzJ/pikD7La6lYDGSsy4QRG56QfO+ybV44FdbU7PExDv7vA+AqE
RgyVtqRpWDSTcKN1wmDLG9ApfRIDvZ2+4XxbeKsUQJ+NgC1YyfKMsOx+znUGFV/7
szM7NwoKVi3GsV2m4lpZ5vd9AvcSnr11C0XiDNkvbujFWNN38UKa1hAUeOgsbNzf
rXB5ymMphPs26b26JR1goCEYinuGbSNcQ4yEKSYJflkvaQJdD97rOahSipr+udmy
VC4QitMLlyrE+iZ2oC8Y6JR4k2l7NkdDDfuoUP87JQKCAQA1OIrAPBE/hLqJr7VI
6lA6zAlSe9BM74DpMmXtiOHLkb1ap/sSc7NgzvvWKEO7mlOwaOP1dKUw1vYuOd6R
uhmcHgnQcrCEZW2/IKo7SFOawloc6xT5NkFXv7GYup3nSWpKCebYK/vyimoGCNkB
PQ9D1iJS0kG4icUd8oShH9ox0lg21Rmz0LLexG5qgZOJH58vFPm2jTZbla+a5LLz
BF1bKPDcUNfNr5oONa0lkRv8ijP1g9EL3Q9EiPWAQoTsVD7GEMjII2A3mHcVPTPe
hE4WEd2adHdRikmNTD6OQbR8Wl6Ei6hzeGssRPMhZOfe1IOevZ4kKoPu089pPy9S
KMwkAhxnBzDFH11KiVDBzAhPL5AxTo6AFFjR4aUBN57R
-----END DSA PRIVATE KEY-----

```

- **Generazione chiave pubblica:** Da ciò possiamo estrarre la chiave pubblica:

```
openssl dsa -in dsaprivkey.pem -pubout -out dsapubkey.pem //genera chiave
    pubblica
```

che visualizziamo con:

```
openssl dsa -pubin -in dsapubkey.pem -text
```

- **Firma del file:** Ora possiamo firmare il messaggio usando la chiave privata. In questo esempio firmiamo il file `test.txt` con il comando `dgst`:

```
openssl dgst -sign dsaprivkey.pem -out signature.bin test.txt //firma un file
```

Il file `signature.bin` contiene la firma digitale del file `test.txt`. È importante notare che viene utilizzata la funzione di hash SHA-256 per generare l'impronta del documento prima della firma.

- **Verifica della firma:** Per verificare la firma, utilizziamo la chiave pubblica, insieme alla firma e al file originale firmato:

```
openssl dgst -verify dsapubkey.pem -signature signature.bin test.txt //
    verifica di una firma
```

Se la firma è autentica e il file non è stato modificato, otterremo l'output `Verified OK`. Se anche un solo byte del file originale viene modificato, otterremo `Verification failure`, dimostrando l'integrità garantita dalla firma digitale.

Nota per DSA Per quanto riguarda DSA, si ha:

- **Sicurezza: DSA richiede l'uso di un generatore di numeri casuali sicuro per ogni firma.** Se lo stesso numero casuale viene riutilizzato per due firme diverse, un attaccante potrebbe essere in grado di ricavare la chiave privata. Questo è un aspetto critico dell'implementazione DSA.
- **Alternative moderne** Esistono comandi più recenti e generici in OpenSSL che possono essere utilizzati al posto dei comandi specifici per DSA:
  - `genpkey` al posto di `gendsa`
  - `pkey` al posto di `dsa`
  - `pkeyparam` al posto di `dsaparam`
  - `pkeyutl` al posto di `dgst` (per specifiche operazioni crittografiche)

9.5.3 OpenSSL – ECDSA KEYs and signatures

Vediamo come creare una firma di tipo ECDSA (Elliptic Curve Digital Signature Algorithm). Con le **curve ellittiche**, non lavoriamo con l'abbondanza di gruppi come in altri sistemi crittografici. Ogni curva ellittica ha il suo generatore, quindi non generiamo un gruppo ma specifichiamo quale curva utilizzare tra le tante standardizzate disponibili.

- **Invocazione parametri:** Invochiamo (e *non generiamo*) i parametri, specificando il nome della curva ellittica da utilizzare (`prime256v1`, equivalente alla curva NIST P-256):

```
openssl ecparam -name prime256v1 -out prime256v1.pem //invoca parametri per
    ECDSA su una specifica curva ellittiva
```

  Questo comando genera il file con i parametri della curva [19]. Se visualizziamo il contenuto del file con l'opzione `-text`, otteniamo:

```
EC-Parameters: (256 bit)
ASN1 OID: prime256v1
NIST CURVE: P-256
-----BEGIN EC PARAMETERS-----
BggqhkjOPQMBBw==
-----END EC PARAMETERS-----
```

- **Generazione chiave:** Generiamo la chiave privata ECDSA utilizzando i parametri della curva ellittica:

```
openssl genpkey -paramfile prime256v1.pem -out ecdsa_key.pem //genera chiave
    privata
```

  Visualizzando il contenuto della chiave privata con l'opzione `-text`:

```
openssl pkey -in ecdsa_key.pem -text
```

  otteniamo:

```
-----BEGIN PRIVATE KEY-----
MIGHAgEAMBMGByqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQgjPWzH3+v8fXgvnBK
4Q5F4ApgCXKrSWrWkuqkFhKNU/ehRANCAARCpgMsjfQTg/b/hfS5GDb90p/JGAJl
aXqshZ1tCOxBsCJTib9OhZQb8LQQP6cLfr3LrZjnQw8AE+U4aRERymC7
-----END PRIVATE KEY-----
Private-Key: (256 bit)
priv:
    8c:f5:b3:1f:7f:af:f1:f5:e0:be:70:4a:e1:0e:45:
    e0:0a:60:09:72:ab:49:6a:d6:92:ea:a4:16:12:8d:
    53:f7
pub:
    04:42:a6:03:2c:8d:f4:13:83:f6:ff:85:f4:b9:18:
    36:fd:d2:9f:c9:18:02:65:69:7a:ac:85:9d:6d:08:
    ec:41:b0:22:53:89:bf:4e:85:94:1b:f0:b4:10:3f:
    a7:0b:7e:bd:cb:ad:98:e7:43:0f:00:13:e5:38:69:
    11:11:ba:60:bb
ASN1 OID: prime256v1
NIST CURVE: P-256
```

  Osserviamo che le chiavi ECDSA sono più compatte rispetto alle chiavi RSA o DSA di sicurezza equivalente, grazie alle proprietà matematiche delle curve ellittiche. Il generatore è semplicemente la curva ellittica scelta, non un lungo testo di caratteri come prima.

- **Chiave pubblica:** Estraiamo la chiave pubblica dalla chiave privata:

```
openssl pkey -in ecdsa_key.pem -pubout -out ecdsapub_key.pem //genera chiave
    pubblica
```

  Visualizzando il contenuto con `-text`:

[19] Se facciamo il `cat` del nome della curva otteniamo solo la codifica della curva, non nulla di importante.

```
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEQqYDLI30E4P2/4X0uRg2/dKfyRgC
ZWl6rIWdbQjsQbAiU4m/ToWUG/C0ED+nC369y62Y50MPABPl0GkREcpguw==
-----END PUBLIC KEY-----
Public-Key: (256 bit)
pub:
    04:42:a6:03:2c:8d:f4:13:83:f6:ff:85:f4:b9:18:
    36:fd:d2:9f:c9:18:02:65:69:7a:ac:85:9d:6d:08:
    ec:41:b0:22:53:89:bf:4e:85:94:1b:f0:b4:10:3f:
    a7:0b:7e:bd:cb:ad:98:e7:43:0f:00:13:e5:38:69:
    11:11:ba:60:bb
ASN1 OID: prime256v1
NIST CURVE: P-256
```

Per il secondo contributo, rieseguiamo i comandi con l'altro file, e quindi poi possiamo fare l'accordo di chiavi.

- **Esportazione nel file:** Generiamo la chiave nel `file.bin`, e quindi per implementare un accordo di chiavi ECDH (Elliptic Curve Diffie-Hellman), possiamo utilizzare:

```
openssl pkeyutl -derive -inkey ecdsa_key.pem -peerkey ecdsapub_key.pem -out
    ecshared.bin
```

Anche qui, generiamo per entrambi i contributi.

- Due diverse esecuzioni dell'accordo di chiavi con le stesse chiavi produrranno sempre lo stesso risultato. Se verifichiamo con `diff`, non troveremo differenze. Anche modificando un solo byte in una delle chiavi, l'accordo di chiavi risulterà diverso dando `failure`.

- **Firma del file:** Per firmare un file di testo con ECDSA:

```
openssl dgst -sign ecdsa_key.pem -out signature.bin test.txt (firma un file)
```

e per verificare la firma:

```
openssl dgst -verify ecdsapub_key.pem -signature signature.bin test.txt (
    verfica di una firma)
```

È possibile utilizzare le chiavi pubbliche ECDSA per scambio di chiavi secondo il protocollo ECDH (Elliptic Curve Diffie-Hellman), quindi sono utilizzabili per DH. ECDH permette a due parti di stabilire una chiave segreta condivisa anche su un canale insicuro, sfruttando le proprietà delle curve ellittiche. Rispetto all'algoritmo DH classico, ECDH offre lo stesso livello di sicurezza con chiavi di dimensioni molto più ridotte.

9.5.4 RSA - Firme e cifratura asimmetrica

RSA è un algoritmo a chiave pubblica che può essere utilizzato sia per la **cifratura** che per la **firma digitale**. A differenza degli algoritmi basati su curve ellittiche (ECC), RSA *non sfrutta tali curve* ma usa la **fattorizzazione** e risulta generalmente meno efficiente a parità di sicurezza.

- **Generazione chiave:** Generiamo la chiave con

```
openssl genrsa -out rsakey.pem 2048 //genera chiavi RSA
```

La chiave privata contiene un modulo n, un esponente pubblico e, un esponente privato d e i due primi p e q utilizzati per la fattorizzazione.

- Esportiamo la chiave pubblica:

```
openssl rsa -pubout -in rsakey.pem -out rsapub.pem //esporta la public key
openssl rsa -pubin -in rsapub.pem -text //mostra la public key
```

La chiave pubblica contiene solamente il modulo n e l'esponente e. Non è possibile risalire ai fattori primi e quindi alla fattorizzazione fatta.

Notiamo che RSA non può essere utilizzata per DH. Ha un suo metodo di cifratura, vediamolo:

- **Cifrare il file:** Per cifrare occorre solo la chiave pubblica:

```
openssl pkeyutl -encrypt -inkey rsapub.pem -pubin -in test.txt -out test.enc
    //cifra
```

Il file `test.enc` contiene il ciphertext. La cifratura asimmetrica ha una dimensione massima limitata (legata alla lunghezza della chiave), quindi non è adatta a cifrare dati di grandi dimensioni.

- **Decifrare il file:** Per decifrare serve però la chiave segreta:

```
openssl pkeyutl -decrypt -inkey rsakey.pem -in test.enc -out test2.txt //
    decifra
```

Per decifrare è necessaria la chiave privata, poiché solo chi possiede d (esponente privato) può invertire l'operazione eseguita con l'esponente pubblico e.

Limite lunghezza dei messaggi la cifratura asimmetrica con RSA può gestire solo messaggi di dimensione *inferiore a un certo limite*, che dipende dalla lunghezza della chiave e dallo schema di padding usato. Ad esempio, con una chiave RSA da 2048 bit (256 byte), la **dimensione massima del messaggio cifrabile** è circa:

$$256 \text{ byte} - \text{overhead del padding (es. OAEP)}$$

Per questo motivo:

  - La dimensione della cifratura (`test.enc`) **è sempre pari alla lunghezza del modulo RSA** (es. 256 byte per 2048 bit), indipendentemente dalla lunghezza del messaggio in chiaro. Fino a che si sta in questo blocco, non vengono dati errori, se si sfora dà errore.
  - RSA **non viene usato per cifrare grandi quantità di dati**: è inefficiente e limitato in dimensione.
  - Nella pratica, RSA viene usato solo per cifrare una **chiave simmetrica** (es. AES), che poi viene usata per cifrare i dati reali. Questo schema si chiama **cifratura ibrida**.

Quindi, essendoci una larghezza massima supportata per la cifratura asimmetrica, questo tipo di cifratura viene utilizzata solo per l'accordo di chiavi (essendo la chiave relativamente piccola).

- **Firma del file:** Per firmare un file, anche se questo è troppo lungo e otterremo un errore:

```
openssl pkeyutl -in text.txt -inkey rsakey.pem -out sig.bin
```

Prima di firmare, quindi, se il file è troppo grande, questo viene hashato (ad esempio con SHA-256). La firma viene applicata all'hash del messaggio.

```
openssl pkeyutl -sign -in test.txt -inkey rsakey.pem -out sig  //firma
```

- **Verifica della firma:** Per verificare la firma, si usa:

```
openssl pkeyutl -verify -in test.txt -sigfile sig -pubin -inkey rsapub.pem //
    verifica
```

La verifica richiede:

  - il file originale,
  - la firma,

– la chiave pubblica.

Se il file è troppo grande, è necessario hasharlo prima, perché RSA firma solo dati piccoli (max lunghezza = lunghezza chiave). La verifica consiste nel confrontare l'hash del file con quello decifrato dalla firma.

Note per l'uso di RSA Per quanto riguarda RSA:

- I comandi moderni preferiscono l'uso di `genpkey` al posto di `genrsa`, come:

```
genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048
```

  e `pkey` invece di `rsa`.

- Lo *message space* è limitato al gruppo modulo n; per superare questo limite, si utilizza la **cifratura ibrida**, dove RSA cifra una chiave simmetrica usata poi per cifrare i dati reali.
- **Importante:** RSA non è adatto allo scambio di chiavi come Diffie-Hellman. Per l'accordo di chiave si utilizzano protocolli dedicati o viene impiegata la cifratura ibrida.

9.5.5 OpenSSL – RSA cifratura ibrida

1. **Input**: file di testo in chiaro (`input`) e chiave pubblica RSA (`rsapub.pem`).
2. **Generazione di una chiave** AES a 256 bit (32 byte esadecimali):

```
openssl rand -hex 32 > aeskey.hex
```

3. **Generazione di un vettore** di inizializzazione (IV) a 128 bit (16 byte esadecimali):

```
openssl rand -hex 16 > aesiv.hex
```

4. Caricamento della chiave AES in una variabile:

```
key=$(cat aeskey.hex)
```

5. Caricamento del vettore di inizializzazione in una variabile:

```
iv=$(cat aesiv.hex)
```

6. **Cifratura simmetrica del file** in chiaro utilizzando AES in modalità CTR (counter mode):

```
openssl enc -e -aes-256-ctr -in input -K $key -iv $iv -out input.enc
```

   La modalità CTR è sicura per cifrare stream di dati, ma è fondamentale non riutilizzare la stessa coppia `(key, iv)`.

7. **Cifratura della chiave AES con RSA** (cifratura asimmetrica), dove la chiave privata AES entra nel blocco RSA:

```
openssl pkeyutl -encrypt -inkey rsapub.pem -pubin -in aeskey.hex -out key.enc
```

   In questo passaggio, la chiave simmetrica AES viene cifrata con una chiave pubblica RSA, proteggendo così la chiave AES durante il trasporto. Questo approccio combina la velocità della cifratura simmetrica (AES) con la sicurezza della cifratura asimmetrica (RSA), consentendo la protezione dei dati e delle chiavi in modo sicuro durante la trasmissione.

8. **Output**: la tripla di file `input.enc` (testo cifrato), `key.enc` (chiave AES cifrata), `aesiv.hex` (IV in chiaro, non segreto).

Ibrido Con questa tecnica quindi, prima si cifra il file con AES e poi si cifra la chiave AES con RSA, combinando così la velocità della cifratura simmetrica con la sicurezza della cifratura asimmetrica.

Sicurezza La sicurezza di questo schema si basa sulla protezione della chiave simmetrica tramite RSA, ma al contrario di RSA puro non abbiamo problemi di lunghezza massima. L'IV non è segreto, ma deve essere unico per ogni cifratura. Il payload cifrato è leggermente più lungo (di circa 16 byte) rispetto al file originale, ma può viaggiare in rete senza problemi di sicurezza.

9.5.6 OpenSSL – RSA Decifratura Ibrida

- **Input**:
  - `input.enc`: file cifrato con AES-256-CTR
  - `key.enc`: chiave AES cifrata con RSA
  - `aesiv.hex`: IV usato nella cifratura simmetrica
  - `rsakey.pem`: chiave privata RSA per la decifratura
- **Decifratura della chiave AES** con la chiave privata RSA:

```
openssl pkeyutl -decrypt -inkey rsakey.pem -in key.enc -out keydec.hex
```

- Caricamento della chiave decifrata in una variabile:

```
key=$(cat keydec.hex)
```

- Caricamento del vettore di inizializzazione:

```
iv=$(cat aesiv.hex)
```

- **Decifratura simmetrica** del file:

```
openssl enc -d -aes-256-ctr -in input.enc -K $key -iv $iv -out input2
```

- **Output**: `input2`, che rappresenta il file originale in chiaro.

9.5.7 Esercizio: note aggiuntive

- **Firma e verifica digitale su messaggi di lunghezza arbitraria**:

  OpenSSL consente di firmare file di qualsiasi lunghezza grazie all'uso di funzioni hash:

```
openssl dgst -sha256 -sign rsapriv.pem -out firma.bin file.txt
openssl dgst -sha256 -verify rsapub.pem -signature firma.bin file.txt
```

- **Cifratura e decifratura a chiave pubblica di messaggi arbitrari**:

  Poiché RSA non supporta direttamente messaggi lunghi, si usa un approccio ibrido:
  - Il messaggio viene cifrato con una chiave simmetrica (es. AES).
  - La chiave simmetrica viene cifrata con RSA.

  Questo schema supporta cifratura/decifratura di file di qualsiasi dimensione in modo sicuro ed efficiente.

Comando	Funzione	Hash automatico	Tipo di file	Per firma digitale
`openssl dgst -sign`	Firma un **digest (hash)** del file con una chiave privata	Sì	Lunghi	✓Raccomandato (standard, sicuro, supporta file grandi)
`openssl pkeyutl -sign`	Firma direttamente i dati senza hash implicito	No	Corti, solo piccoli dati	Sconsigliato (solo per uso avanzato)

Tabella 4: Differenze tra `dgst -sign` e `pkeyutl -sign` in OpenSSL

9.5.8 OpenSSL – Certificate request

Quando una persona desidera ottenere un certificato digitale, deve inviare i dati alla Certification Authority (CA). OpenSSL consente di creare una richiesta di certificato digitale, utilizzando una coppia di chiavi precedentemente generata.

- **Generazione richiesta:** Per generare una richiesta di certificato digitale, si utilizza il comando seguente:

```
openssl req -new -key ecdsa_key.pem -out request.pem -config openssl.cnf
```

  Lo script avvia quindi una serie di richieste per raccogliere informazioni come il nome del paese (country name), stato (state), nome della località (locality name), organizzazione (organization), nome dell'unità (organizational unit), e il Common Name (CN), che rappresenta il nome del dominio per il quale il certificato sarà emesso. Il risultato di questo processo è un file che contiene la richiesta di certificato.

- Per visualizzare il contenuto della richiesta di certificato, si può eseguire il comando:

```
openssl req -in request.pem -text
```

  Nota: `openssl.cnf` è un file di configurazione di default fornito da OpenSSL, ma è possibile utilizzare un file di configurazione personalizzato.

Organizzazione del certificato Quando si crea una richiesta di certificato, la configurazione può essere definita tramite un file specifico, come nel caso di `myconfig.cnf`. Ecco un esempio di come potrebbe essere strutturato questo file di configurazione:

```
[req]
prompt = no
distinguished_name = dn
req_extensions = ext
input_password = 12345678

[dn]
CN = www.aps.com
emailAddress = visconti@unisa.it
O = UNISA
L = Fisciano
C = IT

[ext]
subjectAltName = DNS:www.aps.com,DNS:aps.com,DNS:*.aps.com
```

Con il file di configurazione `myconfig.cnf`, il comando per generare la richiesta di certificato diventa:

```
openssl req -new -key ecdsa_key.pem -out request.pem -config myconfig.cnf
```

In questo caso, non sarà necessario inserire manualmente le informazioni richieste durante la generazione della richiesta, poiché sono già preconfigurate nel file `myconfig.cnf`. Inoltre, il file di configurazione specifica le estensioni del certificato, come ad esempio i nomi alternativi del soggetto (subject alternative names), che permettono di associare più nomi di dominio al certificato.
Nota: tutte queste operazioni vengono eseguite dall'utente sul proprio computer, che utilizza la propria configurazione locale. Successivamente, la richiesta di certificato viene inviata alla CA (Certification Authority), che utilizza un sistema diverso con una propria configurazione per emettere il certificato finale.

9.5.9 CA - playground

Le **Certification Authorities (CA)** possiedono esse stesse un certificato digitale, che può essere emesso da un'altra CA, a meno che non si tratti di una root CA. Una root CA è una CA principale che firma i propri certificati. È possibile creare un certificato autofirmato utilizzando il comando:

```
openssl req -new -x509 -days 365 -key ecdsa_key.pem -out cacert.pem -config openssl
    .cnf //certificato autofirmato
```

Questo comando genera un certificato autofirmato valido per 365 giorni. Come nei passaggi precedenti, verranno richieste informazioni come nome, stato, località, e altri dettagli identificativi. Con questo comando, abbiamo creato un certificato per una root CA. Per esplorare il certificato appena creato, possiamo eseguire:

```
openssl x509 -in cacert.pem -text
```

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            10:fb:39:37:5d:33:61:0a:da:bb:0e:6b:20:17:38:18:27:61:db:62
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: C = IT, ST = Lazio, L = Roma, O = Sapienza, OU = CASapienza, CN = great
        Validity
            Not Before: Apr  4 08:45:13 2025 GMT
            Not After : Apr  4 08:45:13 2026 GMT
        Subject: C = IT, ST = Lazio, L = Roma, O = Sapienza, OU = CASapienza, CN = great
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:48:fd:5d:fb:28:c4:10:11:01:7a:57:02:f9:5e:
                    91:a7:b6:35:64:5d:4e:33:5d:cf:da:aa:12:b6:9b:
                    f0:51:1b:9f:90:85:35:44:b8:c2:2b:14:d3:bb:5e:
                    6f:d7:76:64:ae:7b:3c:f6:be:e3:8a:f2:06:cd:08:
```

Figura 14: Esempio di certificato

Preparazione della struttura della CA Per poter firmare i certificati, è necessario organizzare alcuni file e directory. Creiamo una configurazione per la CA, strutturando il nostro ambiente come segue:

```
mkdir demoCA //directory che contiene la configurazione della CA
mkdir demoCA/private //directory con chiave privata
mkdir demoCA/newcerts //directory con i certificati emessi
touch demoCA/index.txt demoCA/serial //inizializzo il database dei certificati
    emessi
```

La directory `demoCA/private` conterrà la chiave privata della CA, mentre `demoCA/newcerts` conterrà i certificati emessi. Inoltre, il file `demoCA/serial` tiene traccia dei numeri seriali dei certificati emessi dalla CA. Per inizializzare il numero seriale, eseguiamo il comando:

```
echo "00" > demoCA/serial  //Inizializza il contatore dei certificati a 00
```

Successivamente, copiamo il certificato della CA e la chiave pubblica nella directory di lavoro:

```
cp cacert.pem demoCA  // Copia il certificato autofirmato nella directory demoCA
cp ecdsapub_key.pem demoCA/private  // Copia la chiave privata nella directory
    demoCA/private
```

Per firmare le richieste di certificato, è necessario utilizzare la chiave privata della CA. Poiché le richieste di certificato sono firmate dalla CA, la **chiave segreta viene usata per firmare la richiesta**, garantendo che la CA sia l'entità che emette il certificato.

9.5.10 OpenSSL – Rilascio di un certificato

Ora che abbiamo una richiesta di certificato e una Certification Authority (CA), possiamo procedere con il rilascio effettivo del certificato.

- **Rilascio del certificato:** Per rilasciare un certificato, utilizziamo il seguente comando:

```
openssl ca -in request.pem -out newcert.pem -policy policy_anything -config
    openssl.cnf
```

  Questo comando rilascia un certificato a partire dalla richiesta `request.pem` e lo salva come `newcert.pem`. La politica `policy_anything` viene utilizzata per bypassare eventuali vincoli impostati nel file di configurazione. In seguito, il sistema chiede conferma prima di creare il certificato e, se confermato, il certificato viene aggiunto al database della CA.
  Una volta rilasciato, il certificato può essere visualizzato per verificarne il contenuto. Ad esempio, il `issuer` (emittente) sarà la CA che ha firmato il certificato, mentre il `subject` (soggetto) sarà la persona o entità che ha richiesto il certificato. Inoltre, il certificato è emesso utilizzando curve ellittiche, come specificato nella richiesta.

- **Revoca dei certificati:** OpenSSL fornisce anche comandi per revocare i certificati emessi. Questo processo è cruciale per garantire che i certificati compromessi o non più validi vengano ritirati.

- **Estensioni nei certificati:** Le estensioni specificate nella richiesta di certificato vengono ignorate di default durante il rilascio. Se desideriamo includere queste estensioni, è necessario aggiungere l'opzione `-extfile` al comando, indicando un file di estensioni come nel seguente esempio:

```
openssl ca -in request.pem -out newcert.pem -policy policy_anything -config
    openssl.cnf -extfile extensions.cnf
```

  Il file `extensions.cnf` deve specificare le estensioni in formato corretto. Ecco un esempio di come configurare le estensioni nel file:

```
subjectAltName = DNS:*.aps.com, DNS:aps.com
```

  Alcune estensioni sono considerate **"critical"** (critiche), il che significa che devono essere riconosciute e rispettate da chi riceve il certificato. Un esempio comune di estensione critica è `basicConstraints` per una CA. Questa estensione dovrebbe essere impostata come segue per indicare che il certificato è utilizzato da una CA:

```
basicConstraints = critical, CA:true
```

9.5.11 Un web server con HTTPS

In questa sezione, vedremo come avviare un server web che supporta HTTPS, utilizzando un certificato digitale che creiamo noi stessi. Innanzitutto, **creiamo una chiave ECDSA** (Elliptic Curve Digital Signature Algorithm), come fatto in precedenza:

```
openssl genpkey -paramfile prime256v1.pem -out serverkey.pem
```

Successivamente, **estraiamo la chiave pubblica**:

```
openssl pkey -pubout -in serverkey.pem -out serverpubkey.pem
```

Per ottenere il certificato digitale, **generiamo una richiesta di certificato** (CSR) utilizzando il comando `openssl req`:

```
openssl req -new -key serverkey.pem -out requestserverpubkey.pem
```

Durante l'esecuzione di questo comando, verranno richiesti i dettagli relativi al certificato, come il `Common Name`, che deve corrispondere al nome del dominio (ad esempio, `www.sicurezzadiag.it`).

Aggiunta delle estensioni al certificato Possiamo arricchire il certificato con estensioni. Per esempio, vogliamo aggiungere il dominio wildcard `*.sicurezzadiag.it` come subject alternative name (SAN). Creiamo un file `extensions.cnf` con il seguente contenuto:

```
subjectAltName = DNS:*.sicurezzadiag.it, DNS:sicurezzadiag.it
```

A questo punto, utilizziamo il comando di rilascio del certificato, aggiungendo il flag `-extfile` per includere il file di estensioni:

```
openssl ca -in request.pem -out newcert.pem -policy policy_anything -config openssl
    .cnf -extfile extensions.cnf
```

Con l'opzione `-extfile extensions.cnf`, le estensioni definite nel file `extensions.cnf` vengono applicate al certificato. Ora, il certificato include il DNS `*.sicurezzadiag.it` come previsto.

Configurazione del server Apache con HTTPS Per configurare un server web classico come Apache con supporto HTTPS (TLS), bisogna individuare la configurazione del server, che su Ubuntu si trova generalmente in `/etc/apache2`. All'interno di questa directory, dovremo configurare il server per utilizzare i certificati e le chiavi generati.

- Per l'esercitazione, possiamo avviare un server web utilizzando il comando `openssl s_server`, che emula un server HTTPS. Eseguiamo il seguente comando per avviare il server sulla porta `8444`:

  ```
  openssl s_server -port 8444 -key ecdsaserver_key.pem -cert newcert.pem -CAfile
      cacert.pem -WWW
  ```

- L'opzione `-WWW` permette di testare il server come un web server HTTP, restituendo una pagina web di esempio. Senza questa opzione, il server risponderà solo alle richieste TLS, senza restituire contenuti HTTP visibili.
- Il parametro `-CAfile` è necessario perché, durante l'handshake TLS, il server invia anche la propria CA (Certification Authority), affinché il client possa verificare la firma del certificato.

Test del server con il browser Una volta avviato il server, possiamo accedere al sito web dal browser all'indirizzo `https://127.0.0.1:8444`.

1. All'inizio, il browser mostrerà un messaggio di avviso `NET::ERR_CERT_AUTHORITY_INVALID` poiché non riconosce la nostra CA. Poiché abbiamo creato noi stessi il certificato, possiamo continuare con il test.
2. Se visitiamo la pagina, possiamo notare che il certificato è firmato dalla CA non riconosciuta dal browser. In questo caso, possiamo importare la nostra CA nel browser per evitare l'errore. Nelle impostazioni del browser, possiamo aggiungere la nostra CA tra quelle riconosciute.
3. Tuttavia, c'è un altro problema: il certificato è emesso per il common name `www.sicurezzadiag.it`, ma stiamo accedendo al server tramite `127.0.0.1` e quindi il browser ci avverte di ciò. Per risolvere questo problema, possiamo configurare il DNS locale del nostro PC per associare l'indirizzo IP `127.0.0.1` a `www.sicurezzadiag.it`. Modificando il file `/etc/hosts` (su Linux), possiamo aggiungere la seguente riga:

```
127.0.0.1 www.sicurezzadiag.it
```

4. Ora, se accediamo a `https://www.sicurezzadiag.it:8444` nel browser, non riceviamo più errori.

Test del server con un file di esempio Per testare il server, possiamo richiedere un file dal server. Ad esempio, possiamo chiedere `https://www.sicurezzadiag.it:8444/test2.txt`. Il server risponderà con il contenuto del file `test2.txt`. Quando visualizziamo il certificato nel browser, vediamo che è valido, anche se non è stato importato inizialmente con l'installazione del browser, come accade per le CA più autorevoli (root).

Verifica del protocollo TLS Per verificare il tipo di connessione TLS utilizzato, possiamo vedere che il server sta utilizzando TLS 1.3 con cifratura `AES_128`.
È possibile scegliere le ciphersuites con l'opzione `-ciphersuites`, ad esempio:

```
openssl s_server -port 8444 -ciphersuites TLS_AES_128_GCM_SHA256:
    TLS_CHACHA20_POLY1305_SHA256
```

Per elencare le ciphersuites disponibili, usare il comando:

```
openssl ciphers -v
```

Test con OpenSSL Client Oltre ad accedere al server tramite il browser, possiamo testare la connessione utilizzando il terminale. Per farlo, possiamo riavviare il server senza l'opzione `-WWW` e lanciare un altro terminale per utilizzare OpenSSL come client:

```
openssl s_client -connect www.sicurezzadiag.it:8444
```

Questo comando avvia una connessione client TLS al server. Durante l'handshake, possiamo osservare i protocolli e le chiavi condivise tra client e server, permettendoci di analizzare il processo di negoziazione TLS.

9.5.12 Sicurezza e integrità dei Messaggi

In tema di sicurezza, è importante considerare le seguenti minacce e contromisure:

- Il *replay attack* si verifica quando un attaccante intercetta e riutilizza una comunicazione valida (come un messaggio o una transazione) per ottenere un risultato non autorizzato. Questo tipo di attacco può essere mitigato tramite l'uso di tecniche di sincronizzazione e *timestamp*, che rendono ogni comunicazione unica e non riutilizzabile. I *timestamp* sono informazioni temporali aggiunte ai messaggi per garantirne l'unicità e la validità nel tempo.
- I *timing attacks* sfruttano le variazioni nei tempi di risposta di un sistema per ottenere informazioni sensibili. Ad esempio, un attaccante può eseguire operazioni ripetute per determinare la lunghezza di una chiave segreta o altre informazioni critiche. In alcuni casi, attacchi di questo tipo possono essere condotti "un byte alla volta", cioè misurando il tempo impiegato per elaborare ciascun byte di una chiave. Un esempio noto di questo tipo di attacco è quello sfruttato in sistemi come `XBox`, dove è stato possibile dedurre segreti crittografici attraverso l'analisi dei tempi di risposta.

10 Timing Attack e Side Channel Attacks

10.1 Introduzione ai Side Channel Attacks

I Side Channel Attacks sono attacchi che vanno oltre il modello teorico di sicurezza utilizzato nella crittografia. Mentre il modello teorico considera solo le comunicazioni sul canale principale (mittente, ricevente, avversario), i side channel attacks sfruttano informazioni fisiche o ambientali che possono rivelare dati sensibili.
Alcuni esempi di side channel attacks:

- **Temperature attacks**: misurazione delle variazioni di temperatura durante l'esecuzione di operazioni crittografiche
- **Power analysis**: misurazione del consumo energetico durante l'esecuzione
- **Timing attacks**: misurazione del tempo richiesto per eseguire operazioni di sicurezza

Questi attacchi funzionano perché l'implementazione fisica di un algoritmo può rivelare informazioni non considerate nel modello astratto.

10.2 Timing attacks

Un timing attack è un tipo specifico di side channel attack che sfrutta la misurazione del tempo impiegato per eseguire operazioni crittografiche. La premessa fondamentale è che:

> *Il tempo richiesto per eseguire un'operazione può dipendere dal valore dei dati segreti su cui si opera.*

In contesti reali, questa dipendenza temporale può rivelare informazioni critiche come chiavi crittografiche o password.

10.2.1 Esempio storico: l'attacco alla Xbox

Un esempio significativo di timing attack è stato quello condotto con successo contro la console Xbox. Il sistema di verifica per impedire l'esecuzione di giochi non autorizzati utilizzava una verifica di MAC (Message Authentication Code). La vulnerabilità nel codice di verifica ha permesso agli attaccanti di determinare il valore della chiave segreta usata per calcolare il MAC, aggirando così le protezioni.

10.2.2 Vulnerabilità nelle implementazioni di MAC

Consideriamo questa vulnerabilità comune nell'implementazione della verifica di un MAC. Consideriamo il seguente pseudocodice:

```
MAC_VER(K, m, o) {
    o1 = MAC(K, m)
    return o1 == o
}
```

Questa implementazione potrebbe sembrare corretta, ma quando implementata direttamente in linguaggio C usando funzioni come `strcmp()`, introduce vulnerabilità di timing.
Vediamo l'implementazione vulnerabile in C:

```
int mac_verify(const char *k, const char *m, const char *sigma) {
    char *t = MAC(k, m);
    return (strcmp(s, t) == 0);
}
```

Il problema principale è che `strcmp()` termina non appena trova una differenza tra le due stringhe (ricordando che la stringa termina con \0). Questo comportamento crea una vulnerabilità di timing: il tempo di esecuzione dipende dal numero di caratteri uguali dall'inizio delle due stringhe.

Come viene sfruttata la vulnerabilità Per chiarire come questa vulnerabilità possa essere sfruttata:

- Se il primo carattere del tentativo è sbagliato, la funzione termina immediatamente
- Se il primo carattere è corretto ma il secondo è sbagliato, la funzione esegue un'iterazione in più
- Se i primi due caratteri sono corretti ma il terzo è sbagliato, la funzione esegue due iterazioni in più
- E così via...

Un attaccante può sfruttare questa differenza nei tempi di esecuzione per dedurre la stringa segreta carattere per carattere:

1. Per ogni posizione, prova tutti i possibili caratteri
2. Misura il tempo di esecuzione per ciascun tentativo (ripetendo la misurazione più volte per ottenere un dato statisticamente significativo)
3. Il carattere che causa il tempo di esecuzione più lungo è probabilmente quello corretto per quella posizione
4. Congela il carattere scoperto e ripete l'attacco sulla posizione successiva

Questa tecnica, ripetuta sistematicamente, può rivelare progressivamente l'intera stringa segreta, anche se l'attaccante non può vedere direttamente il contenuto della memoria.

10.2.3 Analisi dettagliata della vulnerabilità

Analizziamo diversi problemi dell'implementazione:

1. **Problema con `strcmp()`**: Non è adatto per confrontare dati sensibili perché termina al primo carattere diverso
2. **Problema con `strncmp()`**: Anche limitando la lunghezza del confronto, il comportamento di uscita anticipata rimane
3. **Comparazione sicura**: È necessario un confronto a tempo costante, indipendente dai valori confrontati

Possibile soluzione (non corretta) Una versione corretta ma comunque vulnerabile potrebbe essere:

```
int mac_verify(const char *a, const char *b) {
    for (int i = 0; i < 16; i++) {
        if (a[i] != b[i])
            return 0;
    }
    return 1;
}
```

Questa implementazione è ancora vulnerabile perché il tempo di esecuzione dipende dalla posizione della prima differenza tra `a` e `b`.

10.2.4 Implementazione sicura a tempo costante

Una implementazione sicura deve garantire che il tempo di esecuzione sia indipendente dai valori dei dati:

```c
int secure_mac_verify(const char *a, const char *b) {
    int result = 1;
    for (int i = 0; i < 16; i++) {
        if (a[i] != b[i])
            result = 0;
        // Continuiamo il loop anche dopo aver trovato una differenza (non c'e' il return)
    }
    return result;
}
```

In questa implementazione, il loop viene sempre eseguito completamente, indipendentemente dai valori di input, garantendo un tempo di esecuzione costante. In particolare:

- **Nessuna uscita anticipata:** Non essendoci un return all'interno del ciclo, il tempo di esecuzione non dipende dalla posizione della prima differenza.
- **Utilizzo di una variabile accumulatore:** Invece di uscire subito, la funzione salva l'informazione di non uguaglianza nella variabile result, ma continua comunque l'elaborazione.

10.3 Dimostrazione pratica di un Timing Attack

10.3.1 Setup dell'esperimento

Per dimostrare l'efficacia di un timing attack, è stato analizzato il seguente codice che simula un attacco a una funzione di verifica di password:

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <time.h>

#define KEY "sicurezza"
#define LEN 16
#define DEL 4

int mac_verify(const char *a, const char *b) {
    while (*a && *b) {
        if (*a != *b)
            return 0;
        a++; b++;
        usleep(DEL);  // Timing leak!
    }
    return (*a == *b);
}

long time_verify(const char *guess) {
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    mac_verify(guess, KEY);

    clock_gettime(CLOCK_MONOTONIC, &end);
    long duration_us = (end.tv_sec - start.tv_sec) * 1000000000L
                     + (end.tv_nsec - start.tv_nsec);
    return duration_us;
}

int main() {
    char symbols[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    char solution[LEN];
    int solution_length = strlen(KEY);

    printf("Attack!!!\n");

    for (int j = 0; j < solution_length; j++) {
        long mytime = 0;
        char mychar = 0;
```

```
42
43          for (int i = 0; i < strlen(symbols); i++) {
44              solution[j] = symbols[i];
45              solution[j + 1] = '\0';
46
47              long ttime = 0;
48              for (int k = 0; k < 3; k++) {
49                  ttime += time_verify(solution);
50              }
51
52              if (ttime > mytime) {
53                  mytime = ttime;
54                  mychar = symbols[i];
55              }
56          }
57
58          solution[j] = mychar;
59          printf("One more! %s\n", solution);
60      }
61
62      printf("My guess: %s\n", solution);
63      return 0;
64  }
```

10.3.2 Spiegazione del codice

Il codice implementa un attacco di tipo timing per indovinare una password (la stringa `"sicurezza"`) carattere per carattere:

1. La funzione `mac_verify` confronta due stringhe e ha un'esplicita fuga di informazioni temporali (`usleep(DEL)`) che amplifica la vulnerabilità di timing.
2. La funzione `time_verify` misura il tempo necessario per verificare un tentativo di password.
3. Nel `main()`, l'attacco procede sequenzialmente:
   - Per ogni posizione nella password (da 0 a `solution_length-1`)
   - Prova tutti i caratteri possibili in quella posizione
   - Misura il tempo di verifica per ciascun tentativo
   - Seleziona il carattere che ha causato il tempo di verifica più lungo
   - Procede alla posizione successiva

10.3.3 Principio dell'attacco

Il principio alla base dell'attacco è che, quando il carattere tentato è corretto, la funzione `mac_verify` procede oltre quella posizione prima di fallire, causando un tempo di esecuzione leggermente più lungo. Questo perché:

- Se il primo carattere è sbagliato, la funzione termina immediatamente
- Se il primo carattere è corretto, ma il secondo è sbagliato, la funzione fa un'iterazione in più
- E così via...

Questo attacco riesce perché la funzione `mac_verify` ha un tempo di esecuzione che dipende dai valori di input, specificamente dal numero di caratteri corretti all'inizio della stringa tentata.

10.3.4 Risultati dell'esperimento

Durante la lezione, il codice è stato eseguito modificando il parametro `k`, che rappresenta il numero di ripetizioni della misurazione per ogni carattere testato:

- Per ogni posizione della password, l'algoritmo prova tutti i caratteri possibili (a-z, A-Z, 0-9)
- Per ciascun carattere candidato, esegue la funzione `mac_verify()` k volte
- Calcola la media dei tempi di esecuzione di queste `k` misurazioni
- Il carattere che produce il tempo medio di esecuzione più lungo è considerato corretto (poiché causa più iterazioni nella funzione di verifica)

I risultati hanno mostrato che:

- Con poche ripetizioni della misurazione (`k=3`), l'attacco ha avuto successo parziale, indovinando correttamente solo i primi caratteri.
- Aumentando le ripetizioni della misurazione (`k=300`), l'accuratezza è migliorata significativamente, permettendo di **recuperare l'intera password**.

Questo dimostra un principio importante: i tempi di esecuzione possono variare a causa di fattori esterni (altri processi in esecuzione, interrupt del sistema, ecc.). Ripetendo la misurazione molte volte e calcolando la media, è possibile filtrare questo "rumore" e identificare le piccole differenze di tempo che rivelano informazioni sulla password.

10.4 Contromisure contro i Timing Attack

Per proteggere un'applicazione dai timing attack, è necessario implementare funzioni a tempo costante per tutte le operazioni che coinvolgono dati sensibili:

1. **Evitare uscite anticipate**: Le funzioni non devono terminare prematuramente in base ai valori dei dati.
2. **Tempo costante**: Il tempo di esecuzione deve essere indipendente dai valori dei dati segreti.
3. **Comparazioni sicure**: Utilizzare funzioni specializzate per confrontare dati sensibili a tempo costante.

10.5 Considerazioni generali sui Side Channel Attacks

È importante sottolineare che i timing attack sono solo uno dei molti tipi di side channel attacks possibili. Durante lo sviluppo di sistemi di sicurezza, è necessario considerare:

- **Attacchi basati su temperatura**: Misurazione delle variazioni di temperatura durante operazioni crittografiche.
- **Analisi del consumo energetico**: Monitoraggio del consumo di energia durante l'esecuzione.
- **Attacchi acustici**: Analisi dei suoni prodotti durante le computazioni.
- **Attacchi elettromagnetici**: Misurazione delle radiazioni elettromagnetiche.

La protezione contro i side channel attacks è un aspetto fondamentale della sicurezza dei sistemi moderni, specialmente in dispositivi fisici come smart card, console di gioco e dispositivi embedded.

11 Anonimato su Internet: Onion Routing

11.1 Il problema dell'anonimato nelle connessioni

Quando si effettuano connessioni su Internet utilizzando i protocolli standard (TCP/IP), sorge un problema significativo di privacy:

- I pacchetti TCP/IP mostrano gli indirizzi IP di client e server
- L'indirizzo IP è spesso associabile all'identità reale dell'utente
- L'Internet Service Provider (ISP) può tracciare tutte le connessioni dell'utente

Questo problema si presenta anche quando si utilizzano protocolli di sicurezza come TLS, poiché questi proteggono il contenuto della comunicazione ma non nascondono gli indirizzi IP.

Ragioni per l'anonimato Ci sono diverse ragioni legittime per cui un utente potrebbe desiderare l'anonimato online:

- Protezione della privacy personale
- Necessità di sicurezza per dissidenti politici
- Protezione da profilazione commerciale

11.2 Onion Routing come soluzione

L'Onion Routing (routing a cipolla) è una tecnica che permette di mitigare il problema dell'anonimato, nascondendo la relazione tra il mittente e il destinatario del messaggio.

11.2.1 Principio di base

Il principio di base dell'Onion Routing è:

- Il client identifica N server $(S_1, S_2, \ldots, S_N)$ da utilizzare come intermediari
- Ogni server conosce solo il suo predecessore e il suo successore
- Solo l'ultimo server (S_1) sa chi è il destinatario finale del messaggio
- Nessun server singolo conosce sia il mittente che il destinatario

In questo modo, finché almeno uno dei server scelti è "onesto" (non collabora con altri per tracciare l'utente), l'anonimato è preservato.

11.3 Funzionamento dettagliato dell'Onion Routing

11.3.1 Fase di preparazione e invio del messaggio

Quando un utente vuole inviare un messaggio mantenendo l'anonimato:

1. Il mittente seleziona N server $(S_1, S_2, \ldots, S_N)$ da una lista pubblica di server Onion Routing
2. Ottiene le chiavi pubbliche di cifratura $(PK_1, PK_2, \ldots, PK_N)$ di questi server
3. Prepara un pacchetto IP P_1 contenente il messaggio m per il destinatario finale
4. Cifra P_1 con la chiave PK_1, ottenendo il cifrario c_1, e lo include in un pacchetto P_2 per S_1
5. Cifra P_2 con la chiave PK_2, ottenendo il cifrario c_2, e lo include in un pacchetto P_3 per S_2
6. Continua questo processo fino a ottenere un pacchetto P_{N+1} che include c_N per S_N
7. Invia P_{N+1} al suo ISP, che lo indirizzerà a S_N

Questo processo crea una struttura "a cipolla" con più strati di cifratura, da cui il nome "Onion Routing".

11.3.2 Fase di inoltro attraverso i server

Una volta che il pacchetto è stato inviato:

1. L'ISP consegna P_{N+1} al server S_N
2. S_N decifra c_N usando la sua chiave privata, ottenendo il pacchetto P_N destinato a S_{N-1}
3. S_N inoltra P_N a S_{N-1}
4. S_{N-1} decifra P_N ottenendo P_{N-1} per S_{N-2}
5. Questo processo continua fino a S_1, che decifra e ottiene il pacchetto originale P_1
6. S_1 inoltra P_1 al destinatario finale

Ogni server nel percorso ha una visione limitata della comunicazione:

- L'ISP vede solo che l'utente si connette a S_N
- Ogni server intermedio vede solo il server precedente e quello successivo
- Solo S_1 vede il destinatario finale, ma non conosce il mittente originale..

11.4 Gestione del percorso di ritorno

Un aspetto interessante dell'Onion Routing è come vengono gestite le risposte, dato che il destinatario finale non conosce l'indirizzo IP del mittente originale.

Scambio di chiavi tra nodi adiacenti Il sistema utilizza il protocollo Diffie-Hellman per stabilire canali cifrati tra ogni coppia di nodi adiacenti nel percorso:

1. Durante l'invio del messaggio, ogni nodo include il proprio contributo Diffie-Hellman per il nodo successivo
2. Ogni server nel percorso stabilisce una chiave segreta condivisa solo con il server precedente e il server successivo
3. Nel percorso di ritorno, un server (es. S_1) riceve dati dal destinatario finale, li cifra con la chiave DH condivisa con S_2 e li inoltra
4. Ogni server nel percorso inverso decifra con la chiave del nodo precedente e ricifra con la chiave del nodo successivo

Importante: lo scambio DH avviene sempre tra nodi adiacenti (a coppie), non direttamente tra il mittente originale e il destinatario finale. Questo approccio "a staffetta" permette comunicazioni bidirezionali mantenendo l'anonimato, poiché ogni nodo conosce solo i suoi vicini immediati.

11.5 TOR: The Onion Router

TOR (The Onion Router) è l'implementazione più nota dell'Onion Routing, sviluppata nei primi anni 2000.
Caratteristiche principali di TOR:

- È un software libero e open-source
- Utilizza una rete distribuita di server volontari
- È accessibile facilmente tramite browser dedicati o plugin
- Standardizza la dimensione dei pacchetti per prevenire analisi del traffico

11.6 Considerazioni sull'uso dell'Onion Routing

11.6.1 Vantaggi

- Fornisce un buon livello di anonimato
- Protegge dalla sorveglianza dell'ISP
- Utile per l'accesso a contenuti censurati

11.6.2 Limitazioni

- Maggiore latenza dovuta ai multipli passaggi
- Non adatto per streaming o applicazioni che richiedono bassa latenza
- Sicurezza compromessa se tutti i server nel percorso sono malevoli o controllati dalla stessa entità

L'Onion Routing offre un buon compromesso tra usabilità e anonimato per molte applicazioni come la navigazione web e la comunicazione testuale, ma richiede fiducia nel fatto che almeno uno dei server utilizzati nel percorso sia onesto.

12 Esempi di domande d'esame

12.1 Domanda 1: Pseudorandom Function

12.1.1 Testo della domanda

Sia F un (cryptographically strong) pseudorandom generator, e sia s una stringa random di 128 bit da usare come seed. Sia $PAD(j)$ una funzione che restituisce 128 bit che terminano con i bit di j, e che hanno in precedenza solo zeri.
Considerare la seguente funzione:

$$PRF(s, j) : F(s, 1^{128}) \;\; XOR \;\; PAD(j)$$

È una pseudorandom function? Commentare la risposta.

12.1.2 Analisi

Per rispondere a questa domanda, dobbiamo ricordare la definizione di pseudorandom function (PRF). Una PRF è una funzione che simula una funzione casuale da un dato spazio a un altro, in modo tale che un avversario con potenza di calcolo limitata non possa distinguerla da una vera funzione casuale, anche avendo accesso a query/risposte.

12.1.3 Risposta corretta

No, non è una pseudorandom function.

Giustificazione: Una vera pseudorandom function deve comportarsi come una tabella casuale: se conosci il valore di una cella, non dovresti riuscire a prevedere il valore di altre celle senza effettuare nuove query.
In questa funzione proposta invece:

1. Se un avversario ottiene $PRF(s, 0)$, ottiene $F(s, 1^{128})$ puro (perché $PAD(0)$ è tutto zero)
2. Conoscendo $F(s, 1^{128})$, può calcolare $PRF(s, j)$ per qualsiasi altro j semplicemente facendo $F(s, 1^{128}) \oplus PAD(j)$
3. Così può prevedere tutti i valori senza fare altre query

In pratica, facendo una sola query con $j = 0$, l'avversario può calcolare il valore di PRF per qualsiasi altro indice. Questo è completamente opposto al comportamento di una tabella casuale, dove conoscere un valore non ti dice nulla sugli altri valori.

Esempio concreto: Supponiamo che $F(s, 1^{128})$ restituisca il valore 10101010 (per semplicità, uso solo 8 bit).

Se l'avversario chiede il valore di $PRF(s, 0)$, ottiene:

$$\begin{aligned} PRF(s, 0) &= F(s, 1^{128}) \oplus PAD(0) \\ &= 10101010 \oplus 00000000 \\ &= 10101010 \end{aligned}$$

Ora l'avversario può calcolare qualsiasi altro valore senza fare altre query:

j	$PAD(j)$	Calcolo dell'avversario	$PRF(s, j)$
0	00000000	$10101010 \oplus 00000000 = 10101010$	10101010
1	00000001	$10101010 \oplus 00000001 = 10101011$	10101011
2	00000010	$10101010 \oplus 00000010 = 10101000$	10101000
3	00000011	$10101010 \oplus 00000011 = 10101001$	10101001
...	...	...	...

Mentre in una vera PRF, la tabella dovrebbe apparire completamente casuale:

j	Vera PRF(s,j)	
0	10101010	
1	01110101	← non prevedibile conoscendo PRF(s,0)
2	11000111	← non prevedibile conoscendo altri valori
3	00101100	← non prevedibile conoscendo altri valori
...	...	

In sintesi, una volta conosciuto $PRF(s, 0)$, che è uguale a $F(s, 1^{128})$, l'avversario può facilmente calcolare tutti gli altri valori della funzione semplicemente facendo XOR con il padding appropriato, senza bisogno di interrogare nuovamente l'oracolo.

12.2 Domanda 2: Chosen-Plaintext Attack (CPA)

12.2.1 Testo della domanda

Un avversario di chosen-plaintext attack (CPA) security ha accesso ad un oracolo di cifratura. Descrivere e commentare un attacco nel mondo reale che riesce se lo schema non gode di CPA security ed invece fallisce se lo schema gode di CPA security.

12.2.2 Analisi

Questa domanda richiede di descrivere un attacco reale che sfrutta la mancanza di CPA security. Ricordiamo che CPA security implica che un avversario non può distinguere tra le cifrature di due messaggi di sua scelta, anche se può richiedere la cifratura di altri messaggi a piacere.

12.2.3 Risposta corretta

Un attacco nel mondo reale che funziona contro schemi non CPA-sicuri ma fallisce contro schemi CPA-sicuri è il seguente:

1. In una fase preliminare, l'avversario ottiene cifrature di messaggi a sua scelta utilizzando l'oracolo di cifratura. Successivamente, quando intercetta una cifratura di un messaggio sconosciuto, può confrontarla con le cifrature precedentemente ottenute.

2. Se lo schema non è CPA-sicuro (ad esempio, se è deterministico), la stessa cifratura apparirà identica ogni volta che lo stesso messaggio viene cifrato. L'avversario può quindi identificare il messaggio sconosciuto confrontandolo con le cifrature note.

3. Se invece lo schema è CPA-sicuro (ad esempio, con randomizzazione come in CBC con vettore di inizializzazione casuale), anche lo stesso messaggio produrrà cifrature diverse ogni volta. L'avversario non può quindi stabilire alcuna correlazione tra cifrature precedenti e la nuova cifratura intercettata.

12.3 Domanda 3: Collision Resistance

12.3.1 Testo della domanda

Sia $H : Y \to Z$ una funzione hash. Se Z fosse l'insieme delle stringhe di 256 bit che hanno i primi 220 bit settati a ZERO, H può essere collision resistant? Commentare la risposta (senza dilungarsi e restando strettamente nel topic della domanda).

12.3.2 Analisi

Per questa domanda, dobbiamo considerare la definizione di collision resistance e valutare se la funzione hash descritta può soddisfarla data la struttura del suo codominio Z.

12.3.3 Risposta corretta

No, H non può essere collision resistant.

Giustificazione Lo spazio delle possibili stringhe in output (codominio Z) è limitato a 2^{36} valori possibili (256 bit totali meno 220 bit fissi a zero). Per il paradosso del compleanno ($\sqrt{2^{36}} = 2^{18}$), basteranno circa 2^{18} tentativi per trovare con buona probabilità una collisione.
Questo numero di tentativi è facilmente realizzabile anche con hardware modesto, mentre una funzione hash collision resistant richiede che nessun avversario del mondo reale possa trovare una collisione con risorse computazionali ragionevoli.

12.4 Domanda 4: Certificati HTTPS

12.4.1 Testo della domanda:

Se un server https fosse configurato per rispondere a connessioni www.miosito.it, quali informazioni speciali nel suo certificato digitale sono necessarie per evitare che il browser possa segnalare anomalie in fase di connessione? Commentare la risposta (senza dilungarsi e restando strettamente nel topic della domanda).

12.4.2 Analisi

La domanda riguarda i certificati digitali usati nelle connessioni HTTPS e le informazioni necessarie per evitare warning nei browser.

12.4.3 Risposta corretta

Le informazioni speciali necessarie nel certificato digitale sono:

1. Il certificato deve essere rilasciato da una Certification Authority (CA) affidabile, riconosciuta dal browser o che abbia a sua volta un certificato rilasciato da una CA riconosciuta.
2. Il certificato deve contenere nel campo "Common Name" o nell'estensione "Subject Alternative Name" (SAN) il nome di dominio esatto a cui risponde il server (www.miosito.it).

Queste informazioni attestano che il proprietario del certificato è autorizzato a operare sul dominio specificato, permettendo al browser di stabilire una connessione sicura senza segnalare anomalie.

12.5 Domanda 5: Comandi OpenSSL

12.5.1 Testo della domanda:

I seguenti comandi

```
openssl dgst -sign ecdsa_key.pem -out signature.bin test.txt
openssl dgst -verify ecdsapub_key.pem -signature signature.bin test.txt
```

sono utilizzabili per firmare il contenuto di un file e per verificarne la firma. Spiegare in cosa si differenziano gli argomenti dei due file con estensione .pem che sono passati a riga di comando.

12.5.2 Analisi

Questa domanda riguarda la differenza tra i due file .pem e il loro utilizzo per la firma digitale con OpenSSL.

12.5.3 Risposta corretta

La differenza tra i due file .pem è che:

- `ecdsa_key.pem` contiene la chiave privata necessaria per generare la firma digitale
- `ecdsapub_key.pem` contiene la chiave pubblica necessaria per verificare la firma

Quando si firma un documento, è necessaria la chiave privata per creare la firma, mentre quando si verifica una firma, è necessaria la chiave pubblica corrispondente per confermare che la firma sia stata generata con la chiave privata abbinata.

Glossario

AES	Algoritmo di cifratura simmetrica a blocchi, standard per la protezione dei dati. 25
AES-GCM	Modalità di AES che fornisce cifratura autenticata tramite Galois/Counter Mode. 21
asimmetrica	In crittografia, sistema che usa una coppia di chiavi: una pubblica per cifrare e una privata per decifrare. 6
Authenticated Encryption with Associated Data	Cifratura che garantisce riservatezza, integrità e autenticità del messaggio e dei dati associati. 21
block cipher	Algoritmo di cifratura che lavora su blocchi di dati di lunghezza fissa. 12, 13, 17, 37
CCA	Vedere *Chosen-Ciphertext Attack*. 36, 37
Chosen-Ciphertext Attack	Attacco in cui l'attaccante può scegliere ciphertext da decifrare e ottenere il plaintext. 10, 80
Chosen-Plaintext Attack	Attacco in cui l'attaccante può scegliere il plaintext da cifrare e osservare il ciphertext. 10, 80
Cipher Block Chaining	Modalità in cui ogni blocco cifrato dipende dal blocco precedente. 15
ciphertext	Testo cifrato, ovvero il messaggio dopo l'applicazione di un algoritmo crittografico. 8
Collision-Resistant Hash Functions	Funzioni hash in cui è difficile trovare due input distinti con lo stesso output. 21
CPA	Vedere *Chosen-Plaintext Attack*. 36
deterministico	Algoritmo che se ripetuto con gli stessi input produce sempre lo stesso output. 14
Diffie-Hellman	Protocollo di scambio chiavi che permette a due parti di stabilire una chiave segreta su un canale insicuro. 6
DSA	Tipo di firma digitale basata sul problema del logaritmo discreto. 56
ECB	vedere *Electronic Code Book*. 29
Electronic Code Book	Modalità di block cipher dove ogni blocco è cifrato indipendentemente. 14, 80
encrypt-then-mac	Schema in cui il MAC è calcolato sul ciphertext per garantire integrità e autenticità. 21
forward secrecy	Proprietà che garantisce che compromissioni future delle chiavi non rivelino comunicazioni passate. 49
HMAC	MAC costruito usando una funzione hash crittografica e una chiave segreta. 24, 30
Key Distribution Center	Entità fidata che distribuisce chiavi segrete agli utenti in una rete. 31

key exchange — Procedura con cui due parti stabiliscono una chiave segreta comune. 38

Known-Plaintext Attack — Attacco in cui l'attaccante conosce una o più coppie plaintext-ciphertext. 10

MAC — È una funzione utilizzata per verificare l'integrità e l'autenticità di un messaggio. 17

One-Time Pad — Sistema di cifratura teoricamente perfetto che usa una chiave casuale lunga quanto il messaggio. 8

one-way — Funzione che è facile da calcolare ma difficile da invertire senza informazioni segrete. 11, 34

oracolo casuale — Funzione teorica che restituisce output casuali ma consistenti per input uguali. 23

PBKDF2 — Funzione che deriva una chiave sicura da una password tramite iterazioni di hash. PBKDF2 è solo una funzione di derivazione di chiavi, non una cifratura o decifratura. 25

PRG — Vedere *pseudorandom generator*. 10

problema del logaritmo discreto — Dato $g^x \mod p$, trovare x; problema ritenuto computazionalmente difficile. 34

pseudorandom function — Funzioni calcolabili in modo efficiente, tali che nessun algoritmo efficiente possa distinguere (se non con vantaggio trascurabile) tra una funzione scelta casualmente dalla famiglia PRF e una vera funzione casuale.. 12

pseudorandom generator — Algoritmo che produce sequenze apparentemente casuali a partire da un seme segreto. 10, 81

qubit — Unità base dell'informazione quantistica, può rappresentare 0, 1 o una sovrapposizione di entrambi. 6

rainbow table — Dizionario precomputato di hash di password usato per attacchi di forza bruta. 24

RSA — Algoritmo di crittografia asimmetrica basato sulla difficoltà di fattorizzare grandi numeri primi. 6

salt — Stringa casuale aggiunta alla password per renderne unico l'hash. 24

sicurezza imperfetta — Sicurezza che si basa sulla difficoltà computazionale di attacchi, non garantita teoricamente. 9

sicurezza perfetta — Condizione in cui un crittosistema non rivela alcuna informazione sul plaintext. 9

simmetrica — In crittografia, sistema che usa la stessa chiave per cifrare e decifrare. 6

stateful — Cifratura che dipende dallo stato interno del generatore e dall'output generato fino a quel momento. 11

stateless — Cifratura che non dipende dallo stato interno o dall'output precedente. 12

stream cipher	Algoritmo di cifratura che opera bit o byte alla volta, spesso con PRG. 12, 13, 17
vulnerabilità	Debolezza in un sistema che può essere sfruttata per comprometterne la sicurezza. 7
X.509v3	Formato standard per certificati digitali usati in TLS/SSL. 47
XOR	Operazione logica che restituisce vero solo se i due bit sono diversi; usata spesso nella cifratura. 8